

LAYERZERO · SECURITY REVIEW

LayerZero

Canton VER Updates.

A security review of Canton VER Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
LayerZero	Canton VER Updates	Guardian	June 25 – August 27, 2026	September 3, 2026

§ 00

Contents.

304 pages — overview, confidence ranking, scope, 169 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 36 findings	P. 19
	Remediation Review 1 · 72 findings	P. 68
	Remediation Review 2 · 49 findings	P. 199
	Remediation Review 3 · 8 findings	P. 284
	Remediation Review 4 · 4 findings	P. 296
§ 05	Disclaimer	P. 303
§ 06	About Guardian	P. 304

§ 01

Overview.

Guardian reviewed Canton VER Updates across 5 review rounds. The final report includes 2 Critical, 12 High, 36 Medium, 54 Low, 65 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	LayerZero-Labs/audit-external
CLIENT	METHODOLOGY
LayerZero	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3 · Remediation Review 4
PROJECT	MAIN REVIEW COMMIT
Canton VER Updates	8d7008eb8ce1
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	dc77a706e26c
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
June 25 – August 27, 2026	d48e7725ff37
FINAL REPORT	REMEDATION REVIEW 3 COMMIT
September 3, 2026	ac6345e097cc

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	2	0	0	0	2
● High	12	0	0	0	12
● Medium	36	0	12	0	24
● Low	54	0	37	0	17
● Informational	65	0	52	0	13
All severities	169	0	101	0	68

§ 01 - OVERVIEWCONFIDENTIALP. 03 / 304

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Canton VER Updates in `LayerZero-Labs/audit-external`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Delayed callbacks over-credit rate limits	Unexpected Behavior	MEDIUM	RESOLVED	p. 19
L-01	Reject refund lacks on-ledger reissue	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 21
L-02	Time/minFee setters lack upper bound	Validation	LOW	ACKNOWLEDGED	p. 22
L-03	AdapterConfig is not canonical	Validation	LOW	ACKNOWLEDGED	p. 23
L-04	RequestFactory uses caller-selected fee config	Validation	LOW	ACKNOWLEDGED	p. 25
L-05	Registry allows caller-selected configs	Validation	LOW	ACKNOWLEDGED	p. 27
L-06	Compose payload silently dropped on receive	Validation	LOW	RESOLVED	p. 28
L-07	localDecimals overflow bricks transfers	Validation	LOW	RESOLVED	p. 29
L-08	Pending-transfer delivery can expire unclaimed	DoS	LOW	ACKNOWLEDGED	p. 30
L-09	Locked send funds have no sender timeout	Logical Error	LOW	ACKNOWLEDGED	p. 31
L-10	DiscoveryRegistry shared cap exhaustion	DoS	LOW	ACKNOWLEDGED	p. 32
L-11	Treasury fee floor bypass via negative payout	Validation	LOW	RESOLVED	p. 33

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-12	Caller-selected RBAC bypasses role revocation	Validation	LOW	ACKNOWLEDGED	p. 34
L-13	Rate-limit accounting overcredits capacity	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 36
I-01	Pending callbacks settle on wrong adapter	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 38
I-02	Rate limiter state is not canonical	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 39
I-03	No per-instance treasury accounting	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 41
I-04	RBAC id=None loses per-instance isolation	Access Control	INFORMATIONAL	RESOLVED	p. 42
I-05	Full role map disclosed to every role holder	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 43
I-06	slashProposalFee double-floors fee dust	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 44
I-07	DEFAULT_ADMIN_ROLE is an over-broad delegate	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 45
I-08	Inbound amount $\geq 2^{63}$ SD undeliverable	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 46
I-09	Governance ensure omits treasury-in-members	Validation	INFORMATIONAL	RESOLVED	p. 47
I-10	Untrusted token interfaces fake asset locks	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 48
I-11	Unauthenticated registry receiver swap	Access Control	INFORMATIONAL	RESOLVED	p. 50
I-12	Finalize TransferFactory can fake transfers	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 52
I-13	Governance fee accepts untrusted factory	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 54
I-14	Delegated burner can destroy locked holdings	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 56

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-15	Registry keyed by namespace, not full party	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 57
I-16	Allowlist granularity collapses to namespace	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 58
I-17	Allocation execute ignores settleBefore	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 59
I-18	refundAddress observer leaks payload	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 60
I-19	Finalize asset context can bypass token controls	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 61
I-20	Holding context bypasses token controls	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 63
I-21	Rate-limit zero limit blocks the direction	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 65
I-22	Pending refunds can be delayed	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 66

REMEDIATION REVIEW 1

H-01	Delegated generic requests can suppress inbound unlocks	Validation	HIGH	RESOLVED	p. 68
H-02	Mixed-case message prevents receive settlement	Validation	HIGH	RESOLVED	p. 70
H-03	Base-unit native fees poison commit batches	Unexpected Behavior	HIGH	RESOLVED	p. 72
H-04	Rate-limit timestamps break validator quorum	Logical Error	HIGH	RESOLVED	p. 74
H-05	Foreign commitment blocks settlement	Validation	HIGH	RESOLVED	p. 80
H-06	Expired fee escrow poisons commit batches	Unexpected Behavior	HIGH	RESOLVED	p. 83
H-07	Forged accept callback burns victim payout pool	Access Control	HIGH	RESOLVED	p. 86

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-08	Delegate forges callback to mint unbacked OFT	Access Control	HIGH	RESOLVED	p. 88
H-09	Revoked delegate retains endpoint authority	Access Control	HIGH	RESOLVED	p. 90
H-10	Prototype dispatch omits state commitments	DoS	HIGH	RESOLVED	p. 92
M-01	Payout fees consume cross-chain backing	Compatibility	MEDIUM	RESOLVED	p. 94
M-02	Amulet expiry can leave live claims unbacked	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 97
M-03	Validator build rejects Canton OApp requests	DoS	MEDIUM	RESOLVED	p. 101
M-04	Rejected calls persist orphaned trie nodes	DoS	MEDIUM	RESOLVED	p. 103
M-05	Underpriced fees enable rate-limit exhaustion	DoS	MEDIUM	ACKNOWLEDGED	p. 106
M-06	Appended ULNs can duplicate worker addresses	DoS	MEDIUM	RESOLVED	p. 108
M-07	OneSig CallContext leaf encoding non-injective	Signatures	MEDIUM	RESOLVED	p. 111
M-08	Stale Request time permits expired DVN actions	Validation	MEDIUM	ACKNOWLEDGED	p. 113
M-09	Future NIL entries enable unbounded scans	DoS	MEDIUM	RESOLVED	p. 115
M-10	Unbounded Event Scans Can Exhaust Validators	DoS	MEDIUM	RESOLVED	p. 120
M-11	Uncapped signature array enables node DoS	DoS	MEDIUM	ACKNOWLEDGED	p. 121
L-01	Rate limiter mode toggle ignores active usage	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 122
L-02	Arbitrum price updates overwrite other EIDs	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 123

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-03	Foreign Requests trigger an empty batch loop	Validation	LOW	ACKNOWLEDGED	p. 124
L-04	Unused fee recipient blocks fee-free OFT sends	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 126
L-05	Old DVN approvals can undo later decisions	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 127
L-06	Unbounded cost multipliers brick sends	DoS	LOW	ACKNOWLEDGED	p. 129
L-07	Non-canonical config inflates OFT mint	Validation	LOW	ACKNOWLEDGED	p. 130
L-08	Payout settlement skips local-transfers pause	Validation	LOW	ACKNOWLEDGED	p. 131
L-09	Container slots unpinned; fork resets state	Upgradeability	LOW	ACKNOWLEDGED	p. 133
L-10	Null recipient send burns OFT principal	Validation	LOW	ACKNOWLEDGED	p. 134
L-11	Reject leg unlocks payout pool unbound	Access Control	LOW	RESOLVED	p. 135
L-12	Pending outflow decay leaves phantom rate usage	Logical Error	LOW	ACKNOWLEDGED	p. 136
L-13	Stale prices undercharge executor fees	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 137
L-14	ULN send support ignores unregistered DVNs	Validation	LOW	ACKNOWLEDGED	p. 139
L-15	DVN ACL/msg-lib methods omit admin gate	Access Control	LOW	ACKNOWLEDGED	p. 141
L-16	Whitespace-Only Authentication Passes Validation	Validation	LOW	RESOLVED	p. 143
L-17	Unprefixed Hex Bypasses Fixed-Width Checks	Warning	LOW	RESOLVED	p. 144
L-18	Malformed Hex Inputs Are Silently Truncated	Warning	LOW	RESOLVED	p. 145

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-19	DVN accepts off-curve (unsignable) signer key	Validation	LOW	RESOLVED	p. 146
L-20	Packet codec fails open on malformed input	Validation	LOW	RESOLVED	p. 147
L-21	Quadratic DVN-options grouping DoS	DoS	LOW	RESOLVED	p. 148
L-22	Treasury native-fee ceiling is inert at genesis	Configuration	LOW	ACKNOWLEDGED	p. 149
L-23	Duplicate Keys Make Quorum Unreachable	Validation	LOW	RESOLVED	p. 151
L-24	Executor SDK Does Not Support LockUnlockAdapter	Suggestion	LOW	RESOLVED	p. 152
L-25	Sequencer quorum body not bound to agreed hash	DoS	LOW	ACKNOWLEDGED	p. 153
I-01	OApp ID collision consumes adapter settlements	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 155
I-02	Owable2Step corrupts inherited state	Upgradeability	INFORMATIONAL	ACKNOWLEDGED	p. 157
I-03	Receive caller may gain settlement veto	Warning	INFORMATIONAL	RESOLVED	p. 159
I-04	Registered Simple library permits packet forgery	Warning	INFORMATIONAL	RESOLVED	p. 162
I-05	OFT exposes base units as whole CIP-0056 tokens	Compatibility	INFORMATIONAL	RESOLVED	p. 164
I-06	StateCommitment accepts non-bytes32 state roots	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 166
I-07	Forkable state-commitment genesis equivocation	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 167
I-08	Missing cost config skips options validation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 169
I-09	Options value-cost rounds down, undercharging	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 170

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Config and state contracts over-disclose data	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 171
I-11	RATE_LIMITER_MANAGER can nullify rate limit	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 172
I-12	Config create skips transfer-rule check	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 173
I-13	Options decoder overflows on large native values	Error	INFORMATIONAL	ACKNOWLEDGED	p. 174
I-14	Default DVN Configs Accept Unregistered DVNs	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 176
I-15	Default Config Accepts Unregistered Executors	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 177
I-16	Worker defaultMultiplierBps unbounded (uint16)	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 178
I-17	Bearer tokens never expire, audience unenforced	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 180
I-18	Read endpoint signs reads at caller stale root	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 181
I-19	Free preapprovals retain receiver fee inputs	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 182
I-20	Compose sends burn assets Canton cannot receive	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 184
I-21	Small outflows erase larger rate-limit usage	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 186
I-22	Rejected writes shorten library grace periods	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 188
I-23	No on-ledger inbound replay / nonce / guid dedup	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 190
I-24	Executor fees ignore native-drop fanout	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 192
I-25	Endpoint IDs not bounded to uint32 domain	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 195

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-26	Pending outflows use the wrong decay parameters	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 197
REMEDATION REVIEW 2					
C-01	Head race commits a stale-derived root	Unexpected Behavior	CRITICAL	RESOLVED	p. 199
C-02	Forged callback bypasses gateway auth, mints OFT	Access Control	CRITICAL	RESOLVED	p. 201
H-01	Stale admin nominees can take over the adapter	Validation	HIGH	RESOLVED	p. 203
H-02	Default-admin assignee can seize sibling OApp	Validation	HIGH	RESOLVED	p. 205
M-01	Payout floor withholds charged fees	Warning	MEDIUM	RESOLVED	p. 207
M-02	Archived requests block recovery	Unexpected Behavior	MEDIUM	RESOLVED	p. 209
M-03	Foreign config bricks context discovery	DoS	MEDIUM	RESOLVED	p. 211
M-04	Sybil OApps can fill discovery registry	DoS	MEDIUM	ACKNOWLEDGED	p. 213
M-05	Zero-amount rejects strand adapter payouts	Unexpected Behavior	MEDIUM	RESOLVED	p. 215
M-06	Limiter disable strands pending transfers	Unexpected Behavior	MEDIUM	RESOLVED	p. 217
M-07	Direct accept bypasses admin transfer lifecycle	Logical Error	MEDIUM	RESOLVED	p. 219
M-08	Factory rotation hides pending requests	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 221
M-09	Losing batch pollutes canonical scan data	Unexpected Behavior	MEDIUM	RESOLVED	p. 222
M-10	Pending offers erase adapter obligations	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 224

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-11	One validator blocks sequencer startup	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 226
M-12	Internal faults become final request rejects	Unexpected Behavior	MEDIUM	RESOLVED	p. 227
M-13	Executor SDK cannot settle adapter callbacks	Logical Error	MEDIUM	RESOLVED	p. 229
M-14	DVN approvals replay across deployments	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 231
M-15	OneSig cannot run default-admin transfers	Logical Error	MEDIUM	RESOLVED	p. 233
M-16	Poison AccessControl bricks OApp administration	DoS	MEDIUM	RESOLVED	p. 235
M-17	Admin API sends OAuth tokens over plaintext	Authentication & Session flaws	MEDIUM	RESOLVED	p. 237
M-18	Factory offset seeding strands active requests	DoS	MEDIUM	ACKNOWLEDGED	p. 239
M-19	Initial migration bricks existing Postgres DBs	DoS	MEDIUM	ACKNOWLEDGED	p. 241
M-20	Observer config blocks Canton request signing	DoS	MEDIUM	RESOLVED	p. 242
M-21	Postgres TLS disables server authentication	Trust Assumptions	MEDIUM	ACKNOWLEDGED	p. 244
M-22	Local state roots split ordinary read quorum	DoS	MEDIUM	RESOLVED	p. 246
L-01	Pending callbacks leak ledger data	Warning	LOW	ACKNOWLEDGED	p. 248
L-02	OFT supply excludes hidden holdings	Warning	LOW	ACKNOWLEDGED	p. 249
L-03	Pagination follows full ledger scans	Warning	LOW	ACKNOWLEDGED	p. 250
L-04	Backdated sends preload rate-limit decay	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 251

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-05	Cached synchronizer can halt settlement	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 253
L-06	Plaintext JSON API can forge settlements	Best Practices	LOW	RESOLVED	p. 255
L-07	HTTP key bootstrap lets MITM halt settlement	Best Practices	LOW	RESOLVED	p. 257
L-08	Revoked libraries still receive valid quotes	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 259
L-09	Same-millisecond batches share command identity	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 260
L-10	Old signed responses can pass as current	Validation	LOW	ACKNOWLEDGED	p. 262
L-11	Admin transfer leaves old delegate active	Unexpected Behavior	LOW	RESOLVED	p. 264
I-01	Gateway can forge settlement callbacks	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 266
I-02	Non-443 admin gRPC sends plaintext tokens	Best Practices	INFORMATIONAL	RESOLVED	p. 267
I-03	Callbacks can settle in the wrong asset	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 268
I-04	Plaintext wallet API can halt settlement	Best Practices	INFORMATIONAL	RESOLVED	p. 270
I-05	Random signatures inflate validator quorum	Validation	INFORMATIONAL	RESOLVED	p. 272
I-06	Atomic DVN admin replacement is unreachable	Best Practices	INFORMATIONAL	RESOLVED	p. 274
I-07	Ledger offsets lose precision in JavaScript	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 275
I-08	Hard-fork recovery omits required parent	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 276
I-09	Issuer filters break transfer context API	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 277

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Removing Simple ML blocks the vApp hard fork	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 279
I-11	Completed transfers can short-pay recipients	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 281
I-12	Losing validators keep buffering responses	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 283

REMEDIATION REVIEW 3

L-01	Rule rotation blocks callback finalization	Unexpected Behavior	LOW	RESOLVED	p. 284
L-02	Nonpositive poll settings stall validators	Validation	LOW	ACKNOWLEDGED	p. 286
L-03	Interface registries break finalization	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 288
L-04	Transfer context drops the offer-bound rule	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 290
L-05	Sequencer SDK sends bearer tokens over HTTP	Validation	LOW	RESOLVED	p. 291
I-01	Rights RPC targets the participant admin API	Warning	INFORMATIONAL	RESOLVED	p. 293
I-02	Request poller fans out before batch cap	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 294
I-03	Cached package IDs block DAR switchovers	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 295

REMEDIATION REVIEW 4

M-01	Dense request ranges halt validator polling	DoS	MEDIUM	RESOLVED	p. 296
M-02	Zero peers strand outbound transfers	Validation	MEDIUM	RESOLVED	p. 298
I-01	Recovery buffers the full missed chain	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 300

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-02	Uppercase peers block inbound messages	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 302

MEDIUM **M-01**

RESOLVED

Delayed callbacks over-credit rate limits

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml
REVIEW	Main Review

DESCRIPTION

`commitOutflow` and `reverseOutflow` both decay the current aggregate bucket and then subtract the original `scaledAmount` recorded by an earlier send. The state stores only aggregate `outboundUsage`, aggregate `inboundUsage` and one `lastUpdated` timestamp per bucket. It does not track the remaining contribution of each pending send.

`handleAdapterLzSendAccept` passes the frozen `scaledAmount` to `IRateLimitState_CommitOutflow`, which subtracts it from the current inbound bucket when net accounting is enabled.

`handleAdapterLzSendReject` passes the same frozen amount to `IRateLimitState_ReverseOutflow`, which subtracts it from the current outbound bucket when the original outbound usage was recorded.

If a callback settles after the original contribution has already decayed or after later traffic has updated the same bucket, subtracting the full original amount removes usage that belongs to later transfers. For rejects, an old rejected send can erase outbound usage from newer sends. For accepts, an old accepted send can erase inbound usage from newer receives. In both cases, another transfer can pass immediately even though recent traffic should still consume the configured bucket.

This does not require forged callback data. A normal pending send that is accepted or rejected late can over-credit rate-limit capacity because the callback settlement is applied to aggregate state instead of the pending send's remaining decayed contribution.

Simplified example with an outbound limit of 1000 and a 100-second decay window:

- At `t = 0`, Alice sends 1000. The bucket becomes full: `outboundUsage = 1000`.
- Alice's send remains pending, meaning the later accept/reject callback has not happened yet.
- At `t = 100`, Alice's old 1000 usage has fully decayed. The bucket is now `outboundUsage = 0`.
- Bob now sends 500. The bucket should represent Bob's recent traffic: `outboundUsage = 500`.
- Alice's old send is then rejected.

M-01 Delayed callbacks over-credit rate limits

CONTINUED

- To undo Alice's old send, `reverseOutflow` subtracts Alice's original recorded amount, `scaledAmount = 1000`, from the current bucket.
- But the current bucket no longer contains Alice's usage; it contains Bob's newer 500 usage.
- The calculation becomes `max(0, 500 - 1000) = 0`, so Bob's recent usage is erased.
- The system now thinks the bucket is empty and allows another full 1000 transfer immediately. Correct behavior would leave Bob's `outboundUsage = 500`, so only 500 capacity should be available.

— RECOMMENDATION

Do not subtract the full original `scaledAmount` from the current aggregate bucket during callback settlement. Store enough per-pending-outflow data to compute the settling send's remaining decayed contribution, then subtract only that remaining amount.

If the rate limiter must stay aggregate-only, make delayed callbacks non-adjusting once their original contribution has already decayed out of the bucket. Another safe design is to track pending outflows by request id and recompute aggregate usage from active entries when settling accept or reject.

— RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

Reject refund lacks on-ledger reissue

CATEGORY Unexpected Behavior
LOCATION Reject.daml:47-77
REVIEW Main Review

— DESCRIPTION

On an outbound-send reject, `handleAdapterLzSendReject` refunds `crossChainAmount +  $\Sigma$  payouts` to the original sender (lines 47-64) by proposing a pending `TransferInstruction` from `treasury` for `config.assetInstrumentId` with the standard `_TRANSFER_INSTRUCTION_EXPIRY`. It then reverses the outflow recorded at call time via `IRateLimitState_ReverseOutflow` at line 72. The refund is a pending instruction the sender must accept.

If the sender does not accept before the instruction expires, the locked holdings return to `treasury`, and the reject handler — already finalized and archived — leaves no on-ledger choice to re-propose the refund. Value is conserved (it rests in `treasury`, recoverable by the operator off-ledger using `config.localDecimals` accounting), but the sender has no on-ledger path to reclaim it while rate-limiter capacity was already freed at reject time. The window is the standard Splice transfer expiry.

— RECOMMENDATION

Provide an on-ledger reissue path for a reject refund whose `TransferInstruction` expires unaccepted. For example, retain or recreate a contract that lets the original sender re-propose the transfer after expiry. A sender who misses the acceptance window could then reclaim the refunded amount on-ledger, without operator intervention.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Time/minFee setters lack upper bound

CATEGORY	Validation
LOCATION	RequestFactoryConfig.daml:100-123
REVIEW	Main Review

DESCRIPTION

The request-factory `RequestFactoryConfig_SetLedgerTimeValidityPeriod` setter bounds the new validity period only from below — `> seconds 0` at line 121 — and `RequestFactoryConfig_SetMinFee` is similarly unbounded above. The same lower-bound-only pattern recurs in `AdapterConfig`'s validity setter.

A config admin can therefore set an arbitrarily large `ledgerTimeValidityPeriod`, widening the window in which a caller-supplied timestamp is accepted as fresh. That weakens the documented rate-limiter requirement that the decay `window` exceed `ledgerTimeValidityPeriod`. The admin is trusted, so this is a hardening gap rather than an external exploit, but a misconfigured large period silently relaxes freshness across `send`, `receive`, and `createRequest`.

RECOMMENDATION

Add a sane upper bound to the validity-period setter (consistent with the rate-limiter window invariant `window > ledgerTimeValidityPeriod`) so the freshness window cannot be widened past a documented maximum.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

AdapterConfig is not canonical

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/AdapterConfig.daml
REVIEW	Main Review

DESCRIPTION

`fetchAndValidateAdapterConfig` accepts any supplied `AdapterConfig` whose `oappId` equals the adapter's `oappId` and whose signatories include `oappId.admin`. The `AdapterConfig` template has no contract key and `LockUnlockAdapter` stores no canonical config CID. Therefore more than one active admin-signed config can exist for the same adapter identity and callers can choose which one the adapter uses.

This affects both request creation and settlement. `adapterLzSendImpl` and `adapterLzReceiveImpl` read `oappConfig` from caller-supplied arguments. `adapterDispatchAccept` and `adapterDispatchReject` read a fresh config CID from executor-supplied `executeContext`. If an older or alternate config remains active, an unprivileged caller or finalizer with visibility can select weaker settings. This can bypass the intended current peer, pause, fee, cost-assert or rate-limiter configuration.

The settlement side is especially sensitive because callback data stores base-unit amounts but not the config CID or decimals used at send time. A reject or payout can be converted with `config.localDecimals` from a different active config and treasury transfers are authorized against that config's `assetInstrumentId`. Consequently, an alternate config for the same `oappId` can make settlement refund or distribute the wrong amount or the wrong asset from the shared treasury.

RECOMMENDATION

Bind each `LockUnlockAdapter` instance to exactly one current config. Store the canonical `AdapterConfig` CID on the adapter or give `AdapterConfig` a contract key keyed by `oappId` and resolve that key instead of accepting arbitrary CIDs.

Also bind the config used during request creation into the callback data or store immutable settlement parameters such as `assetInstrumentId` and `localDecimals` in the callback. Accept and reject callbacks should settle only with the same config or with an explicit successor that preserves those immutable accounting fields.

L-03 AdapterConfig is not canonicalCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

RequestFactory uses caller-selected fee config

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/RequestFactory/daml/RequestFactory.daml
REVIEW	Main Review

DESCRIPTION

`RequestFactory.request_CreateImpl` accepts the `requestFactoryConfig` contract id from the caller-provided OApp arguments, fetches it as a `RequestFactoryConfig` and only checks that the current request handler is a signatory of that config. The config template itself is signed only by `handler` and does not carry a key or field binding it to the concrete `RequestFactory` instance, treasury, DSO or canonical active configuration.

This means request creation is parameterized by whichever handler-signed config the caller can disclose. If the same handler has another active config from a previous deployment, an old fee schedule, a test factory or a mistakenly retained zero/low-fee config, a caller can submit that CID instead of the intended current config for this factory. The factory then validates the request fee and ledger-time validity period against the caller-selected config rather than the factory's intended policy.

Callers with visibility to a stale or alternate handler-signed config can bypass updated request fees or freshness settings for `LockUnlockAdapter` send and receive requests. In the worst case, a retained zero-fee or very-long-validity config allows cheap request spam and underpayment of the protocol fee that the request factory is supposed to enforce.

This requires an alternate active `RequestFactoryConfig` signed by the same handler and visible to the caller. That is realistic during upgrades, multi-factory deployments, test-to-production migrations or failed cleanup because the config is not keyed or cryptographically tied to one factory instance.

RECOMMENDATION

Bind the active config to the concrete request factory. For example, store the canonical `RequestFactoryConfig` CID in the `RequestFactory`, add a key over the factory identity and handler or include immutable factory/treasury/DSO fields in `RequestFactoryConfig` and validate them during request creation. Do not accept arbitrary handler-signed config CIDs from user-supplied OApp arguments.

L-04 RequestFactory uses caller-selected fee configCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Registry allows caller-selected configs

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/Registry/daml/Registry.daml
REVIEW	Main Review

DESCRIPTION

`Registry.registry_SetPartyIdImpl` accepts `ownerConfig` from the caller, casts it to `RegistryConfig` and validates only that the config has the same `oappId` and is signed by the OApp admin.

`RegistryConfig` is not keyed to a specific `Registry` contract and the registry does not store a canonical active config CID.

As a result, registration policy is selected by the registering party from any visible admin-signed config for the same OApp. If an old, test or mistakenly retained config remains active with lower `minFee`, longer `ledgerTimeValidityPeriod` or a larger/unlimited `maxEntries`, callers can use it instead of the intended current registry policy.

Callers with visibility to an alternate `RegistryConfig` can underpay registration fees or bypass the registry's intended entry cap. This can weaken spam controls and allow the registry to grow beyond the configured limit, but it does not by itself let an attacker overwrite another fingerprint because `SetPartyId` still derives the fingerprint from `caller`.

This requires an alternate active `RegistryConfig` for the same `oappId` that is visible to the caller. That can happen during upgrades, tests, multi-registry deployments or incomplete cleanup because configs are not keyed or bound to the concrete registry instance.

RECOMMENDATION

Bind each registry to one canonical config. Store the active `RegistryConfig` CID on the `Registry`, add a key over `(oappId, registry identity)` or include immutable registry identity fields in `RegistryConfig` and validate them in `SetPartyId`. Do not accept arbitrary admin-signed config CIDs from registration callers.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

Compose payload silently dropped on receive

CATEGORY	Validation
LOCATION	LockUnlockAdapter/daml/LzReceive/Callback/Accept.daml:53-100; Oft/daml/OApp/LzReceive/Callback/Impl.daml:37-71
REVIEW	Main Review

DESCRIPTION

A cross-chain SEND_AND_CALL message carries a `composeMsg` payload, and the send path emits one whenever `composeMsg` is non-empty, so the protocol produces these messages itself. On receive, the handlers never act on that payload: it is silently accepted and discarded rather than dispatched or rejected.

In the lock/unlock adapter, `handleAdapterLzReceiveAccept` (Accept.daml:53-100) decodes the inbound message into only the receiver fingerprint and the shared-decimal amount, records inbound rate-limit usage, and unlocks the corresponding treasury assets to the receiver; the `composeMsg` is never read. The OFT receive path behaves the same way: `lzReceive` reads only the receiver and amount, delivers the base tokens, and ignores `composeMsg`.

In both cases a well-formed compose message is silently degraded to a plain transfer. The value is delivered to the receiver, but the destination compose action is never executed and never rejected, breaking atomicity for any send-and-call integration. There is no direct token loss; the harm is the silently skipped compose action, and no spec or auditor-facing document declares compose out of scope.

RECOMMENDATION

Fail safe on the receive path: reject compose-bearing messages (assert `composeMsg == ""`, or throw an explicit unsupported-compose error) in both the lock/unlock adapter receive handler and the OFT receive path, until compose is implemented, so the source side receives a clear failure instead of a silent partial settlement.

RESOLUTION

Status: Resolved.

LOW **L-07**

RESOLVED

localDecimals overflow bricks transfers

CATEGORY	Validation
LOCATION	OfftFactoryConfig.daml:149-167
REVIEW	Main Review

DESCRIPTION

`OfftFactoryConfig`'s `ensure` (`OfftFactoryConfig.daml:149-167`) checks `localDecimals >= sharedDecimals` but, unlike `AdapterConfig`, enforces no upper bound on `localDecimals`.

The conversion rate is `10 ^ (localDecimals - sharedDecimals)` (`MessageCodec.daml:20-21`); once the difference reaches 19 it overflows `Int64`, so every cross-chain transfer for that token aborts (e.g. `LzReceive/Call/Validate.daml:49`).

A token configured with a large `localDecimals` (e.g. 25 against `sharedDecimals` 6) is permanently undeliverable. The sibling `AdapterConfig` already bounds this, so the two configs are inconsistent. Admin-set, self-detecting, and recoverable by recreating the config — but a routine misconfiguration bricks the instrument.

RECOMMENDATION

Add to `OfftFactoryConfig`'s `ensure` the same `localDecimals` upper bound that `AdapterConfig` enforces (e.g. `localDecimals <= 10`), so the conversion rate cannot overflow `Int64`.

RESOLUTION

Status: Resolved.

LOW **L-08**

ACKNOWLEDGED

Pending-transfer delivery can expire unclaimed

CATEGORY	DoS
LOCATION	LzReceive/Callback/Accept.daml:75-101 and FeeUtils/Transfer/Distribute.daml:75-115
REVIEW	Main Review

— DESCRIPTION

Adapter inbound release (`handleAdapterLzReceiveAccept`) and settlement payout distribution both deliver value as a pending `TransferInstruction` proposed from the treasury, while the receive request and its callback are consumed and archived in the same transaction. The recipient must accept the instruction before the standard transfer expiry.

If a recipient without a transfer preapproval misses that window, the holdings revert to the treasury and no on-ledger state remains to re-propose the delivery. The result is a lost inbound cross-chain release or a lost settlement payout. No attacker is required and no value is burned, but the recipient has no on-ledger path to reclaim it, and recovery depends on off-ledger operator action.

— RECOMMENDATION

Persist the pending `TransferInstruction` id, or retain a contract, so the recipient or operator can re-propose delivery on-ledger after the instruction expires, instead of relying on off-ledger intervention. Alternatively, complete the transfer atomically before the request and callback are archived.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Locked send funds have no sender timeout

CATEGORY	Logical Error
LOCATION	LzSend/Call/Impl.daml:136-167
REVIEW	Main Review

— DESCRIPTION

When a user sends cross-chain through the lock/unlock adapter, their holdings — the cross-chain amount plus any contingent payouts — are locked into the treasury at the time of the call. From that point the request can only be accepted or rejected by the handler (the validator quorum); there is no sender-initiated or time-based way to cancel or reclaim.

If the handler never finalizes the request, the sender's locked funds are stranded with no on-ledger way to recover them. No funds are lost — an honest handler always accepts or rejects, and a reject refunds the sender in full — so this is a liveness gap that depends on the handler-liveness assumption rather than a loss of funds.

— RECOMMENDATION

Avoid a unilateral sender timeout-reclaim. The sender cannot tell whether the destination already delivered, so a one-sided reclaim risks releasing the funds on both chains; the safe recovery is the existing handler-controlled reject, which refunds the sender in full.

To bound the stranding, give the handler a defined obligation to finalize every request (accept or reject) within a fixed time, and/or add an on-ledger reclaim that can only be exercised once the quorum attests that the message was not delivered.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

DiscoveryRegistry shared cap exhaustion

CATEGORY	DoS
LOCATION	DiscoveryRegistry.daml:119-132
REVIEW	Main Review

DESCRIPTION

The discovery registry enforces a single global `maxEntries` cap shared by every OApp under the same handler/gateway. `SetUrl` only requires the caller to be the OApp admin and the gateway to match the handler, and the new-entry cap check is the only gate.

Any party can mint qualifying OApps with themselves as admin and the handler as gateway, then call `SetUrl` with distinct attacker-derived hashes to consume slots. Because the config permits a zero fee, this is free. Repeating it fills the registry to `maxEntries`, after which any honest OApp that has not pre-registered is permanently rejected with `errDiscoveryRegistry_RegistryFull` until the handler intervenes.

There is no fund loss and no hijack: the hash binds the admin, so existing entries cannot be overwritten. The impact is a griefing/availability denial of service against the shared discovery registry.

RECOMMENDATION

Do not share one `maxEntries` cap across all OApps under a handler. Scope the cap per registrant (for example, per admin namespace), and/or require a non-zero registration fee, so one party cannot exhaust the shared registry for free.

Alternatively, gate `SetUrl` registration behind an allowlist or the handler's approval.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

RESOLVED

Treasury fee floor bypass via negative payout

CATEGORY Validation
LOCATION Request.daml:103-124
REVIEW Main Review

DESCRIPTION

During request settlement the treasury entry is removed from the payout map before payout amounts are validated as positive, and only the combined treasury share is checked to be non-negative.

The settlement handler that authors the payout map can therefore pass a negative treasury payout that cancels the required minimum fee, settling the request while the treasury receives less than the configured `minFee`. Because that payout map is produced by the trusted settlement handler at accept time, this is a missing-validation hardening gap rather than an externally reachable exploit.

RECOMMENDATION

Validate the treasury payout entry before deleting or special-casing it (assert `payouts[treasury] >= 0`, or `treasuryShare >= minFee`), so a negative treasury payout cannot undercut the `minFee` floor.

RESOLUTION

Status: Resolved.

LOW **L-12**

ACKNOWLEDGED

Caller-selected RBAC bypasses role revocation

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/IAccessControl/daml/RbacAssertions.daml
REVIEW	Main Review

DESCRIPTION

`assertRole` authorizes a role-gated choice using the `accessControlCid` supplied by the caller. After fetching that contract, it only checks that the contract is signed by the expected admin and that its view reports the expected admin, category and id. It does not check that the CID is the current RBAC contract for that scope.

This means role revocation is only effective on the specific RBAC contract that callers decide to use. The concrete `AccessControl` template has no key over `(admin, category, id)`, so multiple active admin-signed RBAC contracts can exist for the same OApp scope. If Alice was granted a role on an older `AccessControl` contract and the admin later revokes Alice only on the intended current contract, Alice can still supply the old CID. `assertRole` will accept it because the old contract is still admin-signed and still reports the same scope.

The stale contract does not modify or literally restore the current RBAC state. Instead, it is accepted as an alternate proof of authority. `AdapterConfig` uses this helper for rate-limit, pause, peer, request-factory, fee, observer and cost-assert mutations. `LockUnlockAdapter.oApp_createRequestImpl` uses the same helper for `_ENDPOINT_DELEGATE_ROLE`. Consequently, a revoked stale role holder can continue mutating adapter configuration or creating delegated generic requests if they can disclose an older matching `AccessControl` CID.

This is not forgeable by a normal user. The stale `AccessControl` must have been signed by the trusted scope admin, so the issue depends on a duplicate or leftover admin-signed RBAC contract from deployment, migration, testing or incomplete cleanup. It is still distinct from noncanonical `AdapterConfig`: here the stale object is used to prove authorization for changing the current config, not as the config being changed.

RECOMMENDATION

Bind each RBAC scope to one canonical `AccessControl` contract. The simplest fix is to give `AccessControl` a contract key over `(admin, category, id)` and resolve that key during

L-12 Caller-selected RBAC bypasses role revocationCONTINUED

`assertRole` instead of accepting an arbitrary CID. Alternatively, store the current RBAC CID on the OApp/config object and reject role checks against any other CID.

When roles are rotated, archive or supersede old RBAC contracts in a way that prevents them from authorizing future choices.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Rate-limit accounting overcredits capacity

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml
REVIEW	Main Review

DESCRIPTION

The rate limiter can over-credit capacity in two related aggregate-accounting paths. First, `adapterLzSendImpl` passes the caller-supplied send `timestamp` into `IRateLimitState_RecordOutflow`. The updated rate-limiter implementation then uses that value as the `applyRateLimit` clock, computes decay from it and stores `lastUpdated = effectiveNow`. `assertTimestampWithinPeriod` only requires the timestamp to be no older than `ledgerTimeValidityPeriod`; it does not require the timestamp to equal the ledger time at which the state mutation occurs.

For an empty or fully decayed outbound bucket, a sender can choose the oldest still-valid timestamp. The bucket records the new usage with `lastUpdated` anchored in the past. A follow-up send near the actual ledger time immediately receives decay credit for the time between the stale timestamp and the real submission, even though that time elapsed before the first send consumed rate-limit capacity.

For example, with an outbound limit of 1000, a 100 second window and a 30 second validity period, the sender can submit a first send for 1000 using `timestamp = now - 30s`. That send records full usage but sets the bucket anchor to the stale timestamp. A second send at the current ledger time then sees 30 seconds of apparent decay and can send about 300 more units immediately. The configured invariant `window > ledgerTimeValidityPeriod` bounds the bypass, but it still lets an unprivileged sender exceed the intended short-window outbound limit by the accepted timestamp slack.

Second, delayed send callbacks can over-credit the same aggregate buckets even without a stale caller timestamp. `commitOutflow` and `reverseOutflow` decay the current aggregate bucket and then subtract the original `scaledAmount` recorded by the earlier send. The state stores only aggregate `outboundUsage`, aggregate `inboundUsage` and one `lastUpdated` timestamp per bucket; it does not track the remaining contribution of each pending send. If a callback settles after the original contribution has already decayed or after later traffic has updated the same bucket, subtracting the full original amount removes usage that belongs to later transfers.

L-13 Rate-limit accounting overcredits capacity

CONTINUED

For rejects, an old rejected send can erase outbound usage from newer sends through `IRateLimitState_ReverseOutflow`. For accepts, an old accepted send can erase inbound usage from newer receives through `IRateLimitState_CommitOutflow`.

— RECOMMENDATION

Use ledger time, not the caller-supplied send timestamp, as the rate-limiter decay clock and stored `lastUpdated`. Keep the caller timestamp for request freshness validation only.

Concretely, `IRateLimitState_RecordOutflow` should either call `getTime` internally, as the previous implementation did or receive both values and pass only ledger time into `applyRateLimit`. Add a regression test that submits a send with the oldest valid timestamp, immediately submits another send and asserts that the second send receives no decay credit from time that elapsed before the first send recorded usage.

For callback settlement, do not subtract the full original `scaledAmount` from the current aggregate bucket. Store enough per-pending-outflow data to compute the settling send's remaining decayed contribution, then subtract only that remaining amount. If the limiter must remain aggregate-only, make delayed callbacks non-adjusting once their original contribution has already decayed out.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Pending callbacks settle on wrong adapter

CATEGORY	Trust Assumptions
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/Callback/Dispatcher.daml
REVIEW	Main Review

DESCRIPTION

`adapterDispatchAccept` and `adapterDispatchReject` validate the `PendingCallback` only by `oappId.admin`, derived `appUID` and accept/reject direction. They then settle the callback with the `treasury` captured from whichever `LockUnlockAdapter` contract was supplied as `callbackCid` during finalization.

The `PendingCallback` machinery does not bind the callback to the exact `ioAppCid` that created the request. It stores the app UID and admin, then accepts any `IRequestCallback` whose `oappId` hashes to the same app UID and whose admin signs that callback contract. Therefore two `LockUnlockAdapter` contracts with the same `oappId` are interchangeable at finalize time, even if they have different treasuries or represent different custody pools.

In such a deployment, a finalizer that can see a pending callback can finalize a request created by adapter A against adapter B. The dispatcher on adapter B will parse adapter A's callback data but spend adapter B's treasury holdings. For a rejected send, the sender can be refunded from the wrong treasury while the originally locked holdings remain in adapter A's treasury. For an accepted receive, treasury B can unlock funds for a message that belonged to adapter A.

RECOMMENDATION

Bind each pending callback to the exact callback contract that originated the request. Store the originating `ioAppCid` or expected `IRequestCallback` CID in `Request` or `PendingCallback`, then require `IPendingCallback_FinalizeRequest.callbackCid` to match it before dispatch.

As defense in depth, prevent duplicate live `LockUnlockAdapter` instances for one `oappId` with a contract key or registry. Callback settlement should also verify that the treasury and immutable custody fields match the request that created the callback.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Rate limiter state is not canonical

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/AdapterConfig.daml
REVIEW	Main Review

DESCRIPTION

`requireRateLimiterCids` accepts any supplied rate-limiter state CID whose fetched interface is signed by `oappId.admin` and whose view reports the same `oappId`. It does not require the state CID to equal a canonical state stored in `AdapterConfig` and the state contract is not keyed so that only one active state can exist for an OApp.

The adapter then relies on caller- or executor-supplied state CIDs at each rate-limit update.

`adapterLzSendImpl` records outbound usage on the state supplied in send call arguments. Later, `handleAdapterLzSendAccept` reads another state hint from `executeContext` and commits the serialized `scaledAmount` and `resolvedEid` to that state. `handleAdapterLzReceiveAccept` also records inbound usage on a state selected from `executeContext`.

This is not directly exploitable by an arbitrary user from a clean deployment. Exploitation requires all of the following conditions: rate limiting is enabled, a second active `RateLimiterState` exists for the same `oappId`, that state is signed by the same `oappId.admin` and the alternate state CID is visible or disclosed to the sender or finalizer. A normal user cannot create such a valid alternate state unless the trusted admin created, signed or left behind a duplicate/stale state during deployment, testing, migration or upgrade operations.

If those conditions hold, capacity can be split across the accepted states. A send can also record usage on one state and settle against another. Consequently, the configured rate limit is enforced per supplied state CID instead of per OApp and two-phase outflow accounting can leave stale or incorrect usage in the original state. Under the current evidence this is best treated as deployment-hardening / misconfiguration risk rather than an independently user-exploitable vulnerability.

RECOMMENDATION

Bind each adapter to one canonical rate-limiter state. Store the current state CID in `AdapterConfig` or give `RateLimiterState` a contract key keyed by `oappId` and require `requireRateLimiterCids` to resolve that canonical state.

I-02 Rate limiter state is not canonical

CONTINUED

For two-phase sends, also bind the exact state used by `IRateLimitState_RecordOutflow` into the callback data. Accept and reject callbacks should commit or reverse only against that same state or against its explicit successor if the state contract is intentionally consumed and recreated.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

No per-instance treasury accounting

CATEGORY Trust Assumptions
LOCATION Accept.daml:75-101
REVIEW Main Review

DESCRIPTION

Custody for the lock-unlock adapter is a per-party, per-instrument `Splice Holding` pool, not a per-contract balance. On inbound receive-accept the adapter releases from `treasury` using `treasuryAssetCids` chosen by the executor, validated only by treasury-owner and instrument-match checks around lines 85-100. Two adapter instances that share the same `treasury` party and the same `assetInstrumentId` therefore draw from one fungible pool with no isolation by contract identity.

One instance's inbound release can spend holdings that were locked to back the other instance's outstanding cross-chain liability. Custody conservation holds only globally per `(treasury, instrument)`, not per adapter. The harm is bounded to deployments that co-locate two independent liabilities in one pool; a deployment that gives each adapter its own treasury or instrument is unaffected.

RECOMMENDATION

Document and enforce at deployment that an adapter's `(treasury, assetInstrumentId)` pair is unique per outstanding-liability domain, or add per-instance custody accounting so an instance cannot release holdings backing another instance's sends.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

RESOLVED

RBAC id=None loses per-instance isolation

CATEGORY Access Control
LOCATION RbacAssertions.daml:45
REVIEW Main Review

DESCRIPTION

Governance, RequestFactory, and DiscoveryRegistry pass `expectedId = None` to the RBAC check, so the id branch of `assertRole` is skipped (`RbacAssertions.daml:45`). A single `AccessControl` instance then authorizes every same-category contract under one admin party, rather than scoping authority to a specific instance (`Some <id>`) as the OApp/Registry scopes do. No cross-party escalation (still admin-owned), but the per-instance isolation present elsewhere is lost for these stacks.

RECOMMENDATION

Pass a stable `Some <id>` for these scopes so RBAC authority is bound per instance, matching the OApp/Registry pattern.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Full role map disclosed to every role holder

CATEGORY Informational
LOCATION AccessControl.daml:32
REVIEW Main Review

DESCRIPTION

`AccessControl` adds every role holder to its observer set (`observer (dedup $ observers <> roleParties roles)`, `AccessControl.daml:32`), and the contract's `view.roles` exposes the complete role map (all minter/burner/blacklister/admin holders). Any single low-privilege holder can therefore enumerate the entire privilege topology. The observers are required so role holders can `fetch` the contract during `assertRole`, so this is disclosure-only with no integrity impact.

RECOMMENDATION

If privilege-topology privacy matters, split role storage so a holder can verify its own role without observing the full map (e.g. per-role contracts), or document the disclosure as intended.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

ACKNOWLEDGED

slashProposalFee double-floors fee dust

CATEGORY Rounding
LOCATION Governance.daml:419-428
REVIEW Main Review

DESCRIPTION

When the multisig committee slashes a proposer's fee, `slashProposalFee` first truncates the slashed amount to an integer and then divides that integer across the committee members. The two successive floors strand the fractional remainder.

The dust that should reach committee members is instead left with the treasury on every slash. The per-event amount is sub-unit and the divisor is positive-checked, so there is no failure or material loss — only a small, systematic drift of dust toward the treasury rather than the intended recipients.

RECOMMENDATION

Compute the per-member share before truncating, or explicitly account for the remainder (distribute it or document where dust accrues), so the floor-then-divide does not silently short committee members.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

DEFAULT_ADMIN_ROLE is an over-broad delegate

CATEGORY Access Control
LOCATION Rbac.daml:40-50
REVIEW Main Review

— DESCRIPTION

In the net-new `Rbac` module, granting `_DEFAULT_ADMIN_ROLE` confers near-full OApp configuration control — `SetPeer`, `SetFeeDeposit`, `RecoverFunds` and the other default-admin-gated choices — rather than the OpenZeppelin convention where a role-admin only administers role membership.

Grant and revoke themselves remain correctly pinned to the literal admin party, so there is no self-escalation. The footgun is one of expectation: an operator familiar with the OpenZeppelin pattern may grant `_DEFAULT_ADMIN_ROLE` believing it delegates only role management, when it actually hands over broad configuration authority.

— RECOMMENDATION

Rename or clearly document `_DEFAULT_ADMIN_ROLE` as a broad config-authority delegate (not an OpenZeppelin role-admin), or split role-management authority from configuration authority so that granting it does not also confer `SetPeer` / `SetFeeDeposit` / `RecoverFunds`.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

ACKNOWLEDGED

Inbound amount $\geq 2^{63}$ SD undeliverable

CATEGORY	Validation
LOCATION	LzReceive/Call/Validate.daml:48-51
REVIEW	Main Review

DESCRIPTION

The inbound validator requires `amountSD  $\geq$  0`, but the decoded value comes from `hexToInt`, which yields a signed `Int64`. Any 64-bit value with the high bit set (at or above 2^{63}) decodes negative and is therefore permanently rejected, stranding the corresponding source-chain debit.

In practice 2^{63} shared-units is on the order of 9.2e12 tokens at six shared decimals — an unrealistic supply — so this is informational rather than a reachable failure.

RECOMMENDATION

If tokens with very large supply or decimals could ever be in scope, decode `amountSD` as unsigned (or document the supported maximum) so that large legitimate amounts are not silently rejected.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

RESOLVED

Governance ensure omits treasury-in-members

CATEGORY Validation
LOCATION Governance.daml:130-137
REVIEW Main Review

DESCRIPTION

The `Governance` contract's `ensure` clause requires the multisig to be a member and the treasury to differ from the multisig, but it does not require the treasury to be excluded from the members list. If the multisig misconfigures the treasury into `members`, the fee-redistribution recipients (members other than the proposer) come to include the treasury.

That produces a treasury-to-treasury self-payout that can abort, bricking `Reject`, `Expire`, `SetMembers` and the auto-slash path. The condition is self-inflicted by the trusted multisig at configuration time and affects liveness only — there is no fund loss.

RECOMMENDATION

Add `treasury notElem members` to the `Governance` `ensure` clause so the treasury cannot be configured as a fee-receiving member.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-10

ACKNOWLEDGED

Untrusted token interfaces fake asset locks

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/damL/LzSend/Call/Impl.damL
REVIEW	Main Review

DESCRIPTION

`adapterLzSendImpl` treats the sender-supplied `assetCids` and `transferFactoryCid` as proof that the requested amount was locked into the adapter treasury. It first sums the supplied `Holding` interface views, then calls `assertTransferFactoryAdmin`, then calls `transferViaFactory` and ignores the returned receiver holdings.

Those checks do not authenticate the underlying token implementation. `damL.yaml` imports the generic `Splice Holding` and `TransferFactory` interfaces and `parseAdapterLzSendArgs` accepts interface CIDs directly from `callArgs`. `Transfer.Core.assertTransferFactoryAdmin` only exercises `TransferFactory_PublicFetch`; the helper itself documents that a malicious factory can fake `PublicFetch` and the transfer result. The same problem applies to the supplied `Holding` CIDs, because `sumUnlockedHoldingAmounts` trusts the interface view fields `owner`, `instrumentId`, `amount` and `lock`.

A sender can therefore provide fake `Holding` contracts whose views claim to be unlocked holdings of `config.assetInstrumentId`, together with a fake `TransferFactory` that returns a successful transfer result without archiving or moving any real token. The send request is still created with a valid `LzMessage`. If the remote peer accepts it, the destination can mint or unlock value that was never locked on Canton. Even if real holdings are supplied, a fake factory can let the sender keep them while creating an apparently backed cross-chain request.

The existing negative tests only use an honest `OfftFactory` with the wrong admin, which fails because that implementation enforces `expectedAdmin`. They do not cover a malicious implementation that reports the expected admin and returns a fabricated transfer result.

RECOMMENDATION

Do not accept arbitrary token interface implementations from users for custody-critical transfers. Store an admin-approved `TransferFactory` in `AdapterConfig` or otherwise bind the factory to the expected token package and issuer authority before `0App_LzSend` can use it.

I-10 Untrusted token interfaces fake asset locks

CONTINUED

As defense in depth, authenticate every supplied holding and factory by checking that `config.assetInstrumentId.admin` is a signatory of the fetched contracts, then verify the `transferViaFactory` result. The receiver holdings returned by the transfer should be fetched and checked to be treasury-owned, unlocked, for `config.assetInstrumentId` and equal to the amount that must be locked. Reject the send unless those checks prove that real custody moved into the treasury.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

RESOLVED

Unauthenticated registry receiver swap

CATEGORY	Access Control
LOCATION	RegistryResolve.daml:18-25
REVIEW	Main Review

— DESCRIPTION

Inbound receiver resolution is shared by both inbound settlement paths through `resolvePartyByFingerprint` (`0ftCommon/daml/0App/Callback/RegistryResolve.daml:18-25`), which reads a registry id from the finalize-time `executeContext` and resolves the receiver via `Registry.GetPartyId` from the message fingerprint. The check is incomplete: it asserts only the registry's self-reported `registryView.oappId`, never that `oappId.admin` signs the registry. Every other `executeContext`-supplied CID (config, rate-limiter state) is bound by an `oappId.admin elem signatory` check; the registry CID gets none, and the resolved party is never re-bound to the fingerprint.

Finalize is permissionless by design: `IPendingCallback_FinalizeRequest` is `controller actor`, so any party that can see the inbound `PendingCallback` supplies the `executeContext`. The attacker authors a self-signed registry whose view reports the victim `oappId` and whose `GetPartyId` returns the attacker; the equality passes and admin authority flows from the `PendingCallback` signatory, so the attacker selects the recipient.

This visibility precondition can become public if choice-context endpoints expose pending callbacks and explicit disclosures without authorization. In that deployment, an HTTP caller can obtain the disclosed `PendingCallback`, submit `IPendingCallback_FinalizeRequest` directly, and choose the same finalize-time registry CID used by this exploit. The API exposure is not required for the on-ledger bug, but it makes the open-finalize precondition reachable by untrusted clients.

The redirect lands on whichever settlement path finalizes. On the lock/unlock adapter (`handleAdapterLzReceiveAccept`) it diverts the treasury unlock to the attacker, stealing the full `amountLD` per message. On the burn-mint OFT (`0ft/daml/0App/LzReceive/Callback/Impl.daml:40,65`, `create 0ft with owner = sendTo`) it mints the full inbound amount as fresh, unbacked supply to the attacker while the legitimate receiver receives nothing.

— RECOMMENDATION

Complete the on-chain-resolution fix in the shared `resolvePartyByFingerprint`: after `PublicFetch`, assert that `oappId.admin` is a signatory of the fetched registry, the same

I-11 Unauthenticated registry receiver swap

CONTINUED

admin-signatory check `requireRateLimiterCids` already applies to the rate-limiter state. This single check covers both the adapter unlock and the OFT mint call sites. Self-attested `registryView.oappId` equality does not authenticate a registry supplied at finalize time through the open-finalize `executeContext`. As defense in depth, also re-check the resolved party against the message fingerprint after resolution.

Also require authentication and authorization on any API endpoint that returns `PendingCallback` payloads, explicit disclosures, or finalize-request contexts. That reduces who can exercise open finalization, but it should be defense in depth: the callback must still reject unauthenticated registries on ledger.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/commit/df15cdabba5b05dc3388bc7aba7c237f14586be9>

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-12

ACKNOWLEDGED

Finalize TransferFactory can fake transfers

CATEGORY Validation

LOCATION contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/Callback/ExecuteContext.daml

REVIEW Main Review

DESCRIPTION

`Callback.ExecuteContext` reads `transferFactoryCid` and `assetTransferContext` from finalize-time `executeContext`. The settlement handlers then use those values for treasury transfers.

`handleAdapterLzReceiveAccept` treats `proposePendingTransfer` as proof that the treasury released `amountLD` to `sendTo`. `handleAdapterLzSendReject` similarly treats it as proof that the sender refund was proposed. `handleAdapterLzSendAccept` treats `distributePayouts` as proof that contingent payouts were delivered.

That check does not authenticate an arbitrary `TransferFactory` implementation.

`assertTransferFactoryAdmin` only exercises `TransferFactory_PublicFetch` and the transfer helper explicitly assumes the factory is trusted because a malicious implementation can fake both `PublicFetch` and the transfer result. A finalizer that can see the `PendingCallback` can therefore supply a malicious factory whose `TransferFactory_Transfer` returns `Completed` or `Pending` without consuming the treasury holdings or creating a real transfer instruction for the receiver.

The callback then succeeds and the `PendingCallback` is archived. On inbound accept, the inbound rate limiter has already recorded the inflow, so the accepted LayerZero receive is considered settled while the receiver receives no token release. On outbound reject, the rate limiter can be reversed while no refund instruction is created for the sender. On outbound accept, fee or external payout recipients can be left unpaid. This breaks the settlement invariant that a finalized callback must either move the promised treasury assets or revert.

RECOMMENDATION

Do not accept the asset transfer factory from finalize-time `executeContext`. Bind the canonical `TransferFactory` for `config.assetInstrumentId` in `AdapterConfig` or resolve it through an admin-authenticated registry or contract key.

As defense in depth, verify transfer results before finalizing the callback. For a completed transfer, fetch returned holdings and assert that they are owned by the expected receiver, match `config.assetInstrumentId`, are unlocked and sum to the expected amount. For a

I-12 Finalize TransferFactory can fake transfers

CONTINUED

pending transfer, persist the instruction id or require a trusted factory implementation whose pending instruction can be authenticated.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/guardian/contracts/protocol/canton/contracts/tests/daml/Guardian/UnbackedGenericDrawin.daml>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-13

ACKNOWLEDGED

Governance fee accepts untrusted factory

CATEGORY Access Control
LOCATION Governance.daml:200-227
REVIEW Main Review

DESCRIPTION

`Governance_Propose` (controller `submitter`, gated only by `submitter elem members`) transfers the proposal fee using the raw caller-supplied `amuletContext : transferAmulet` (`Governance.daml:217-220`) pulls `externalPartyAmuletRulesCid` from `amuletContext` (`Extractors.daml:24-28`, key `_KEY_EXTERNAL_PARTY_AMULET_RULES`) and coerces it to a `TransferFactory`, authenticated only by the spoofable view check `validateExternalPartyAmuletRules` (`Validators.daml:104-116`, just `TransferFactory_PublicFetch`) — the same unauthenticated-factory mechanism as the adapter custody issue.

A trusted override `withTrustedExternalParty` (`Governance.daml:404-410`) was added and applied to the slash and Expire paths (`:207`, `:341`) but NOT to the main `Governance_Propose` fee transfer. A single committee member (untrusted in a multisig model) can supply a forged `TransferFactory` whose `Transfer / Accept` return a fabricated treasury-owned holding without consuming the member's amulet.

Impact: (distinct from the adapter factory issue — no bridge custody): (1) anti-spam fee bypass, the proposer keeps their funds; (2) permanent governance brick — the fabricated holding is stored unvalidated as `ProposalData.amuletCid` (`Governance.daml:222-227`), and every later path that touches the fee (`Accept`, `Reject` via `slashProposalFee`, `Expire`, `SetMembers`) does a real `fetch` on the fake CID and aborts; no choice clears an active proposal without touching its fee, so even the honest multisig cannot recover.

RECOMMENDATION

Apply the existing trusted override to the `Governance_Propose` fee transfer: pass the signatory-bound `externalPartyAmuletRulesCid` via `withTrustedExternalParty` (as the slash and Expire paths already do) instead of the raw caller `amuletContext`. As defense in depth, validate the returned holding (treasury-owned, real DSO Amulet, expected amount) before storing it as `ProposalData.amuletCid`.

I-13 Governance fee accepts untrusted factory

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Delegated burner can destroy locked holdings

CATEGORY Access Control
LOCATION 0ftFactory.daml:394-404
REVIEW Main Review

DESCRIPTION

Both OFT burn paths reach the shared `executeBurn` helper, which archives token holdings without requiring them to be unlocked, unlike the recover path which checks lock state.

The delegated burner path (`0ftFactory_RbacBurn` , gated by a delegable burner role) lets a non-issuer destroy holdings locked for escrow, allocation, or transfer settlement; the issuer burn/mint path shares the same defect under admin authority. Locked, in-flight value can be destroyed with no recovery.

RECOMMENDATION

Require `mustBeUnlocked` in the shared `executeBurn` helper before archiving, mirroring the recover path; if locked holdings must ever be burnable, expose a separate explicit migration choice with settlement-state checks.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

Registry keyed by namespace, not full party

CATEGORY	Validation
LOCATION	Registry.daml:59-102
REVIEW	Main Review

— DESCRIPTION

The receiver registry stores hints keyed only by the Canton namespace fingerprint rather than the full party id, and permits any caller sharing that fingerprint to overwrite an existing entry.

Two distinct parties hosted under the same namespace share the fingerprint, so a co-namespace party can overwrite a victim's receiver hint and redirect inbound cross-chain mints/unlocks to itself.

This is a distinct root cause from the unauthenticated-registry-CID issue and is not fixed by binding the registry to the admin signatory.

— RECOMMENDATION

Key registry hints by the full Canton party id (or store and verify the full party id on overwrite), so a co-namespace sibling cannot overwrite another party's receiver hint.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-16**

ACKNOWLEDGED

Allowlist granularity collapses to namespace

CATEGORY	Access Control
LOCATION	IAllowlist.daml:113-124
REVIEW	Main Review

— DESCRIPTION

The token allowlist/blacklist checks only the Canton namespace fingerprint of a party, not its full id. Whitelisting one party therefore authorizes every party hosted under the same namespace, so a co-namespace sibling can pass transfer and send allowlist checks it was never explicitly approved for.

— RECOMMENDATION

Store and compare full party ids in allowlist/blacklist entries rather than the namespace fingerprint suffix; if suffix matching is needed, require a registry proof binding the suffix to the exact party.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

Allocation execute ignores settleBefore

CATEGORY Validation
LOCATION OftAllocation.daml:60-91
REVIEW Main Review

DESCRIPTION

The allocation execution choice stores a `settleBefore` deadline but never checks it before moving the locked holding. This contradicts the Splice allocation standard it implements, which enforces the deadline on-ledger.

A settlement executor can therefore complete an allocation after the sender's deadline, delivering the locked tokens under terms the metadata says are no longer valid.

RECOMMENDATION

Enforce `getTime <= settleBefore` in `allocation_executeTransferImpl` (as the Splice reference does), and add an admin/DSO post-deadline reclaim path.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

RESOLVED

refundAddress observer leaks payload

CATEGORY Unexpected Behavior
LOCATION RequestFactory.daml:109
REVIEW Main Review

DESCRIPTION

`RequestFactory` appends the optional caller-supplied `refundAddress` to the `Request` observer set at line 109 (`observers ++ permanentObservers ++ optionalToList refundAddress`), and that set propagates to the settle-phase `PendingCallback`. A `Request` observer reads the whole payload: `callContext` and `callbackContext` (cross-chain amount, payout receivers and amounts) plus `sender`. But `refundAddress` only needs its fee remainder, which `distributePayouts` already delivers as an independent `TransferInstruction` when the refund party differs from `sender`.

So listing `refundAddress` as an observer over-discloses: a party that should see only its own refund can read every counterparty and value in the request. This is material when `refundAddress` is a third-party funder not otherwise entitled to the request — distinct from `sender`, `treasury`, `handler`, `oappAdmin`. Where it is the executor that already finalizes the callback, there is no marginal leak.

RECOMMENDATION

Do not add `refundAddress` to the `Request` / `PendingCallback` observer set. Deliver its fee remainder solely via the standalone `TransferInstruction`, disclosed only to the refund recipient, so the refund party cannot read the full request payload.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-19

ACKNOWLEDGED

Finalize asset context can bypass token controls

CATEGORY	Trust Assumptions
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/Callback/ExecuteContext.daml
REVIEW	Main Review

DESCRIPTION

`extractAssetTransferContext` accepts an arbitrary `AV_Map` from finalize-time `executeContext` and converts it directly into the `ChoiceContext` used for treasury settlement transfers.

`handleAdapterLzReceiveAccept`, `handleAdapterLzSendAccept` and `handleAdapterLzSendReject` pass that context into the wrapped token's `TransferFactory`.

This lets the finalizer choose the wrapped token policy context at settlement time. For token factories that use `ChoiceContext` to select policy or config contracts, the finalizer can select stale or alternate policy state. The in-repository `OftFactory` demonstrates the pattern: `TransferFactory_Transfer` extracts a config CID from `extraArgs.context`, checks only that it matches the instrument and is admin-signed, then enforces `localTransfersPaused` and allowlist checks from that supplied config.

Consequently, an accepted receive, send payout or reject refund can settle under token controls that the issuer no longer intended to allow. For example, if the current token config has local transfers paused or no longer allowlists a receiver, a finalizer can supply an older valid config in `assetTransferContext` and make the treasury transfer proceed anyway.

RECOMMENDATION

Do not accept the wrapped token transfer context as an arbitrary finalize-time map. Bind the expected transfer context or at least the token policy/config CID inside it, to `AdapterConfig` or to an issuer-authenticated registry for `assetInstrumentId`.

If the context must be supplied at finalize time for disclosure reasons, parse the expected keys and verify that every policy/config CID resolves to the canonical current contract before calling the token `TransferFactory`. Add regression tests where a stale same-instrument token config is unpaused or more permissive, then assert that receive accept, send accept and send reject settlement all reject it.

I-19 Finalize asset context can bypass token controls

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-20

ACKNOWLEDGED

Holding context bypasses token controls

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzSend/Call/Parser.daml
REVIEW	Main Review

DESCRIPTION

`parseHoldingTransferContext` accepts any `AV_Map` supplied in `callArgs` and converts it directly into the `ChoiceContext` used for the wrapped token transfer. `adapterLzSendImpl` then passes that context into `transferViaFactory` when it locks the sender's holdings into the adapter treasury.

This makes the sender choose the wrapped token's transfer context. For token factories that use `ChoiceContext` to select their own policy contract, this can select stale or alternate policy state. The in-repository `0ftFactory` demonstrates the pattern: `TransferFactory_Transfer` extracts an `0ftFactoryConfig` CID from `extraArgs.context`, validates only that it matches the instrument and is admin-signed, then enforces `localTransfersPaused` and allowlist checks from that supplied config.

If more than one valid token config exists for the same instrument, a sender can provide a permissive context even when the intended current token config would pause transfers or reject the sender or treasury by allowlist. The adapter separately checks holding owner, instrument and balance, but those checks do not enforce the wrapped token's current transfer policy. Consequently, the adapter can create a cross-chain send request under token controls that the token issuer no longer intended to allow.

RECOMMENDATION

Do not accept the wrapped token transfer context as an arbitrary user-supplied map. Bind the expected transfer context or at least the token policy/config CID inside it, to `AdapterConfig` or to an issuer-controlled registry for `assetInstrumentId`.

If the context must stay caller-supplied for disclosure reasons, parse the expected keys and verify that policy/config CIDs resolve to the canonical current contracts for the wrapped token before calling `transferViaFactory`. Add regression tests where a stale same-instrument config has local transfers unpaused or a more permissive allowlist and assert that `0App_LzSend` rejects it.

I-20 Holding context bypasses token controls

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-21

ACKNOWLEDGED

Rate-limit zero limit blocks the direction

CATEGORY DoS
LOCATION RateLimiterConfigTypes.daml:98-102
REVIEW Main Review

DESCRIPTION

A rate-limiter config entry accepts a `limit` of `0` while that direction is enabled and not globally disabled (`isValidConfigEntry`, `RateLimiterConfigTypes.daml:98-102`).

The config's own comment acknowledges that `limit = 0` with the direction enabled permanently zeroes available capacity, so every legitimate transfer in that direction aborts with `errRateLimit_Exceeded` (`RateLimiterState.daml:341` — `available (0) >= amount` can never hold).

A routine misconfiguration therefore bricks all transfers in that direction until an admin corrects it. No funds are lost and it is admin-recoverable, but the throughput control becomes a denial-of-service switch.

RECOMMENDATION

Reject `limit == 0` while the direction is enabled in `isValidConfigEntry` (require `limit > 0`), or treat `0` explicitly as "disabled", so an enabled direction cannot be configured into a permanent block.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-22

ACKNOWLEDGED

Pending refunds can be delayed

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzSend/Call/Impl.daml
REVIEW	Main Review

DESCRIPTION

`adapterLzSendImpl` transfers `crossChainAmount + payoutAmount` into the shared `treasury` pool before creating the LayerZero request, but it does not store or reserve the returned treasury holding CIDs for a possible reject.

This does not cause a partial refund or corrupt treasury state. `handleAdapterLzSendReject` uses fresh `treasuryAssetCids` from the finalizer's `executeContext` and the transfer helper checks that those holdings still cover the refund amount. If they do not, the reject finalization aborts atomically and the pending callback remains retryable.

The issue is narrower: the sender's locked value is pooled with general treasury liquidity while the request is unresolved. Another valid settlement can consume the same treasury liquidity before the reject is finalized. If the rejected send is finalized later and the finalizer cannot provide enough fresh unlocked treasury holdings, the refund cannot be created until the treasury has enough liquidity again.

Example:

- Alice sends 100 tokens through `0App_LzSend`.
- `adapterLzSendImpl` moves Alice's 100 tokens into the adapter treasury, then creates the outbound request.
- The request does not reserve the treasury holding created by Alice's send.
- Before Alice's reject callback is finalized, Bob settles a separate valid inbound receive for 100 tokens.
- Bob's settlement can use the same treasury liquidity because the pool does not distinguish Alice's pending refund backing from general liquidity.
- Alice's reject is finalized later. It uses fresh `treasuryAssetCids`, not the send-time holding CIDs.
- If the supplied fresh treasury holdings no longer cover 100 tokens, the reject transaction aborts atomically.
- Alice's callback remains pending and retryable, but her refund is delayed until enough treasury liquidity is available again.

I-22 Pending refunds can be delayed

CONTINUED

— RECOMMENDATION

If rejected sends are expected to be immediately refundable from their original backing, reserve the locked holdings or record an equivalent liability for each unresolved outbound send. Reject settlement should either consume the send's reservation or prove that the treasury remains solvent after accounting for pending reject liabilities.

If shared liquidity netting and retry-until-funded behavior are intended, document that model and add monitoring for pending rejects that cannot currently be refunded.

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

RESOLVED

Delegated generic requests can suppress inbound unlocks

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LockUnlockAdapter.daml
REVIEW	Remediation Review 1

DESCRIPTION

`oApp_createRequestImpl` lets an `_ENDPOINT_DELEGATE_ROLE` holder create an arbitrary OApp request after only checking the adapter config, timestamp and delegate role. It forwards the caller-supplied `callContext` and `callbackContext` to `RequestFactory` without rejecting endpoint functions that consume or preempt adapter messages.

This lets a delegate create a generic request whose call context targets `EndpointV2.clear` for the adapter app UID. The endpoint exposes `clear` as a write method and documents it as a way for the OApp or its delegate to skip or burn a verified message. Its implementation calls `MessagingChannel.clearPayload`, which deletes the verified inbound payload hash and marks the nonce delivered without executing `LzReceive`.

The same generic request can invoke `MessagingChannel.skip` before a payload has been verified. `RequestFactory` accepts a context shaped as `OApp -> appUID -> EndpointV2`, but it does not prove that the requested function belongs to the declared target contract. The runtime instead searches the endpoint and its components by function name, so an `EndpointV2` request can dispatch to the component method `skip`.

`skip` requires the supplied nonce to be the next inbound nonce and then advances `lazyInboundNonce` without recording a payload hash. A delegate can therefore preempt the next valid adapter message before its verification arrives. Verification of the real packet then fails because the skipped nonce is no longer ahead of `lazyInboundNonce`, while no payload hash exists that could later be delivered.

The adapter then creates a normal pending callback for the accepted request. If the delegate supplied an empty or unsupported callback context, `adapterDispatchAccept` returns `pure ()` instead of rejecting the callback. Consequently the pending callback can be archived even though `handleAdapterLzReceiveAccept` never ran, no registry resolution happened and no treasury transfer was proposed to the receiver.

With `clear`, the delegate consumes an already verified inbound packet. With `skip`, the delegate needs no verified payload, guid, message or payload hash and can invalidate the

H-01 Delegated generic requests can suppress inbound unlocks

CONTINUED

next packet in advance. In both cases, the ordinary `oApp_LzReceive` request cannot settle the message, so a non-admin delegate can strand the destination-side unlock for a valid cross-chain transfer.

The skip variant was reproduced at both boundaries. A Daml test showed that the scoped adapter accepts and finalizes a delegate-created generic request carrying `skip` with an empty adapter callback. A runtime test showed that an `EndpointV2` transaction dispatches to `MessagingChannel.skip`, advances `lazyInboundNonce` and causes later verification of that nonce to fail with `EndpointV2_PathNotVerifiableError`.

— RECOMMENDATION

Do not allow `LockUnlockAdapter.oApp_createRequestImpl` to create generic requests for endpoint functions that consume, suppress or preempt adapter custody messages. At minimum, reject `clear`, `skip`, `nilify`, `burn`, `lzReceive`, receive-library changes and `send` unless the request is produced by a dedicated adapter entrypoint that binds the expected message and settlement data.

Align on-ledger target validation with runtime dispatch. `RequestFactory` or the validator should prove that the requested function belongs to the declared target contract or the runtime dispatcher should honor the contract named by the function signature instead of resolving functions across every endpoint component.

Also make adapter callback dispatch fail closed for empty or unknown callback contexts. A request that touches adapter-owned endpoint state should not be finalizable as a no-op.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/GenericClearBlocksInboundUnlock.daml>

— RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

Mixed-case message prevents receive settlement

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/damL/LzReceive/Call/Impl.damL
REVIEW	Remediation Review 1

DESCRIPTION

`adapterLzReceiveImpl` validates `lzMessage` and then stores the original text unchanged in both the endpoint `callContext` and the adapter `callbackContext`. The shared decoder accepts uppercase and lowercase hex digits, but it does not canonicalize them. At callback time, `LzReceive/Callback/Accept.damL:52-62` decodes the stored text again. `decodeMessage` returns the first 64 characters verbatim as `sendToFingerprint` and `resolvePartyByFingerprint` uses that text in the registry's case-sensitive `TextMap.lookup`.

The off-ledger endpoint interprets the same field differently. `EndpointV2.lzReceive` declares `message` as ABI `bytes` and the production byte-array schema decodes mixed-case and lowercase hex into the same byte array. Therefore a request carrying a case-only variant of a real message matches the verified payload hash, succeeds and deletes the endpoint's only stored payload. The resulting `PendingCallback` still contains the original mixed-case DamL text. Its receiver lookup fails because the registry key is the canonical fingerprint extracted from the receiver's Party ID, so the callback cannot create the treasury transfer.

An untrusted sequencer or receive relay or another fee payer with the standard OApp disclosures, can observe an inbound packet and race or pre-submit the normal request with a case-only variant of its recipient fingerprint. If the variant is ordered first, validators accept and consume the byte-identical endpoint payload. Every later request using the canonical spelling is rejected because the payload has already been deleted, while retries of the accepted callback keep failing on the same poisoned fingerprint. The attacker can repeat this for inbound transfers and strand assets already locked or burned on the source chain. This uses the dedicated `OApp_LzReceive` function and honest packages.

RECOMMENDATION

H-02 Mixed-case message prevents receive settlement

CONTINUED

Canonicalize the message exactly once before validation and before building either context. For example, require `lzMessage` to equal its lowercase, unprefixed canonical hex representation or decode it to bytes and re-encode it canonically. Store that same canonical value in both `callContext` and `callbackContext`.

Also make receiver resolution consume a canonical bytes32 fingerprint rather than representation-sensitive free-form text.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/MixedCaseReceiveStrand.daml>

— RESOLUTION

Status: Resolved.

HIGH **H-03**

RESOLVED

Base-unit native fees poison commit batches

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/StateTransition/daml/Committer/CommitterImpl.daml
REVIEW	Remediation Review 1

DESCRIPTION

The scoped runtime and Daml settlement define incompatible units for the same payout field.

`EndpointV2.finishSend` records raw `uint256` invoice fees as bigint payouts through `Context.pay` (`endpoint-v2.ts:276-304,597-621`; `context.ts:54-61,116-149`). `BatchEntry.payouts` instead declares those values as `Map Party Decimal`, then `commitBatchImpl` forwards them unchanged into request settlement (`CommitterImpl.daml:17-23,85-96`). `distributeRequestEscrow` spends each `Decimal` directly as whole Canton Coin (`Request.daml:98-136`).

The production Canton integration exposes the mismatch. It converts a Daml `Decimal` escrow remainder into a bigint scaled by 10^{10} before constructing `Context` (`canton-chain-client.ts:128-136`), but serializes each recorded bigint with only `amount.toString()` when building `CommitBatch` (`canton-chain-client.ts:243-250`). Consequently, the runtime sufficiency check and on-ledger settlement disagree by a factor of 10^{10} .

An ordinary party can trigger this without OApp or admin authority. `Request_Create` is controlled only by `sender` when `oAppInput = None` (`IRequestFactory.daml:44-71`), and non-OApp validation only binds the call-context caller address to that sender (`RequestFactoryValidation.daml:50-63`). The sender can therefore submit a normal `EndpointV2.send` request. For example, with a native fee of `100`, an escrow remainder of `0.01` CC becomes a runtime balance of `100000000`. The endpoint accepts the fee and records a payout of `100`, but the resulting Daml command tries to pay `100.0` CC from the `0.01` CC escrow.

That payout aborts `CommitBatch`. The committer processes every entry and creates the next state commitment in one atomic transaction (`CommitterImpl.daml:64-110`), so a failing payout also rolls back benign entries in the same batch. The failed request stays active. The Canton request poller subsequently returns it again from the same persisted starting offset, causing every retry and later batch to fail. A sender can therefore stop state commitments and keep all subsequent cross-chain requests pending at the cost of one ordinary request. The handler can recover by manually rejecting the request, but the sender can repeat the attack until the unit mismatch is fixed.

H-03 Base-unit native fees poison commit batches

CONTINUED

— RECOMMENDATION

Define one native-fee denomination at the boundary between the runtime and Canton. The robust in-scope fix is to make `BatchEntry.payouts` carry explicit CC base units, preferably `Map Party Int` or a dedicated `CcBaseUnits` type. Perform one checked division by 10^{10} inside `commitBatchImpl` before exercising `IRequest_Accept` or `IRequest_Reject`, rejecting negative, overflowing, or non-representable values. The smallest integration fix is to format each runtime bigint as a ten-decimal CC string before constructing `CommitBatch`. Do not copy an untyped integer directly into a Daml Decimal.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/packages/protocol/lz-ver-protocol/ver-api-common/test/Guardian/native-fee-scaling.test.ts>

— RESOLUTION

Status: Resolved.

HIGH **H-04**

RESOLVED

Rate-limit timestamps break validator quorum

CATEGORY	Logical Error
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/daml/0App/LzSend/Call/RateLimiterArgs.daml
REVIEW	Remediation Review 1

DESCRIPTION

Every validator must produce the same prepared transaction before the committee can settle a request. Rate-limited sends break that requirement because the Daml callback stores a historical timestamp as a `Time`, while the TypeScript normalizer assumes that every `Time` inside a create or exercise argument came from the validator's current `getTime` call.

The rate limiter returns `recordedAt` when the send first records its outflow. This value is the decay anchor later used by `CommitOutflow` or `ReverseOutflow`; it must remain unchanged. The callback encoder nevertheless stores it as `AV_Time`:

```
encodeLzSendRateLimiterCallbackArgs : Optional (Int, Int, Bool, Time) -> [(Text, AnyValue)]
encodeLzSendRateLimiterCallbackArgs rateLimiterResult =
case rateLimiterResult of
Some (scaledAmount, resolvedEid, forwardRecorded, recordedAt) | scaledAmount > 0 ->
[ (_KEY_RL_SCALED_AMOUNT, AV_Int scaledAmount)
, (_KEY_RL_RESOLVED_EID, AV_Int resolvedEid)
, (_KEY_RL_FORWARD_RECORDED, AV_Int (if forwardRecorded then 1 else 0))
, (_KEY_RL_RECORDED_AT, AV_Time recordedAt)
]
_ -> []
```

Accepting or rejecting the request copies this callback context into a newly created `PendingCallback`. The same historical value therefore appears in a create argument during either settlement decision:

```
case this.appInfo of
Some AppInfo{oappAdmin, appUID} ->
void $ create PendingCallback with
sender = this.sender
oappAdmin
callbackContext = this.callbackContext
actionContext = acceptCtx
```

H-04 Rate-limit timestamps break validator quorum

CONTINUED

```
observers = this.handler :: this.observers
appUID
isAccept = True
None -> pure ()

case this.appInfo of
Some AppInfo{oappAdmin, appUID} ->
void $ create PendingCallback with
sender = this.sender
oappAdmin
callbackContext = this.callbackContext
actionContext = rejectCtx
observers = this.handler :: this.observers
appUID
isAccept = False
None -> pure ()
```

After each validator prepares the transaction, the normalizer calculates the difference between the committee's target time and that validator's local preparation time. It then shifts every timestamp inside create arguments and exercise values by that difference:

```
// WARNING: this assumes all timestamps in create args / chosenValues are
// getTime-derived. If any literal/constant timestamp exists in those
// locations, the shift will make it diverge between participants.
const timeDelta = params.preparationTime - metadata.preparationTime;
if (timeDelta !== 0n) {
  shiftTimestampsInNodes(transaction.nodes, timeDelta);
}
```

H-04 Rate-limit timestamps break validator quorum

CONTINUED

```

export function shiftTimestampsInValue(value: Value, delta: bigint): void {
  const { sum } = value;
  switch (sum.oneofKind) {
    case 'timestamp':
      sum.timestamp = String(BigInt(sum.timestamp) + delta);
      break;
    case 'record':
      for (const field of sum.record.fields) {
        if (field.value) {
          shiftTimestampsInValue(field.value, delta);
        }
      }
      break;
  }
}

```

This produces the opposite of the intended result for `recordedAt`. Suppose the stored value is `C`, the shared target time is `T` and two validators prepare at `P1` and `P2`. Their normalized values become `C + T - P1` and `C + T - P2`. Since `P1` and `P2` differ, the supposedly fixed timestamp is different in each prepared transaction.

The Canton client cannot prevent the divergence because it does not pass a shared ledger interpretation time into `prepareForSigning`. It prepares with local time first and normalizes afterward:

```

const prepared = await this.#sdk.prepareForSigning(
  { actAs: commands.actAs },
  commands.commands,
  {
    disclosedContracts: commands.disclosedContracts,
    commandId: params.commandId,
  },
);
const normalized = await normalize(prepared, params);

```

The quorum checker groups responses by `preparedTransactionHash`. A validator whose historical timestamp was shifted by a different amount lands in a different group, so a write threshold greater than one is never reached:

```

getTransactionPayloadHash(transaction: CantonValidatorTransaction): string {
  return transaction.preparedTransactionHash;
}

```

H-04 Rate-limit timestamps break validator quorum

CONTINUED

```
const group = groups.get(key) ?? new Map();
groups.set(key, group);
group.set(signature, response);
if (group.size >= this.#quorum) {
  return [...group.values()];
}
```

An ordinary sender can trigger this. For the lock/unlock adapter, a positive rate-limited send first records the outflow, then transfers the tokens into treasury custody and finally creates the request. This is the complete relevant code:

```
-- Outbound rate limit (phase 1): record usage before locking assets
rateLimiterResult <- case rateLimiter of
Some (rateLimiterStateCid, rateLimiterConfigCid) ->
Some <$> exercise rateLimiterStateCid IRateLimitState_RecordOutflow with
  user = sender
  amount = amountToSendCrossChain
  eid = sendParam.dstEid
  timestamp
  configCid = rateLimiterConfigCid
None -> pure None
-- Contingent payouts as (receiver, base-unit amount). Normalized once here for
-- parity with accept/reject (drop non-positive, merge duplicate receivers).
contingentPayouts <- buildLzSendContingentPayouts
  adapterFeeAmount
  config.feeDeposit
  sender
  payoutReceivers
  payoutAmounts
let (payouts, payoutTotalInt) = mergeLzSendPayouts contingentPayouts
let crossChainAmount = rawAmountToDecimal amountToSendCrossChain config.localDecimals
let payoutAmount = rawAmountToDecimal payoutTotalInt config.localDecimals
-- Lock the cross-chain amount plus the contingent payouts into the single
-- treasury pool (same-tx propose+accept). Multiple input holdings are merged;
-- the resulting CIDs are not pinned, settlement pulls fresh treasury holdings.
_ <-
  transferViaFactory
  transferFactoryCid
  assetCids
  (crossChainAmount + payoutAmount)
```

H-04 Rate-limit timestamps break validator quorum

CONTINUED

```

sender
treasury
config.assetInstrumentId
timestamp
config.ledgerTimeValidityPeriod
holdingTransferContext
let callContext =
buildLzSendCallContext
oappId
config.endpointAddress
lzMessage
sendParam.dstEid
options
peer
let callbackContext = buildAdapterLzSendCallbackContext oappId amountToSendCrossChain payouts
rateLimiterResult
-- Create the request via IRequestFactory (protocol fee in amuletCids / amuletContext)
_ <- exercise config.requestFactoryCid Request_Create with
sender
oAppInput = Some OAppInput with oappAdmin = oappId.admin; ioAppCid = coerceContractId
ioAppCid
-- Append oappGateway so LayerZero has visibility on pending callbacks.
observers = oappGateway :: observers
amuletCids
feeAmount
timestamp
amuletContext
callContext
callbackContext
ownerConfig = requestFactoryConfig
refundAddress

```

By the time validators attempt settlement, the sender's tokens are already in the treasury and the request is active. Both Accept and Reject create the affected `PendingCallback`, so neither decision provides a normal recovery route. `CommitBatch` is atomic: failure to obtain signatures for this request also prevents the next state commitment and every other request in that batch. The request remains active, is polled again and can keep later cross-chain requests from settling.

H-04 Rate-limit timestamps break validator quorum

CONTINUED

This issue only requires rate limiting to be active, a validator write threshold above one and ordinary differences between validator preparation times.

— RECOMMENDATION

Do not encode the historical decay anchor as a Daml `Time` while the normalizer shifts every protobuf timestamp. The smallest fix is to store `recordedAt` as a stable scalar, such as checked epoch microseconds in `AV_Int` and reconstruct the `Time` only when the callback calls `CommitOutflow` or `ReverseOutflow`.

The stronger fix is to make every validator interpret the transaction at the same ledger time, for example by supplying a shared absolute minimum ledger time to `prepareForSigning` and then remove the blanket timestamp shift. If timestamp shifting must remain, the prepared transaction needs to distinguish fresh `getTime` values from historical timestamps copied out of contracts.

Add a two-validator regression test with deliberately different local preparation times. Both Accept and Reject should produce the same normalized transaction hash for a real rate-limited send. Cover OFT and the lock/unlock adapter. The processor should also isolate a request that cannot be committed so one bad request cannot indefinitely block unrelated batches.

— RESOLUTION

Status: Resolved.

HIGH **H-05**

RESOLVED

Foreign commitment blocks settlement

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/StateTransition/daml/StateCommitment/StateCommitment.daml
REVIEW	Remediation Review 1

DESCRIPTION

Any Canton party can create the official `StateCommitment` template with itself as `owner` and the LayerZero owner in `viewers`. In Daml, `signatory owner` means that the party stored in `owner` is the only party whose authority the ledger requires to create the contract. That party also controls the generated `Archive` choice. By contrast, parties listed in `viewers` are observers: they can see the contract but do not approve its creation and cannot archive it:

```
template StateCommitment
with
  owner : Party
  viewers : [Party]
  stateRoot : Text
  prevStateCommitmentCid : Optional (ContractId StateCommitment)
  requestCids: [ContractId IRequest]
  v : Int
where
  ensure isValidHex stateRoot
  signatory owner
  observer viewers
```

An attacker can therefore set itself as `owner`, authorize a valid commitment and add the LayerZero owner as a viewer. The attacker retains control over the contract. The LayerZero owner can see it but cannot prevent its creation or remove it afterward.

The SDK asks the shared singleton helper for the latest commitment visible to the configured owner:

```
const contract = await getActiveUniqueContract(this.#sdk, this.#cache, {
  packageName: PACKAGE_STATE_TRANSITION,
  templateName: TEMPLATE_STATE_COMMITMENT,
  party: this.#owner,
});
```


H-05 Foreign commitment blocks settlement

CONTINUED

However, `party` only selects whose active-contract view is queried. The helper applies the uniqueness check before it verifies the commitment's payload owner or signatories:

```
const contracts = await sdk.queryActiveContracts(opts.party, offset, {
  templateIds: [tid],
});
if (contracts.length > 1) {
  throw new Error(
    'Expected at most one ${opts.templateName} contract, found ${contracts.length}',
  );
}
return contracts[0];
```

If a genuine commitment already exists, the attacker-owned commitment raises the visible count to two and the lookup throws. If no genuine commitment exists, the same helper returns the foreign commitment as the current head. The attacker can keep the foreign contract active indefinitely because only the attacker can archive it.

Every validator resolves the current commitment before it polls or executes pending requests:

```
const commitment = await this.#stateCommitmentFinder.findCurrent();
if (!commitment) {
  throw new Error('No state commitment found on chain');
}
const requestIds = await this.#requestPoller.poll(timestampMs);
const { entries, finalize, state } = await this.#requestExecutor.execute(
  requestIds,
  commitment.stateRoot,
);
```

Consequently, the cardinality error stops the validator before it can produce either an accept or reject batch. This affects users after the adapter has already transferred their assets into treasury and created the pending request. The transfer occurs first:

```
_ <-
transferViaFactory
transferFactoryCid
assetCids
(crossChainAmount + payoutAmount)
sender
treasury
```

H-05 Foreign commitment blocks settlement

CONTINUED

```
config.assetInstrumentId
timestamp
config.ledgerTimeValidityPeriod
holdingTransferContext
```

The request is then created in the same successful transaction:

```
_ <- exercise config.requestFactoryCid Request_Create with
sender
oAppInput = Some OAppInput with oappAdmin = oappId.admin; ioAppCid = coerceContractId
ioAppCid
observers = oappGateway :: observers
amuletCids
feeAmount
timestamp
amuletContext
callContext
callbackContext
ownerConfig = requestFactoryConfig
refundAddress
```

The pending request cannot be accepted or rejected while the poisoned lookup remains in use and later requests are blocked as well. This attack needs no LayerZero role, malicious DAR or owner-managed duplicate. It works because the SDK mistakes visibility of an ordinary party's correctly signed contract for ownership by the LayerZero party.

— RECOMMENDATION

Authenticate commitments before applying the singleton cardinality check. Keep only contracts whose `createArgument.owner` equals the configured LayerZero owner and whose created-event signatories include that owner. Then return the one authenticated commitment, return no commitment when only foreign contracts are visible and reject only when multiple authenticated commitments exist.

Apply the same owner and signatory checks to commitment-by-address lookups, recovery logic, event conversion and cached committer discovery. Add regression tests for one genuine commitment plus multiple foreign commitments, foreign commitments only and foreign commitments with a valid state root and expected software version.

— RESOLUTION

Status: Resolved.

HIGH **H-06**

RESOLVED

Expired fee escrow poisons commit batches

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/Request/daml/Request.daml
REVIEW	Remediation Review 1

DESCRIPTION

`RequestFactory.request_CreateImpl` transfers the caller's fee into an ordinary handler-owned Amulet and stores that exact output CID in the new `Request`. This assumes the CID remains active until settlement. Amulets instead become eligible for expiry as holding fees accrue. Normal DSO maintenance can then consume them through `Amulet_ExpireV2` before the request is settled.

Both terminal decisions distribute the same stored escrow before archiving the request:

```
request_AcceptImpl reqCid IRequest_Accept{amuletContext = amuletCtx, payouts, acceptContext =
acceptCtx} = do
  validateContext amuletCtx this.sender this.dso
  distributeRequestEscrow this payouts amuletCtx
request_RejectImpl reqCid IRequest_Reject{amuletContext = amuletCtx, payouts, rejectContext =
rejectCtx} = do
  validateContext amuletCtx this.sender this.dso
  distributeRequestEscrow this payouts amuletCtx
```

`distributeRequestEscrow` then uses `amuletCid` directly for the treasury transfer or subsequent distribution:

```
escrowCids <-
  if treasuryShare > 0.0
  then do
    (_, changeCids) <- transferAmulet [amuletCid] treasuryShare handler treasury dso now
    validityPeriod amuletCtx
    pure changeCids
  else pure [amuletCid]
if null escrowCids && Map.null remainingPayouts
then pure ()
else
  distributeAmulet
  escrowCids
```

H-06 Expired fee escrow poisons commit batches

CONTINUED

```
remainingPayouts
handler
sender
dso
now
validityPeriod
amuletCtx
refundAddress
```

Once the DSO expires the Amulet, these operations cannot fetch the holding. Both Accept and Reject abort, so the `Request` remains active. This does not require malicious behavior by the trusted DSO or deployer.

An ordinary sender can reach this state cheaply when the request minimum fee is zero or low.

`RequestFactoryConfig` permits `minFee = 0` and the repository's integration deployment helper creates that configuration by default. Although a request still needs a positive fee, the sender chooses its amount. At a one-dollar CC price, the configured `0.0000190259` USD-per-round holding fee makes a `0.00001` CC escrow expiry-eligible after one round. Exploitation requires the request to remain pending until the DSO reference state advances and dust cleanup runs. A sender can target that maintenance boundary and create a backlog of low-value requests. A materially higher production minimum fee lengthens this window but does not provide a terminal recovery mechanism.

`commitBatchImpl` settles every entry and creates the next state commitment in one atomic transaction. Therefore one request with an expired escrow rolls back healthy entries in the same batch and preserves the previous commitment. The request poller continues returning the still-active contract from the same persisted range and the sequencer retries while any request remains pending. Later cross-chain requests can remain locked until operators patch or bypass the poisoned request. Unlike the existing base-unit payout issue, manually choosing Reject does not recover this request because both settlement decisions need the inactive escrow.

— RECOMMENDATION

Enforce an escrow-lifetime invariant when creating a request and add a terminal cleanup choice that does not fetch an already expired holding. For example, record enough authenticated expiry data in `Request` to let the handler archive a request after its escrow expires, then make the off-chain processor deterministically quarantine that request instead of combining it with healthy entries. Also require the escrow's remaining lifetime to exceed a configured maximum settlement and recovery interval. Raising `minFee` alone only delays the failure and is not a complete fix.

H-06 Expired fee escrow poisons commit batches

CONTINUED

— RESOLUTION

Status: Resolved.

HIGH **H-07**

RESOLVED

Forged accept callback burns victim payout pool

CATEGORY	Access Control
LOCATION	contracts/protocol/canton/contracts/Layerzero/Oft/daml/OApp/Callback/Dispatcher.daml:61-72
REVIEW	Remediation Review 1

— DESCRIPTION

When an LzSend accept callback is finalized, the dispatcher parses two holding references out of the untrusted callback context: the main locked cross-chain holding and the locked payout pool. It asserts that the request sender owns the main holding via `assertSenderOwnsHolding`, but applies no equivalent ownership check to the payout pool. The pool is then passed to `requireLzSendPayoutPool`, which validates only that the referenced holding is locked and that its amount equals the caller-supplied payout total — never who owns it or which send created it.

Because an OFT holding is signed only by the token admin (the owner is a mere observer), the subsequent `BurnMintFactory_BurnMint` archives the referenced pool using the factory's admin authority alone; the victim owner's authority is not required. A party holding the `_ENDPOINT_DELEGATE_ROLE` — which gates request creation and is not the admin — can therefore forge a callback that names its own small locked holding as the main holding (passing the ownership check) while naming a victim's locked pool as the payout pool, with the payout amounts set so their sum matches the pool's amount. On accept and finalize, the victim's pool is burned and fresh unlocked holdings are minted to attacker-chosen receivers. Any locked OFT holding of a matching amount is a valid target.

The attacker must be able to reference the victim's locked-pool contract id — disclosed to the gateway and observers when the original send locks the pool — and the mint receivers must be on the OApp allowlist. The destruction of the victim's locked funds is itself unconstrained.

— RECOMMENDATION

H-07 Forged accept callback burns victim payout pool

CONTINUED

Bind the payout pool to the request sender the same way the main holding is bound: fetch the pool and assert its owner equals the pending callback's request sender before it is passed to `distributeLzSendPayouts`, or carry the pool reference from trusted call-time state created during the original send rather than accepting it from the callback context. More broadly, the burn path should validate the owner of every input holding it destroys rather than relying on the admin-only signatory model.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/ForgedPayoutPoolBurn.daml>

— RESOLUTION

Status: Resolved.

HIGH **H-08**

RESOLVED

Delegate forges callback to mint unbacked OFT

CATEGORY	Access Control
LOCATION	contracts/protocol/canton/contracts/Layerzero/Oft/daml/OApp/Callback/Dispatcher.daml:77-80
REVIEW	Remediation Review 1

DESCRIPTION

A holder of `_ENDPOINT_DELEGATE_ROLE` — a role that is neither the OApp admin nor a minter — can create OFT supply from nothing. `OftSelf.oApp_createRequestImpl` validates the config, timestamp and delegate role, then forwards the caller's request verbatim through `createGenericRequest` into `RequestFactory.request_CreateImpl`. That factory validates the routing identity of `callContext` but only size-checks `callbackContext`; there is no function-name allowlist and no cross-binding between the two. The delegate therefore supplies a benign `callContext` that the off-chain validator accepts while embedding a forged `LzReceive_callback` payload — a receiver fingerprint they control and an arbitrary amount — in `callbackContext`.

On accept, `dispatchAccept` parses the `callbackContext` and, seeing the `LzReceive_callback` key, routes to `handleLzReceiveAccept` and `lzReceive`, whose `create Oft` issues brand-new holdings with no source burn, no lock, no allowlist check and no peer/origin verification. Unlike the `LzSend` branch of the same dispatcher, which applies `assertSenderOwnsHolding`, the `LzReceive` branch applies no ownership or provenance check. `fetchAndVerifyPendingCallback` enforces only that the admin is a signatory, the `appUID` matches and `isAccept` is true — it binds nothing to an endpoint-verified message, nonce or GUID, and the admin's authority reaches the mint through the signatory chain, so no live admin action is required. The peer/nonce anti-spoof check in `validateLzReceiveRequest` exists only on the legitimate call path and is entirely bypassed.

A non-admin, non-minter principal can thus mint OFT to a receiver they register, capped only by the inbound rate limiter per window and entirely uncapped when rate limiting is globally disabled. This breaks the supply-backing invariant; in open or blacklist allowlist modes the tokens are immediately transferable and can be bridged out to draw down genuine cross-chain backing.

H-08 Delegate forges callback to mint unbacked OFT

CONTINUED

RECOMMENDATION

Bind the callbackContext to an endpoint-verified inbound message before minting on the accept path: require the LzReceive callback to carry, and verify, the nonce, GUID and payload hash the endpoint recorded at receive time, and apply the same peer/origin check that `validateLzReceiveRequest` performs on the call path. Better, enforce this at the shared generic `OApp_CreateRequest` / `createGenericRequest` path — reject callbackContexts carrying a settlement-callback function (or self-targeting callbacks) there — so a single fix protects every OApp implementation that routes callbacks through this hatch (both the OFT mint path and the adapter custody path), not just one. More broadly, the accept path should never act on a holding-affecting instruction that was not produced by a verified inbound-message flow.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/ForgedReceiveMintsUnbackedOft.daml>

RESOLUTION

Status: Resolved.

HIGH **H-09**

RESOLVED

Revoked delegate retains endpoint authority

CATEGORY Access Control

LOCATION contracts/protocol/ver-endpoint/src/endpoint-base/index.ts:44-60

REVIEW Remediation Review 1

DESCRIPTION

The generic request hatch (`oApp_createRequestImpl`) lets an `_ENDPOINT_DELEGATE_ROLE` holder submit a caller-selected endpoint function under the OApp's app UID, and it does not exclude `setDelegate`. The runtime endpoint stores the supplied `oapp -> delegate` entry in its own `#delegates` MapState, and every later `assertAuthorized` call trusts that entry with no expiry and no check against the current on-ledger role assignment.

A delegate can set the runtime delegate to the global address derived from its own Canton party. The admin can then revoke `_ENDPOINT_DELEGATE_ROLE` through the normal `AccessControl_RevokeRoles` choice — but that revocation only updates the on-ledger role registry and cannot touch the runtime `#delegates` map, which is decoupled. Afterward the former delegate calls the public non-OApp `Request_Create` with `oAppInput = None`; that path binds the caller address to `partyToText(sender)`, which derives to the same global address stored as the delegate, so `assertAuthorized` still passes for the victim OApp even though the party now holds no role.

The now-unprivileged party can therefore call `skip`, `clear`, `nilify`, `burn`, and the library and verification-config setters, or rotate the runtime delegate again. For example, skipping the next inbound nonce after a source transfer has locked or burned value makes verification of the real packet fail, so the destination can never unlock the corresponding funds. Resetting the on-ledger role does not repair an already-skipped message; the admin must separately clear the runtime delegate and recover or compensate the affected transfer. Only the initial `setDelegate` uses the role — every later destructive request comes from the public non-OApp interface after a correct revocation.

RECOMMENDATION

Do not allow `_ENDPOINT_DELEGATE_ROLE` holders to call `setDelegate` through generic requests; expose a typed, admin-only operation if runtime delegation is required. Bind the runtime authorization credential to a lifecycle that follows the current on-ledger role assignment, or explicitly clear the runtime `#delegates` entry whenever the role is revoked, so revocation actually removes runtime authority. Additionally, prevent non-OApp

H-09 Revoked delegate retains endpoint authority

CONTINUED

`Request_Create` calls from targeting endpoint methods that authorize against an OApp or mutate its receive and verification state.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/ver-endpoint/test/Guardian/revoked-delegate-retains-authority.test.ts>

— RESOLUTION

Status: Resolved.

HIGH **H-10**

RESOLVED

Prototype dispatch omits state commitments

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/container/component.ts
REVIEW	Remediation Review 1

DESCRIPTION

The runtime API is built from ordinary JavaScript objects and `ContainerComponent.#resolveMethod` reads `methodMap[functionName]` without checking that the key is an own property. An ordinary sender can reach that lookup through the public non-OApp `Request_Create` choice: the factory binds the caller address and validates the target address, but it does not restrict the requested contract or function name. In the checked-in native build, the endpoint is registered as `_EndpointV2`. A non-OApp request for `_EndpointV2.toString` therefore reads inherited `Object.prototype.toString` values from `Endpoint` and its subcomponents. The resolver sees multiple apparent matches and throws raw `Error: Function signature is ambiguous` before the normal `UserError` conversion.

The shipped settlement integration turns that in-scope dispatch fault into a commitment-lineage gap. `RequestExecutor` catches the raw error and returns a `Reject` without a `ContainerUpdate`. Its `#finalize` returns immediately when the complete batch has no updates, assuming that an unchanged virtual root means no Canton commitment was created. The in-scope Daml `commitBatchImpl` does the opposite: every nonempty batch accepts or rejects its `Requests`, archives the previous `StateCommitment` and creates a successor, including when every entry is rejected and `stateRoot` is unchanged.

Suppose the persisted and on-ledger head is C0 with root A. Canton successfully commits the `Reject`, archives its `Request` and C0 and creates same-root successor C1. The zero-update finalizer stores nothing, so C1 is absent locally. A later valid `Request` can still execute because local lookup by root A finds C0. The attacker can supply this step too—for example, the PoC uses an authorized `setDelegate` update for the attacker's own runtime address. Canton then creates changed-root C2 with `C2.previousId = C1`. The production PostgreSQL table rejects C2 with SQLSTATE `23503` because its parent C1 is missing. Canton now advertises C2's root B while the validator database has no commitment for B, so every later `WRITE` fails before execution with `PreviousStateCommitmentNotFoundError`.

The shipped automatic recovery path cannot repair the gap. `StateRecoverer` walks `C2 -> C1 -> C0` and tries to replay C1 first, but successful `CommitBatch` already archived C1's `Request` and `RequestSdk.getRequest` uses an active-only Canton lookup. Replay therefore

H-10 Prototype dispatch omits state commitments

CONTINUED

aborts with `RequestFetchError` before inserting C1. Even a history-aware Request fetch would need a fixed finalizer, because replaying the same raw Reject again produces zero updates and the current `#finalize([])` again stores nothing. Canton retains historical events, so a patched import path or manual database repair can recover service; this is not permanent data loss.

A fee-paying sender needs no OApp role, endpoint-delegate role, administrator, malicious package or compromised key. The triggering batch must contain no `ContainerUpdate`: any successful call or handled `UserError` in the same batch causes the single batch commitment to be persisted. An attacker can target an idle batch or make every selected entry use a raw-error trigger. C1's omission is initially dormant because C0 and C1 share root A; the durable halt begins after a later state-changing C2 commits and fails local persistence. The immediate parent-FK halt applies to PostgreSQL-backed validators; the memory repository does not enforce that foreign key.

— RECOMMENDATION

Make API lookup use `Object.hasOwn(methodMap, functionName)` or null-prototype maps and convert resolver failures into deterministic handled errors. Use stable explicit protocol contract identifiers instead of compiler-generated `Function.name` values. These changes remove the demonstrated unprivileged trigger.

As the necessary cross-layer safety fix, persist the on-ledger `StateCommitment` for every successful nonempty `CommitBatch`, including batches with no runtime updates. Pass a committed entry's Request ID into `#finalize`, poll the commitment by that ID, verify its root even when unchanged and insert it before any child, Request or event records. Apply the same rule during recovery and add a history-aware way to replay or import archived Requests. Trigger hardening alone is insufficient because invalid fetched Requests and future raw exceptions can also produce batches without updates.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/commit/6e57dae329ee7a6a73c9a670e51352cf25b55e11>

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

Payout fees consume cross-chain backing

CATEGORY	Compatibility
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzSend/Call back/Accept.daml
REVIEW	Remediation Review 1

DESCRIPTION

`adapterLzSendImpl` locks exactly the cross-chain amount plus the nominal contingent payouts. After the request is accepted, `handleAdapterLzSendAccept` gives the treasury holdings to `distributePayouts` and assumes that only those nominal payouts are removed, so the cross-chain amount remains in custody. The handler does not read the frozen `crossChainAmount` or verify the value left in the returned sender-change holdings.

The codebase supports two other fees, but neither covers this case. `feeAmount` pays the RequestFactory protocol fee in Amulet. `AdapterConfig.adapterFeeBpsPerEid` defines an adapter fee that is calculated before the send and included in the nominal contingent payouts. `AdapterConfig` explicitly says that fee is unrelated to the wrapped-token issuer. There is no equivalent setting for a fee charged inside the wrapped token's `TransferFactory`: no compatibility flag, fee quote, maximum fee, separate reserve or post-transfer reconciliation.

Nothing in the adapter restricts it to Amulet or another fee-free instrument. `LockUnlockAdapter` describes itself as a generic wrapper for an existing token. Its package imports the generic `Splice Holding` and `TransferFactory` interfaces rather than a concrete token implementation. `AdapterConfig` accepts an arbitrary `assetInstrumentId` and its `ensure` clause has no fee-free restriction. The audit scope document likewise describes "underlying-token generality," with the external token's own factory supplied at runtime. Its stated custody invariant is that the treasury must remain at least equal to the outstanding cross-chain liability.

The exact Canton Token Standard dependency compiled into the adapter explicitly permits fee-bearing results. In

```
contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/.daml/dependencies/2.2/55ba4deb0ad4662c4168b39859738a0e91388d252286480c7331b3f71a517281/splice-api-token-transfer-instruction-v1-1.0.0-55ba4deb0ad4662c4168b39859738a0e91388d252286480c7331b3f71a517281/Splice/Api/Token/TransferInstructionV1.daml:45-55
```

, `TransferInstructionResult.meta` is defined as extensibility metadata and names charged fees as an example. Lines 71-73 also state that a failed instruction may return the sender's holdings minus fees. Therefore, a token issuer can legitimately deduct fees and report them through the

M-01 Payout fees consume cross-chain backing

CONTINUED

standard result. This behavior does not require a malicious package or fabricated transfer result.

The historical customer comments assign TransferFactory policy to the asset issuer and treat multiple issuer-defined contexts as admissible. The exact fee policy can therefore be chosen by each issuer. An honest issuer-charging factory fits the accepted package-vetting and factory-policy model; it is not a malicious-DAR case.

The only fee policy encoded in the transfer helpers is the opposite assumption. `Transfer.Distribute` says CIP-78 guarantees exact value conservation, then subtracts only each requested payout from its local `remaining` value without comparing that value with the actual change holdings. CIP-78 removed transfer fees from Canton Coin/Amulet specifically. It does not make every token-standard implementation fee-free. `Transfer.Core` also ignores `TransferInstructionResult.meta` and never verifies how much value remains in the returned change CIDs.

The repository's current implementations and tests do not expose this mismatch. The bundled Amulet configuration requires transfer fees to be zero after CIP-78, `OfSelf` splits holdings exactly and returns empty result metadata and the LockUnlockAdapter fixture tests only Amulet using a zero-transfer-fee configuration. The resulting compatibility rule is therefore fee-free-only by an unenforced assumption: fee-bearing tokens are not safely supported, but they are not rejected or documented as unsupported either.

An ordinary sender can amplify this mismatch because the adapter permits 50 distinct payout recipients. The PoC uses an honest issuer-signed factory that verifies the expected admin, archives every input holding, gives every receiver exactly the requested amount, returns genuine sender-change holdings and reports its fixed fee in result metadata. With six decimals, the sender submits `50.00005` tokens: 50 tokens for the cross-chain message and fifty one-base-unit payouts. The sender pays one token for the initial treasury transfer. When the accepted callback distributes the 50 tiny payouts, the factory charges 50 one-token fees to the treasury. Finalization succeeds and archives the callback, but the treasury falls from `50.00005` to zero even though the callback committed a 50-token cross-chain amount.

Consequently, a valid packet can release 50 tokens on the destination while its source backing has been consumed as transfer fees. The sender can sell the destination liquidity before the deficit is realized. Later receives or rejected-send refunds can fail or consume other users' pooled backing. This does not require a malicious package, a fabricated transfer result or admin abuse; the factory applies its documented issuer policy correctly.

Example:

M-01 Payout fees consume cross-chain backing

CONTINUED

1. An ordinary sender requests a 50-token cross-chain transfer and supplies the maximum 50 distinct one-base-unit payout rows.
2. The dedicated send honestly transfers the 50-token principal and nominal payouts to treasury, paying one source-transfer fee.
3. The endpoint accepts the normal request and creates its genuine `PendingCallback`.
4. Callback finalization executes 50 exact issuer-authorized payout transfers. Each charges one token from treasury change.
5. The callback archives with no treasury asset left, while the destination packet still commits the full 50-token amount.
6. The sender can realize the unbacked destination value; subsequent users encounter failed settlement or loss of pooled backing.

— RECOMMENDATION

Either restrict the adapter to issuer-authenticated fee-free asset factories or account for transfer fees explicitly. Enforce the restriction on ledger by fetching and summing the actual sender-change holdings after every payout, then abort unless their value decreased by exactly the requested payout and the frozen cross-chain amount remains backed.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/PayoutFeesConsumeBacking.daml>

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

ACKNOWLEDGED

Amulet expiry can leave live claims unbacked

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzSend/Call/Impl.daml
REVIEW	Remediation Review 1

DESCRIPTION

`adapterLzSendImpl` establishes a remote obligation without recording the lifetime of the treasury holdings that back it. It transfers the cross-chain amount, plus any contingent payouts, into the treasury. It then discards the CIDs and lifecycle data of the returned holdings. The outbound message and callback retain only the fixed nominal amount. The adapter also keeps no aggregate record of outstanding claims that can be compared with the usable treasury balance. When that amount is redeemed later, the finalizer supplies the active treasury holding CIDs to be used for settlement through `executeContext`.

The failure is straightforward. A zero-payout outbound send creates an Amulet holding in the treasury and a remote claim for the same nominal amount. Accepting the send does not transfer that holding again, so it can remain untouched. If the DSO later archives it with `Amulet_ExpireV2`, the remote claim is unchanged because the adapter never linked the claim to the holding's expiry. A later redemption must then use other active treasury holdings, including holdings that may back other outstanding claims. If the treasury does not have enough, callback finalization fails with `ERR_TRANSFER_INSUFFICIENT_HOLDINGS`. The callback stays active, but the receiver gets no transfer instruction.

This can happen because Amulet does not guarantee that a holding remains available indefinitely. Each Amulet transfer output stores an `ExpiringAmount`. Its expiry is based on the output's US-dollar value when it is created:

```
expiryRound = createdAt + ceil(USD value / 0.0000190259 USD per round)
```

At the default ten-minute round interval, a holding worth one dollar reaches mathematical expiry after about one year. For example, at a price of one dollar per CC, a `0.01` CC output reaches mathematical expiry after 526 rounds or about 3.65 days. The CC-denominated rate calculated from the creation-time price, together with the creation round, is stored in the holding. Later CC price changes therefore do not alter its expiry. Amulet's [output construction](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/AmuletRules.daml#L1235-L1254), [price conversion](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/AmuletRules.daml#L1560-L1583) and [expiry

M-02 Amulet expiry can leave live claims unbacked

CONTINUED

calculation](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/Expiry.daml#L116-L149) implement this behavior. The [default rate and round interval] (https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/apps/common/src/main/scala/org/lfdecentralizedtrust/splice/util/SpliceUtil.scala#L174-L195) produce the approximate one-dollar-per-year lifetime.

Reaching the expiry round does not reduce the holding's displayed amount or automatically archive it. A valid transfer can still consume the holding and recreate its full nominal value. Actual removal happens only when `Amulet_ExpireV2` is exercised through the DSO-authorized process, normally by an authorized SV expiry service. That choice archives the holding without creating a replacement. Under normal V2 config-state rotation, the holding usually becomes archivable about 24 to 48 hours after its mathematical expiry. An ordinary user cannot exercise this choice. The relevant rules are the [`Amulet_ExpireV2`] (https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/Amulet.daml#L121-L158)[choice](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/Amulet.daml#L121-L158), its [archive-only implementation] (https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/AmuletRules.daml#L1268-L1303) and the [two-state eligibility check] (https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/daml/splice-amulet/daml/Splice/AmuletRules.daml#L948-L972).

All of the following conditions are required. The deployed `AdapterConfig` must use Amulet or another instrument with equivalent destructive expiry. `Amulet_ExpireV2` must actually be exercised through the DSO-authorized process using the required eligible DSO-signed config states. Enough participants hosting the treasury party must be online, synchronized and have the relevant packages vetted so Canton can confirm that transaction.

The affected holding must remain unconsumed and unrefreshed until it is archived. There must be no effective maintenance process that refreshes every reserve holding, including a singleton, before it becomes eligible for archival. Finally, enough short-lived value must be archived to create a material deficit.

The repository proves the contract behavior, but it does not prove those deployment conditions. The production adapter uses generic `Holding` and `TransferFactory` interfaces. No live Amulet-backed `AdapterConfig` or `LockUnlockAdapter` deployment manifest is present and the available script deploys the separate burn/mint `0ft` implementation. The exact Splice `v0.6.10` SV image also [ships

M-02 Amulet expiry can leave live claims unbacked

CONTINUED

](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/cluster/images/sv-app/app.conf#L64-L73)[ExpiredAmuletTrigger](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/cluster/images/sv-app/app.conf#L64-L73)[paused](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/cluster/images/sv-app/app.conf#L64-L73). A deployment operator can override or resume the trigger, while an authorized SV/DSO process can also submit expiry manually. When enabled, the [trigger selects eligible holdings](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/apps/sv/src/main/scala/org/lfdecentralizedtrust/splice/sv/automation/delegatebased/ExpiredAmuletTrigger.scala#L23-L130). The actual production setting is unknown.

Reserve maintenance is also deployment-specific. A bare adapter treasury has no built-in refresh service. A standard wallet can merge multiple Amulet holdings when the treasury has a `WalletAppInstall`, the validator can act as the treasury and automatic merging is enabled. However, the wallet deliberately does nothing when only one holding exists and there are no rewards. External-party automation has its own delegation and holding-count requirements. Ordinary inbound transfers and nonempty payouts can also refresh the particular holdings they consume. These mechanisms reduce the likelihood of expiry, but none is a mandatory adapter-level guarantee that every reserve holding will be refreshed. See the [wallet install trigger](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/apps/wallet/src/main/scala/org/lfdecentralizedtrust/splice/wallet/automation/WalletAppInstallTrigger.scala#L46-L80), [singleton behavior](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/apps/wallet/src/main/scala/org/lfdecentralizedtrust/splice/wallet/treasury/TreasuryService.scala#L730-L736) and [external-party automation setup](https://github.com/hyperledger-labs/splice/blob/63cfb340ce0f8f254386d2d5df58905d695de902/apps/wallet/src/main/scala/org/lfdecentralizedtrust/splice/wallet/automation/ExternalPartyWalletAutomationService.scala#L63-L71).

The adapter places no lifecycle-aware minimum on reserve outputs, so repeated zero-payout sends can create many short-lived holdings. After an honest Amulet configuration, ordinary senders can create these outputs through normal sends; no malicious DAR, OApp administrator or treasury administrator is required. The sends remain subject to configured fees and limits.

The repository does not show that material fragmentation is economical. Archiving 100 dollars through one-cent outputs would require about ten thousand sends. Archiving 100,000 dollars through one-dollar outputs would require about one hundred thousand sends. Production fees, execution costs, cost assertions, rate limits and service policy are unknown. The PoC itself pays a `1.0` CC Request fee to create a `0.01` CC holding at a fixture price of one dollar per CC. It proves the state transition, not a cheap attack.

M-02 Amulet expiry can leave live claims unbacked

CONTINUED

If the required conditions hold, authorized expiry permanently removes part of the treasury's backing while the corresponding claims remain valid. Later redemptions can consume backing deposited for other claims or fail until the treasury is recapitalized or liabilities are reduced. The broken invariant is therefore:

```
safely spendable treasury backing >= outstanding nominal cross-chain liabilities
```

The potential deficit is material, but the asset choice, actual expiry submission, affected holdings remaining unrefreshed and practical economics are all unconfirmed.

— RECOMMENDATION

Do not use the lock/unlock adapter with an expiring instrument unless reserve maintenance is mandatory and monitored. Prefer the burn/mint design or a non-expiring custody wrapper for Amulet.

If Amulet custody is required, record the aggregate outstanding liability and the lifecycle data of every holding returned to the treasury. A maintenance service should refresh or consolidate every reserve holding, including a singleton, before a conservative safety deadline. Exclude holdings that are too close to expiry from the safely spendable reserve balance. Pause new sends when maintenance is unhealthy or when safe backing falls below outstanding claims. Operators should also be alerted when the DSO expiry trigger or treasury maintenance setting changes.

A minimum transfer amount is not sufficient because an untouched holding with a nonzero fee still becomes eligible for expiry eventually. Zero-rate holdings and amounts whose calculated expiry exceeds the representable maximum are separate cases. Authenticate replacement holdings using the expected interface, instrument ID, instrument administrator and DSO identity rather than an upgrade-unstable CID.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Validator build rejects Canton OApp requests

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-protocol-common/src/contract-like.ts
REVIEW	Remediation Review 1

DESCRIPTION

`ContractLike.getName()` uses `this.constructor.name` as a protocol identifier. `ContainerComponent` also keys its contract-class registry by `ContractClass.name`, while `endpointVappApi` uses `EndpointV2.name`. These JavaScript names are compiler-controlled and are not stable identifiers.

The shared `tsup` configuration enables splitting and tree-shaking without preserving class names. The native `ver-endpoint` build therefore emits `var EndpointV2 = class _EndpointV2 extends Contract`, making both `EndpointV2.name` and an endpoint instance's `constructor.name` equal to `_EndpointV2`. The checked-in `ver-vapp` Dockerfile runs the affected workspace build and starts the compiled `dist` application. Turbo's `^build` dependency also compiles `ver-vapp`'s workspace dependencies, including `ver-endpoint`. This confirms the mismatch for the repository's supplied validator Docker build rather than only for an artificial test configuration.

The Daml packages use the fixed target name `EndpointV2` when constructing OApp send and receive call contexts. `RequestFactoryValidation` also requires that exact name. The Canton converters copy the target name into `functionSignature.contract` without normalization. During genesis, however, the affected validator stores `_EndpointV2` in the component registry. Dispatch later compares the stored name with the incoming name using exact string equality:

```
registered name: _EndpointV2
requested name: EndpointV2
result: Contract not found
```

Consequently, every Canton OApp send and receive request processed by that build is rejected before the requested endpoint function runs. This affects fresh deployments and does not require an attacker, admin mistake, malicious DAR or package-vetting failure.

The audit snapshot does not include CI release configuration, Helm charts, Kubernetes manifests or another operator deployment record proving that a real production environment will deploy this exact image. The failure is therefore confirmed for the checked-in validator

M-03 Validator build rejects Canton OApp requests

CONTINUED

Docker build, but the final production artifact remains a deployment assumption. Deploying only the Daml packages does not trigger the rename; building and running the compiled validator service does.

If operators deploy the supplied image, the failure is deterministic and does not require an attacker. The validator cannot process genuine Canton OApp sends or receives. Outbound requests may be rejected after assets have entered adapter custody, forcing users to wait for the existing rejection and refund process and potentially incur additional costs. Inbound delivery remains unavailable until compatible validator software is deployed and the message is retried.

A local test initialized the real Endpoint component with the native `ver-endpoint` build. It observed `EndpointV2.name` as `_EndpointV2`, returned `Contract not found` for the Daml literal and accepted the otherwise identical call when the target was changed to `_EndpointV2`. This isolates the failure to name resolution before endpoint business logic.

— RECOMMENDATION

Define an explicit immutable protocol name for every runtime contract and use it consistently in the contract registry, vApp API and Daml call context. Do not derive serialized or persisted identifiers from `Function.name` or `constructor.name`. Enabling compiler name preservation may be useful as defense in depth, but it should not be the compatibility mechanism.

Add a production-bundle integration test that imports the compiled package through its production export, initializes the real `EndpointComponent` and successfully dispatches the literal Daml target `EndpointV2`. Add the same check as a smoke test for the final validator Docker image.

— RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

Rejected calls persist orphaned trie nodes

CATEGORY DoS

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/container/component.ts, packages/protocol/lz-ver-protocol/ver-api-common/src/container/container.ts

REVIEW Remediation Review 1

DESCRIPTION

`ContainerComponent.call` executes a contract against a shallow trie copy. It does not open a checkpoint or transaction-local write overlay before execution:

```
const trie = this.#trie.shallowCopy();
trie.root(hexToBytes(state));
```

State writes therefore reach the shared backing database immediately. `MapState.set` calls `Trie.put` directly:

```
async set(key: K, value: V): Promise<void> {
  await this.#trie.put(this.#deriveSlot(key), z.encode(this.#valueCodec, value));
}
```

ULN makes the issue easy to amplify because it stores a raw receive configuration before validating the resolved configuration:

```
// Set the receiveUlnConfig storage
await this.#receiveUlnConfig.set([oapp, srcEid], config);
// Verify the resolved config is valid by calling get0appUlnReceiveConfig
await this.#get0appUlnReceiveConfig(srcEid, oapp);
```

When the later validation throws a `UserError`, `Container.call` returns the state captured before component execution. It does not revert the database writes already made through the shallow copy:

```
} catch (error) {
  if (error instanceof UserError) {
    this.#assertContextSnapshot(context, snapshot);
    return { error, state, nonce };
  }
  throw error;
}
```

M-04 Rejected calls persist orphaned trie nodes

CONTINUED

Consequently, the failed component state disappears logically because the returned root does not reference it. The content-addressed trie nodes nevertheless remain in the physical database.

An ordinary Canton Party can amplify this without an OApp, endpoint-delegate, admin, validator or package-upload role. A non-OApp `Request_Create` lets the sender select the runtime target, function and arguments while binding the runtime caller address to that sender. The Party can therefore call `EndpointV2.setReceiveConfig` for its own derived runtime address. The endpoint accepts this self-authorization and forwards the complete parameter array to ULN-302.

The sender supplies many distinct, valid same-key overwrites followed by a configuration that disables all defaults while selecting no DVN. ULN writes the final item and only then rejects its resolved configuration. The container exposes none of the supplied values through the returned state root, but all database nodes created during the rejected call remain durable.

The paired PoCs use the exact chain-domain address of an allocated ordinary Party. The Daml Script proves that `RequestFactory` accepts a 33-element batch within its 32 KiB and 256-node limits. The runtime PoC executes 32 distinct valid overwrites plus the post-write-invalid tail against the full Endpoint + ULN + Container topology. A control rejection before any ULN write added 4 rows and 429 key/value bytes for the deliberately committed nonce. The attack added 103 rows and 32,571 bytes, leaving an excess 99 unreachable rows and 32,142 bytes. A second request with different ignored confirmation words reproduced the same growth exactly, while reads from both returned roots showed the original empty OApp configuration.

Because the fee is charged once per Request rather than per runtime write, a fee-paying sender can repeat this process with varied ignored fields and make every validator retain linearly growing state that no committed root references. Filling or materially degrading the PostgreSQL trie volume eventually makes affected validators unable to execute and sign new state transitions. If enough validators are affected, settlement quorum stops until operators expand or repair storage.

POCs:

- Runtime storage PoC: <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/packages/protocol/lz-ver-protocol/ver-api-common/test/Guardian/rolled-back-trie-writes-persist.test.ts>
- Daml reachability PoC: <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/RolledBackTrieWriteReachability.daml>

M-04 Rejected calls persist orphaned trie nodes

CONTINUED

RECOMMENDATION

Execute the complete `Container.call` continuation chain against one transaction-local trie write overlay shared by the container and every component. Flush it only after all component calls, continuations, response encoding, event hashing and context checks succeed. Discard the overlay on every error while retaining the intentionally committed nonce transition separately.

Do not fix only ULN's validation order: any callable method that writes before throwing or any successful early continuation followed by a failing later continuation, recreates the same leak. Add per-request execution/storage metering as defense in depth and provide an operator-safe garbage-collection procedure for existing nodes that are unreachable from every retained state commitment.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/commit/a0a5a7f8dce7ded34da0174311705524e943e94a>

RESOLUTION

Status: Resolved.

MEDIUM **M-05**

ACKNOWLEDGED

Underpriced fees enable rate-limit exhaustion

CATEGORY	DoS
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/daml/0App/CostAsserts.daml
REVIEW	Remediation Review 1

DESCRIPTION

The adapter locks tokens on Canton and asks a separate runtime to send the cross-chain message. Before doing so, it uses `CostAsserts` to check that the sender supplied enough Canton Coin to pay for the runtime execution. This check runs before the adapter locks the tokens and consumes capacity from the shared outbound rate limiter.

`CostAsserts` does not calculate the complete runtime fee. It charges only for the receive and compose gas written into the options by the sender, plus any native value included in those options. The runtime also charges mandatory receive base gas. It can add base gas for every compose operation, a 2% surcharge for ordered execution, calldata costs, DVN fees, treasury fees and other configured premiums. The Daml configuration and decoded options do not contain all of this information, so the on-ledger check cannot reproduce the fee later enforced by `EndpointV2`.

Consequently, a sender can pay a fee that passes `CostAsserts` even though the runtime is guaranteed to reject the request as underfunded. By that time, the adapter has already locked the sender's tokens, recorded the full transfer against the shared rate limit and created the asynchronous `Request`. Rejecting that `Request` only creates a `PendingCallback`. The tokens and rate-limit charge remain in place until a separate transaction finalizes the callback.

Example:

- The outbound rate limit is 1,000,000 tokens.
- A sender requests one unit of receive gas with ordered execution and supplies the 1 CC fee accepted by `CostAsserts`.
- The adapter locks all 1,000,000 tokens, consumes the complete outbound limit and creates the `Request`.
- The runtime adds 100 units of mandatory receive base gas and the ordered-execution surcharge. It therefore charges 103 CC rather than 1 CC.
- `EndpointV2` rejects the request because only 1 CC was supplied.

M-05 Underpriced fees enable rate-limit exhaustion

CONTINUED

- Rejecting the `Request` does not immediately release the tokens or rate-limit capacity. Other users have no initial capacity in that bucket until it decays or the callback is finalized.
- Callback finalization creates the refund and reverses the remaining rate-limit charge. The sender accepts the refund, recovers the full principal and can repeat the same cycle.

The attached POCs reproduce this exact example. The Daml test accepts the 1 CC threshold, fills the complete 1,000,000-token bucket and proves that the lock and rate-limit usage survive `Request` rejection. The runtime test calculates a 103 CC fee for the same options and proves that `EndpointV2` rejects the 1 CC payment.

This lets an ordinary sender temporarily deny shared outbound capacity without an administrator role, malicious package, fake interface or compromised key.

— RECOMMENDATION

Verify the complete runtime fee before recording the outflow or locking the sender's tokens. The strongest approach is a short-lived runtime quote that is bound to the sender, destination, exact message and options, fee, pricing version and expiry. The adapter should verify that quote before calling `RecordOutflow`.

If the fee is calculated directly in Daml instead, the Daml and runtime implementations must use the same versioned formula. It must include receive base gas, compose base gas and compose count, ordered execution, calldata, premiums, DVN fees and treasury fees. Run the same fee vectors through both implementations in CI and require the Daml threshold to cover the complete endpoint invoice.

As defense in depth, a deterministic insufficient-fee rejection should release its rate-limit reservation during settlement instead of waiting for a separate application callback.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/commit/94c9d681359892202c6d653a290e8aab3d18b82f>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

RESOLVED

Appended ULNs can duplicate worker addresses

CATEGORY	DoS
LOCATION	contracts/protocol/ver-endpoint/src/message-library/uln/uln-302/uln-302.ts
REVIEW	Remediation Review 1

DESCRIPTION

A ULN is a message-library component that manages workers used to verify and execute cross-chain messages. A DVN is one of those workers: it records whether a packet has received the required verification. Each ULN component keeps separate worker state, while the top-level container routes calls using the workers' public addresses.

Neither the ULN owner nor an allowlisted deployer chooses a DVN name or assigns the string `dvN-0`. The caller chooses the DVN's payee, VID, price feed, signer set, quorum and administrator list. `registerDvn` does not accept a worker name, address or nonce. The code generates the address internally.

`#deriveWorkerAddress` combines a hard-coded worker-type hint with `#workerNonce`, a counter stored separately inside each `Uln302` instance:

```
worker address = hash(worker type + "-" + this ULN's worker nonce)
```

A newly created ULN has its own counter starting at zero. Consequently, a DVN that consumes nonce zero receives the address derived from `dvN-0`, regardless of its VID, administrators or signer committee. A second ULN whose DVN also consumes nonce zero receives exactly the same address. The duplicate is produced by the internal allocator, not by an administrator reusing a configured name.

The counter is shared by DVNs, executors and price feeds, so the exact suffix depends on registration order. For example, registering a DVN first produces `dvN-0`, while registering a price feed before it produces `price-feed-0` followed by `dvN-1`. A second ULN therefore does not collide under every possible registration order. The collision occurs when both ULNs register the same worker type at the same local nonce. Repeating the existing ULN's provisioning sequence reproduces each type-and-nonce pair and deterministically recreates its worker addresses.

Only the ULN owner or an allowlisted deployer can register these workers. This is therefore an operator-triggered upgrade failure, not an attack in which an untrusted caller selects an existing worker name. However, the registration API gives operators no unique namespace to supply and does not reject a duplicate against container-wide state. Avoiding the

M-06 Appended ULNs can duplicate worker addresses

CONTINUED

failure by changing registration order or creating dummy workers would be a fragile operational workaround, not a uniqueness guarantee.

Example:

- ULN A is provisioned with a worker sequence in which one of its DVNs consumes local nonce `n`.
- A later append-only upgrade adds ULN B with different static addresses, owners, VIDs and signer committees.
- Operators provision ULN B with the same worker sequence. Its corresponding DVN also consumes nonce `n`, so the code returns the same address as ULN A's DVN without the operator supplying that address.
- The initialization-time duplicate check has already finished. ULN B therefore records the duplicate only in its component-local registry.
- When that address is called, `Container.#resolveComponent` selects the first component that claims it. Calls intended for ULN B's DVN reach ULN A's DVN state instead.
- Operators configure ULN B as the receive library for a live route and require the colliding DVN for that route.
- ULN B can still report the DVN as registered, but ULN B's administrator and signer committee cannot use it to record verification. `commitVerification` cannot complete for packets that require that DVN. Source-side assets already locked or burned for those packets can remain stranded until operators migrate or repair the deployment.
- Once the duplicate is persisted, the next container initialization detects it and fails, obstructing a later upgrade as well.

The checked-in server currently instantiates only one ULN component, so a fresh deployment using that configuration does not encounter this collision. This finding assumes that a supported upgrade may append a second worker-bearing ULN. If the intended deployment model permanently permits only one such ULN, adding another is an unsupported operator configuration and this scenario should not be treated as a vulnerability. The implementation should enforce that restriction explicitly rather than allowing the invalid state to be created. Separately, `scope.txt` for the current LockUnlockAdapter review does not include this runtime code, so this should remain a validated out-of-scope observation unless the review scope is expanded.

— RECOMMENDATION

First define and enforce the supported topology. If only one worker-bearing ULN is allowed, reject a second one during container initialization and document that restriction.

If multiple ULNs are supported, include the creating ULN's address and a stable component or deployment identifier in every worker-address derivation, in addition to the worker

M-06 Appended ULNs can duplicate worker addresses

CONTINUED

type and nonce. Runtime registration should atomically claim the address in one container-wide registry and reject an address that is already claimed. `Container.#resolveComponent` should also require exactly one matching component instead of silently selecting the first.

— RESOLUTION

Status: Resolved.

MEDIUM **M-07**

RESOLVED

OneSig CallContext leaf encoding non-injective

CATEGORY Signatures
LOCATION Layerzero/OneSig/daml/OneSigEncoding.daml
REVIEW Remediation Review 1

DESCRIPTION

OneSigEncoding.daml states the leaf encoding is injective: "two distinct calls can never produce the same bytes, and a signed leaf can never be reinterpreted as a different call." This is the security basis for OneSig multisig approvals (signers approve a Merkle leaf that encodes the exact call, including its CallContext).

The claim is false. `encodeTextMap` / `encodeAddressMap` (OneSigEncoding.daml:438-445) emit only `key <> value` per entry with NO per-map entry count. Every sibling collection encoder DOES include a count: `encodePartyList`, `encodeIntList`, `AV_List` and `AV_Map` all prepend `show (length ...) / show (size ...)`. Because `encodeCallContext` nests six count-free maps, entries can migrate across map levels and produce identical bytes for two DISTINCT CallContext trees -> identical Merkle leaf. Since keys are length-prefixed and `DA.Map.toList` is ascending, each ascending-key parse of the byte string corresponds to a real, re-encodable tree, so the ambiguity is a genuine encoder collision, not just a decoder artifact.

POC: Shows two distinct CallContext values both encode to `0:4:text5:3:int2:10`:

```
ctx1 = 2 top-level callers ({"": {addr "3:int": {}}, "10": {}})
ctx2 = 1 caller with nested target ({"": {addr "3:int": {"10": {}}}})
```

> A non-degenerate collision (different AnyValue leaf content, identical bytes) also exists.

RECOMMENDATION

Make the map encoders self-delimiting so injectivity does not depend on adjacent validators:

1. In `encodeTextMap` and `encodeAddressMap`, prepend the entry count (`lengthPrefix (show (Map.size entries))`) and/or length-prefix each encoded value, mirroring `encodePartyList` / `encodeIntList` / `AV_List` / `AV_Map`.

M-07 OneSig CallContext leaf encoding non-injective

CONTINUED

2. Mirror the exact change in the off-chain Canton SDK (this is a wire-format change; the SDK must reproduce the bytes byte-for-byte).
3. Add golden encoding vectors plus negative tests, including the collision pair in this PoC (ctx1/ctx2 -> 0:4:text5:3:int2:10), asserting distinct CallContext values never share encoded bytes.

Until fixed, keep `validateCallContextSinglePath` mandatory on every context that is hashed into a signed leaf, and do not add ops/consumers that hash multi-path or size-only-validated contexts.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/5>

— RESOLUTION

Status: Resolved.

MEDIUM **M-08**

ACKNOWLEDGED

Stale Request time permits expired DVN actions

CATEGORY	Validation
LOCATION	contracts/protocol/ver-endpoint/src/dvn/dvn.ts
REVIEW	Remediation Review 1

DESCRIPTION

`Dvn` rejects an instruction only when its signed `expiration` is at or before `Context.blockTimestamp`. The guard describes that value as the current block timestamp and `Context` defines it as the `block.timestamp` equivalent. The Canton integration instead constructs every write context with the Request contract's creation time. It never updates the timestamp when validators execute and settle the Request.

Consequently, an instruction submitted before its deadline can change DVN state after the deadline. For Request time `R`, expiration `E` and execution time `X` where $R < E \leq X$, `Dvn` compares `E` with `R` and accepts. The same repository's Stellar DVN compares `expiration` with the ledger timestamp at execution and rejects this case.

This affects every signature-authorized DVN operation. The clearest permission-boundary break is `quorumChangeAdmin`, because any holder of a genuine unused quorum-signed bundle can relay it without already being an administrator. A bundle that adds the relayer remains usable after its signed deadline when its Request was created in time. The newly added administrator can redirect future DVN fees by proposing and activating a new payee. It can also corrupt destination pricing and worker configuration. Exploitation requires a previously valid bundle whose VID, signer membership and quorum remain acceptable, plus enough backlog or validator downtime for execution to occur after expiration.

RECOMMENDATION

Evaluate DVN `expiration` against a consensus execution or settlement timestamp, not the Request creation timestamp. Pass the batch's authenticated ledger-effective time into `Context.blockTimestamp` or enforce on ledger that a Request carrying a DVN instruction cannot be accepted at or after its signed expiration. Keep Request ordering time separate if historical creation time is still needed for deterministic sorting.

M-08 Stale Request time permits expired DVN actions

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

RESOLVED

Future NIL entries enable unbounded scans

CATEGORY	DoS
LOCATION	contracts/protocol/ver-endpoint/src/messaging-channel/index.ts
REVIEW	Remediation Review 1

DESCRIPTION

`nilify()` lets an ordinary user fill arbitrary future nonce positions with `NIL_PAYLOAD_HASH`, even when no packet has ever occupied those positions. `inboundNonce()` later treats every NIL entry as an occupied position and reads forward until it reaches the first empty nonce. Since the loop has no iteration limit, one endpoint call can be made arbitrarily expensive by state that the caller prepared in advance.

The problem starts with the way `nilify()` handles an empty future position. An unused position returns `EMPTY_PAYLOAD_HASH`. The caller can supply that same value as `payloadHash`, so the first comparison succeeds. The only empty-position check applies when `nonce` is at or below the lazy inbound nonce. A nonce above the lazy value bypasses that check and is written as NIL:

```
// Location: MessagingChannel.nilify()
const currentPayloadHash = await this.#getInboundPayloadHash(oapp, srcEid, sender, nonce);
if (currentPayloadHash !== payloadHash) {
  throw new MessagingChannel_PayloadHashNotFoundError(currentPayloadHash, payloadHash);
}
if (
  nonce <= (await this.lazyInboundNonce(context, oapp, srcEid, sender)) &&
  currentPayloadHash === EMPTY_PAYLOAD_HASH
) {
  throw new MessagingChannel_InvalidNonceError(nonce);
}
await this.setInboundPayloadHash(context, oapp, srcEid, sender, nonce, NIL_PAYLOAD_HASH);
```

This means a caller can start with a lazy inbound nonce of zero and write NIL into nonces 1, 2, 3 and so on. None of those writes advances the lazy nonce.

The nonce scanner considers any value other than `EMPTY_PAYLOAD_HASH` to be occupied. `NIL_PAYLOAD_HASH` therefore keeps the loop running just like a real packet hash:

M-09 Future NIL entries enable unbounded scans

CONTINUED

```
// Location: MessagingChannel.#hasPayloadHash()
async #hasPayloadHash(
  receiver: Address,
  srcEid: bigint,
  sender: Address,
  nonce: bigint,
): Promise<boolean> {
  return (
    (await this.#getInboundPayloadHash(receiver, srcEid, sender, nonce)) !==
    EMPTY_PAYLOAD_HASH
  );
}

// Location: MessagingChannel.inboundNonce()
async inboundNonce(
  _context: Context,
  receiver: Address,
  srcEid: bigint,
  sender: Address,
): Promise<bigint> {
  let nonceCursor = await this.#lazyInboundNonces.get(receiver, srcEid, sender);
  while (await this.#hasPayloadHash(receiver, srcEid, sender, nonceCursor + 1n)) {
    ++nonceCursor;
  }
  return nonceCursor;
}
```

`skip()` makes the behavior repeatable because it calculates the complete inbound nonce before it checks whether the caller supplied the correct next nonce:

```
// Location: MessagingChannel.skip()
if (nonce !== (await this.inboundNonce(context, oapp, srcEid, sender)) + 1n) {
  throw new MessagingChannel_InvalidNonceError(nonce);
}
```

For example, assume the lazy nonce is zero and a user has placed NIL in positions `1` through `10,000`. Calling `skip(..., 1)` first reads all 10,000 NIL entries and the empty position at `10,001`. Only after doing that work does it calculate that the correct skip nonce is `10,001` and reject the supplied value `1`. The rejection does not delete the NIL entries or advance the lazy nonce. A later request can therefore force the same scan again.

M-09 Future NIL entries enable unbounded scans

CONTINUED

Any ordinary user can perform this attack. The user does not need an OApp registration, an endpoint-delegate role, administrator authority or control over another user's application. For a non-OApp request, the on-ledger validation only requires the caller address to represent the Request sender:

```
-- Location: RequestFactoryValidation.validateCallContextIdentity
None ->
assertMsg errRequestFactory_CallContextInvalidSenderAddr $
callerAddr == Addr_Text (partyToText sender)
```

The validator converts that Party into its deterministic runtime address:

```
// Location: Canton call-context conversion
return {
  caller: deriveGlobalAddress(call.caller, AddressDomain.CHAIN),
  transaction: transactionSchema.parse({
    functionSignature: {
      contract: call.contract,
      function: call.function,
    },
    address: call.address,
    arguments: call.arguments,
  }),
};
```

The endpoint then accepts the call whenever the supplied `oapp` is that same address:

```
// Location: EndpointBase.assertAuthorized()
async assertAuthorized(context: Context, oapp: Address): Promise<void> {
  if (context.msgSender === oapp) {
    return;
  }
  const delegate = await this.getDelegate(context, oapp);
  if (delegate === AddressState.default() || context.msgSender !== delegate) {
    throw new EndpointV2_UnauthorizedError();
  }
}
```

The user is only modifying a channel tuple associated with its own derived address. That is enough for the attack because the expensive computation runs on shared validator infrastructure.

Example of exploitation:

M-09 Future NIL entries enable unbounded scans

CONTINUED

- A normal Canton user creates non-OApp Requests through the public RequestFactory. The user supplies its own deterministic runtime address as `oapp`, so the endpoint's normal self-authorization succeeds.
- The user chooses a channel tuple under that address and submits one paid `nilify()` Request for each future nonce. Every call supplies `EMPTY_PAYLOAD_HASH` as the expected current value. Nonces `1` through `N` become NIL while the lazy inbound nonce remains zero.
- Building the run requires `N` accepted Requests and positive fee transfers. The setup is therefore not free. However, the resulting NIL state is persistent and the protocol places no limit on `N`.
- After preparing the run, the user submits malformed `skip(..., 1)` Requests. Each validator reads the local EID, the lazy nonce, all `N` NIL entries and the first empty entry before rejecting. This is exactly `N + 3` trie reads per malformed skip.
- If the Requests finish within the validator deadline, they are rejected and archived. The user can submit fresh malformed skips to pay for and repeat the same scan without rebuilding the NIL run.
- The default validator batch contains five Requests and executes them sequentially. Five attacker Requests therefore require `5 * (N + 3)` trie reads on every validator before the batch can finish.
- Validator HTTP clients use a 30-second deadline. If a prepared run makes the five scans exceed that deadline on enough validators, the sequencer cannot form quorum and submits no ledger transaction. The five Requests remain active and can be selected again immediately. The client timeout does not cancel validator-side execution, so a retry can overlap scans that are still running.
- An authenticated read client can also invoke `inboundNonce()` directly and fan out the uncapped scan without creating another ledger Request. This is only a secondary amplifier because the production HTTP service restricts access to allowlisted JWT clients.

The validated PoC used `N = 256`. Each of two malformed skips performed exactly 259 trie reads and the lazy nonce remained zero after both attempts. The relationship is deterministic: setup costs `O(N)`, each trigger costs `O(N)` and `M` triggers impose `O(N * M)` work. The exact run length needed to exceed the 30-second deadline depends on the deployed database, hardware and validator load. That threshold was not reproduced against a production environment.

This attack consumes CPU and database-read capacity on every validator. Since Requests in a batch execute sequentially, attacker-controlled scans delay unrelated Requests behind them. Once a batch crosses the client deadline, the unchanged active Requests can create a self-reinforcing retry loop that interferes with settlement quorum until a request eventually completes or operators intervene.

M-09 Future NIL entries enable unbounded scans

CONTINUED

RECOMMENDATION

The safest small change is to reject `nilify()` when the current position is `EMPTY_PAYLOAD_HASH`, regardless of whether the nonce is above or below the lazy inbound nonce. A caller should only be able to nilify a position that already contains the exact non-empty packet hash it supplies.

If empty future nilification is required by the protocol, enforce both a maximum distance above the lazy nonce and a maximum number of outstanding NIL positions for each receiver, source EID and sender tuple. `inboundNonce()` must also have a fixed work limit. One option is to maintain the highest consecutive nonce as explicit state and advance it incrementally, with a strict maximum number of positions processed by one call.

Add an execution or trie-read budget to every runtime request. Apply a separate rate limit to authenticated read calls, because they do not pay ledger Request fees. Operators should also have a safe compaction mechanism for attacker-created NIL state that cannot erase legitimate packet hashes.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/commit/2bf54c3f35631ca274aaa02dcb03bd853bd66dd7>

RESOLUTION

Status: Resolved.

MEDIUM **M-10**

RESOLVED

Unbounded Event Scans Can Exhaust Validators

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/2bf54c3f35631ca274aaa02dcb03bd853bd66dd7/packages/protocol/lz-ver-protocol/ver-api-common/src/controllers/validator-scan.ts#L140-L181
REVIEW	Remediation Review 1

— DESCRIPTION

An authenticated caller can request an arbitrarily large event nonce range. The sequencer forwards the request to every validator, and each validator loads, serializes, and signs all matching database records. Because there is no maximum range, pagination, or SQL result limit, repeated or concurrent requests can exhaust database, CPU, memory, and network resources. Enough affected validators may prevent the system from reaching quorum and disrupt normal `VER` processing.

— RECOMMENDATION

Enforce a maximum nonce range and paginate results using a bounded page size. Add a database LIMIT, rate-limit scan requests, and cancel database work when requests time out or disconnect.

— RESOLUTION

Status: Resolved.

MEDIUM **M-11**

ACKNOWLEDGED

Uncapped signature array enables node DoS

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/services/verifier/quorum-verifier.ts:20-57
REVIEW	Remediation Review 1

DESCRIPTION

When a validator or gateway reads or scans Canton state, it verifies the peer's response against the known committee with `Secp256k1QuorumVerifier.verify()`. That method loops over every element of the response's `signatures` array, performing one keccak256 hash and one secp256k1 public-key recovery per entry, and only checks whether quorum was reached after the entire loop finishes. The array is taken verbatim from the peer response, and the wire schemas declare it as a plain array with no maximum length (`signatures: z.array(...)` in `schemas/sequencer.ts` and `schemas/canton.ts`).

Because the querying node explicitly does not trust that peer for data integrity — verifying signatures is the whole reason the response is checked — a malicious or compromised sequencer/scan endpoint (or an on-path attacker where transport is not TLS-protected) can return a well-formed response whose `signatures` array is arbitrarily long. Unrecognized entries never match a committee key, so quorum is never reached and there is no early exit: the node performs an attacker-chosen number of expensive elliptic-curve recoveries synchronously, blocking its single-threaded event loop and inflating process memory. The effect degrades or crashes the whole node rather than merely failing the one request.

RECOMMENDATION

Bound the signatures array length at the wire boundary — e.g. `z.array(...).max(committee.publicKeys.length)` on the response schemas — and/or add an explicit length guard at the top of `verify()` before the recovery loop.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Rate limiter mode toggle ignores active usage

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/AdapterConfig.daml
REVIEW	Remediation Review 1

DESCRIPTION

`IRateLimiterConfig_SetGlobalFlags` can switch `useGlobalState` directly without checkpointing or migrating the existing `RateLimiterState.eidStates` buckets. The scoped `AdapterConfig` implementation writes the new flag as a plain config update. `OftSelfConfig` contains the same implementation outside the current scope.

The rate limiter uses different state keys depending on this flag. When `useGlobalState` is `False`, each endpoint uses its own `eid` bucket. When `useGlobalState` is `True`, `getStateEntry` maps every endpoint to bucket `0`. Therefore, switching from per-EID mode to global mode makes newly submitted transfers read bucket `0` and ignore usage already accumulated under the individual EID buckets. Switching back has the opposite problem: transfers read per-EID buckets and ignore accumulated usage in bucket `0`.

This is a role-gated configuration issue rather than an arbitrary-user exploit. The caller must hold `_RATE_LIMITER_MANAGER_ROLE` or be the OApp admin. Still, the change can silently reset the effective rate-limit accounting for active traffic, so the configured rate limit may not be enforced across a global/per-EID migration.

RECOMMENDATION

Do not allow `useGlobalState` to change as a plain flag update after deployment or replace `SetGlobalFlags` with an explicit migration choice.

When enabling global mode, decay the existing per-EID buckets under the old configuration and conservatively aggregate their remaining usage into bucket `0`. When disabling global mode, either require bucket `0` to be fully decayed or seed the relevant per-EID buckets with a conservative amount.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Arbitrum price updates overwrite other EIDs

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/ver-endpoint/src/price-feed/price-feed.ts
REVIEW	Remediation Review 1

DESCRIPTION

`setPriceForArbitrum()` stores the base `Price` under the supplied EID, but it stores `ArbitrumPriceExt` under the constant key `0`. Every Arbitrum fee estimate later reads that same key. Therefore, the most recent Arbitrum update replaces `gasPerL2Tx` and `gasPerL1CallDataByte` for every other Arbitrum destination.

An honest price updater can update one destination with correct chain-specific values and silently change quotes for another destination whose own base price was not touched. An underestimated quote can underfund workers or delay delivery. An overestimated quote charges senders more than the destination requires. The issue needs at least two configured Arbitrum destinations whose extension values differ. It does not grant an untrusted party price-update authority.

The repository's Stellar implementation also stores one global extension, so this may reflect an inherited design assumption. That assumption is not enforced by the scoped API: `setPriceForArbitrum()` accepts an EID and describes the extension as belonging to that EID. If all supported Arbitrum destinations must share one extension, the interface should make that restriction explicit and prevent per-EID updates from implying isolation.

RECOMMENDATION

Store `ArbitrumPriceExt` by EID and read the matching entry in `estimateFeeWithArbitrumModel()`. Change the getter to accept an EID. If a single global extension is intentional, move it to a separate global setter and document that updating it changes every Arbitrum quote.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Foreign Requests trigger an empty batch loop

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/RequestFactory/daml/RequestFactory.daml
REVIEW	Remediation Review 1

DESCRIPTION

`RequestFactory` copies its `permanentObservers` into every new `Request`. Any ordinary Canton party can deploy a genuine factory under the reviewed package, control that factory's handler and treasury and name the real LayerZero owner as a permanent observer. The resulting official `Request` remains visible to the LayerZero owner even though the owner did not authorize it and cannot archive it.

The production services treat visibility as evidence that useful work exists.

`CantonChainClient.hasPendingRequests()` returns true for the first active `Request` visible to the owner without checking its `handler` or factory. The validator poller instead scans visible parent `Request_Create` exercises. Daml privacy projection does not make a child observer a witness of its parent exercise, so the same foreign contract produces zero pollable request IDs.

The sequencer therefore asks every validator to process a write. The empty result reaches

`StateCommitmentSdk.prepareCommitBatch()`, which throws `No entries to resolve`. `RequestProcessor` catches that failure and immediately checks again without the configured idle delay because the foreign child is still active. One attacker-funded Request can sustain Ledger API queries, validator HTTP calls, error logging and CPU work until the service is patched or the attacker voluntarily archives the contract. Legitimate requests can still be processed when they appear, so this is availability degradation rather than a proven global settlement halt.

RECOMMENDATION

Authenticate Requests before using them as the pending-work signal. Decode the candidate and require `handler == lzOwnerParty`; ideally bind discovery to the expected RequestFactory deployment or another stable handler/factory identity before applying the active-contract limit. Do not rely on visibility alone.

Also make the sequencer sleep or exponentially back off after a batch attempt yields no pollable entries or fails before submission.

L-03 Foreign Requests trigger an empty batch loopCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Unused fee recipient blocks fee-free OFT sends

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/Oft/daml/0App/LzSend/Call/Impl.daml
REVIEW	Remediation Review 1

DESCRIPTION

`lzSendWithCallArgs()` always includes `oftSelfConfig.feeDeposit` in the parties checked by `assertAllowlisted()`. It performs this check before calculating the destination's issuer fee.

The shared debit calculation defaults a missing destination fee to zero. It also omits the `feeDeposit` payout when the calculated fee is zero. Therefore, `feeDeposit` is not a recipient and has no economic role in a fee-free send, but its allowlist status still controls whether the send can proceed.

For example, an issuer can configure a fee-free destination and later blacklist the fee recipient because that party should no longer receive token payments. Every sender and actual payout recipient may remain allowed. Even so, all otherwise valid cross-chain sends revert with `ERR_ALLOWLIST_NOT_ALLOWLISTED`. Whitelist mode has the same problem when senders are listed but the unused `feeDeposit` is not.

RECOMMENDATION

Calculate `issuerFeeAmount` before assembling the allowlist party set. Include `feeDeposit` only when the actual fee is positive and the party differs from the sender. Continue checking every sender and every external receiver whose positive payout is included.

Reuse the same computed payout list for allowlist validation and callback construction so a party is checked exactly when it can receive value.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Old DVN approvals can undo later decisions

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/ver-endpoint/src/dvn/multisig/hashes.ts
REVIEW	Remediation Review 1

DESCRIPTION

The DVN signs each governance instruction independently. The signed data identifies the requested change, the DVN and an expiration time. It does not include an instruction number, a policy version or the state that the signers expected to change. `usedHashes` only stops the exact same instruction from executing twice. It does not cancel an older instruction when the committee later approves the opposite change.

For example, the committee may sign an instruction that adds `X` as an administrator but decide not to execute it. The committee can later sign and execute an instruction that removes `X`. Since `X` is not yet an administrator, the removal completes without changing anything. The old addition remains unused and valid, so it can still be submitted afterward. This makes `X` an administrator despite the committee's later decision to remove it.

Quorum changes have the same problem. `verifySignatures` reads the number of required signatures from the current quorum instead of binding the instruction to the quorum that was in force when it was signed. If three signers approved an old instruction and the current quorum later becomes two, the holder can remove the third signature and submit the remaining two. An old instruction that lowers the quorum to one can therefore undo a newer quorum increase.

After the quorum becomes one, a single signer can approve a new administrator and signing key. The new key can then remove the original signers and administrator. A restored administrator can also redirect future DVN fees. If this DVN is sufficient to satisfy an application's verification requirements, control of its administrator and signer set can be used to approve attacker-chosen packet hashes. Other required DVNs would still need to approve the packet.

The sequence requires the old instructions to remain unused and unexpired. Their signers must still belong to the current committee. The party named by the old administrator instruction must also be controlled by the attacker. Replacing the complete committee additionally requires one current signer.

L-05 Old DVN approvals can undo later decisions

CONTINUED

— RECOMMENDATION

Add a monotonic governance sequence or policy epoch to the DVN. Include it in every signed governance instruction. An instruction should only execute when its epoch equals the DVN's current epoch, and execution should advance the epoch so that older instructions become invalid.

Provide an explicit cancellation or epoch-increment operation so the committee can invalidate signed instructions that will no longer be executed. Bind quorum and signer changes to the expected current quorum and policy state. Signature verification must use the policy recorded in the signed instruction instead of reinterpreting an old bundle under the current quorum.

Reject attempts to add an existing administrator or remove an administrator who is already absent. These checks make contradictory or ineffective changes visible, although they do not replace the policy-epoch fix.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Unbounded cost multipliers brick sends

CATEGORY	DoS
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/daml/0App/CostAsserts.daml:139-140
REVIEW	Remediation Review 1

DESCRIPTION

`isValidCostAsserts`, which gates every stored cost config through the config `ensure` clause, bounds the cost parameters only from below: `maxPriceRatio > 0`, `maxGasPrice > 0`, and the scale exponents within their limits. It places no upper cap on `maxGasPrice` or `maxPriceRatio`.

When a send routes through a destination that has a cost config, `assertOptionsCost` calls `computeOptionsCost`, which evaluates the gas cost as `totalGas * maxGasPrice * gasPriceScaleRatio` and then `(gasCost + valueCost) * maxPriceRatio`, all in `Decimal`. The module pre-divides the power-of-ten scale factors to avoid intermediate overflow, but the two admin-chosen multipliers are themselves unbounded. A sufficiently large `maxGasPrice` or `maxPriceRatio` drives the product past `Decimal`'s magnitude limit and raises an arithmetic error.

Because that error aborts `assertOptionsCost`, every send to the affected destination reverts. A cost-config administrator can thus set a value that passes validation yet bricks outbound sends to that endpoint until the config is corrected — the same missing-upper-bound shape already addressed for the time and minFee setters.

RECOMMENDATION

Add sane upper bounds on `maxGasPrice` and `maxPriceRatio` in `isValidCostAsserts`, mirroring the upper-bound pattern applied to the time/minFee setters, so no accepted cost config can drive `computeOptionsCost` into `Decimal` overflow.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Non-canonical config inflates OFT mint

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/Oft/daml/OftSelfConfig.daml:64-68
REVIEW	Remediation Review 1

— DESCRIPTION

On inbound receive settlement, the OFT config used to convert the wire amount into a local mint amount is supplied by the finalizer and validated by `fetchAndValidateConfig`, which requires only that the fetched config is signed by the instrument admin and reports the matching instrument. It does not require the config to be a single canonical instance pinned for that instrument, and the config contract is not keyed, so multiple admin-signed configs for the same instrument can coexist.

The receive accept handler reads `localDecimals` and performs the decimal conversion using whichever config the finalizer supplies. If a second admin-signed config for the same instrument exists with a different `localDecimals`, finalizing the inbound receive against that alternate config converts the same wire amount into a different local amount, inflating (or deflating) the minted holding relative to the canonical config.

Exploitation requires a second admin-signed same-instrument config to exist and be visible to the finalizer; a normal user cannot create one. It is a canonicalization/deployment-hardening gap on the OFT mint path.

— RECOMMENDATION

Bind the OFT config to a single canonical instance — store the current config in the OApp state or key the config by instrument — and require receive settlement to resolve that canonical config rather than accepting any admin-signed same-instrument config at finalize time.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Payout settlement skips local-transfers pause

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ft/daml/0ftSelf.daml:201-218
REVIEW	Remediation Review 1

— DESCRIPTION

When a cross-chain send with contingent payouts settles, the payouts are now distributed through the burn-and-mint factory choice (`burnMintFactory_burnMintImpl`). That choice validates the config, the token admin, the instrument, and the receiver allowlist, but it does not check whether local transfers are paused. In the previous design the same contingent payouts settled through the locked transfer offer and the transfer rule, which enforced the local-transfers-paused gate.

As a result, contingent payouts continue to mint new local holdings to receivers even while an administrator has set the local-transfers-paused flag — a state that the send side and the ordinary transfer path both honor. The sibling factory operations on the same contract, for ordinary transfers and for allocations, still respect the pause; only this settlement path omits it, an omission introduced when the payout path was moved off the transfer rule.

No attacker is involved and no value is over-minted: the affected payouts belong to already-escrowed, allowlist-checked, in-flight sends whose pool was locked before the pause, and all new sends, transfers, and allocations remain blocked. The issue is a behavioral inconsistency in the scope of the pause control. It is rated Low pending confirmation of whether settling an already-committed, in-flight cross-chain send is intended to be covered by the local-transfers pause.

— RECOMMENDATION

If the pause is meant to halt all local settlement, add the local-transfers-paused check to the burn-and-mint settlement path (or gate payout distribution on it), matching the transfer and allocation paths. If completing in-flight cross-chain sends is intentionally exempt from the pause — consistent with the inbound receive mint path, which is likewise ungated — document that exemption so the pause control's scope is unambiguous.

L-08 Payout settlement skips local-transfers pause

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Container slots unpinned; fork resets state

CATEGORY	Upgradeability
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/container/container.ts:97-130
REVIEW	Remediation Review 1

— DESCRIPTION

Container-level state fields are assigned storage slots purely by allocation order from a fresh slot allocator, with no per-field pinning and no reserved gap. The container currently allocates nonce (slot 0) then componentStateRoots (slot 1). On a state-inheriting hard fork, initialize(previousStateRoot) reads componentStateRoots from its allocated slot of the inherited trie to decide which components to carry over.

If a future upgrade inserts any new container-level field before componentStateRoots — the natural "declare it near the top" habit — every slot after it shifts by one. initialize then reads componentStateRoots from a slot the previous binary never wrote, sees length 0, and treats every component as non-inherited: each is re-initialised from genesis. The entire inherited state (nonces, message-library configs, balances) is silently discarded with no revert.

The likelihood is bounded: the container is a small bespoke struct whose fields are written explicitly, and appending a new field after componentStateRoots is safe — only an insert-before is unsafe. It is rated Low as a robustness gap on a first-class supported upgrade path rather than a live defect.

— RECOMMENDATION

Pin container field slots by name, reserve a gap, or assert a layout-version in initialize before inheriting inherited state; at minimum document that new container fields must be appended after componentStateRoots.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Null recipient send burns OFT principal

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/OftCommon/damL/OApp/LzSend/Call/Validate.damL:26-30
REVIEW	Remediation Review 1

DESCRIPTION

The shared send validator `validateSendArgs` checks that `dstEid` and `amountLD` are positive, that `minAmountLD` is non-negative and not greater than `amountLD`, and that every receiver payout amount is strictly positive. It never inspects `sendParam.to`, the destination recipient. A send therefore passes validation with an all-zero bytes32 recipient (64 hex zeros); the message encoder accepts it (valid hex, left-padded to 64 chars) and commits it into the cross-chain message.

The sender's principal is locked at send time and burned on the accept callback, before the recipient is ever checked. An all-zero recipient resolves to no reachable party on the destination, so the burned tokens have no counterpart mint. Because custody is committed before recipient validation, a caller who supplies the null recipient irreversibly loses the bridged principal, with no on-ledger recovery for that leg.

Because the check is missing from the shared validator, every OApp variant that routes through it is affected; adding the guard there closes the gap for all of them at once.

RECOMMENDATION

Reject a null (all-zero) `sendParam.to` in `validateSendArgs` before any holding is locked, so a null-recipient send fails up front rather than burning the sender's principal to an unreachable destination.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

RESOLVED

Reject leg unlocks payout pool unbound

CATEGORY	Access Control
LOCATION	contracts/protocol/canton/contracts/Layerzero/Oft/daml/OApp/Callback/Dispatcher.daml:109-118
REVIEW	Remediation Review 1

— DESCRIPTION

On an LzSend reject callback, the dispatcher parses the main cross-chain holding and an optional payout pool out of the untrusted callback context. It asserts that the request sender owns the main holding, but applies no ownership check to the payout pool. The reject handler then unlocks both holdings unconditionally. Because unlocking a locked OFT holding requires only the token admin's authority (the owner is a mere observer), a party holding the endpoint delegate role can forge a reject callback that names a victim's locked payout pool and cause it to be unlocked without the victim's consent.

Because unlocking returns the pool to its owner rather than transferring it to the attacker, no value is stolen. The harm is availability: if the victim has a concurrent in-flight send whose settlement depends on that pool remaining locked, unlocking it prematurely strands that send, since its later accept can no longer consume the expected locked pool.

It is rated Low because it moves no value to the attacker and is gated on two preconditions: the attacker's forged request must reach the reject leg (the accept-or-reject decision is made by the trusted settlement flow), and a concurrent victim send must exist that depends on the targeted pool.

— RECOMMENDATION

Apply the same sender/owner binding to the payout pool as to the main holding before unlocking: assert the pool's owner equals the pending callback's request sender, or carry the pool reference from trusted call-time state created during the original send rather than accepting it from the callback context.

— RESOLUTION

Status: Resolved.

LOW **L-12**

ACKNOWLEDGED

Pending outflow decay leaves phantom rate usage

CATEGORY	Logical Error
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml:354-382
REVIEW	Remediation Review 1

— DESCRIPTION

The rate limiter keeps per-destination usage in an aggregate leaky bucket on `RateLimiterState` that decays continuously. A two-phase outflow freezes each pending send's `scaledAmount` and its anchor `recordedAt`; at settlement `commitOutflow` / `reverseOutflow` subtract only the send's remaining contribution, which `remainingContribution` computes by decaying the frozen `scaledAmount` on its own and flooring it at zero. That per-item decay is not additive with the aggregate decay: once elapsed time exceeds the `scaledAmount`'s own decay, `remainingContribution` clamps to zero while the aggregate bucket still carries that amount.

So a rejected send — or the net accounting on an accepted send — subtracts less than it still occupies, leaving phantom usage in `outboundUsage` / `inboundUsage` that over-reports consumed capacity and throttles later legitimate transfers to that destination. It needs no configuration change, moves no holdings, and is self-healing as the bucket keeps leaking, so the impact is a temporary availability blip.

— RECOMMENDATION

Settle a pending outflow so the bucket returns to the state it would hold had that send never been recorded. Rather than having `remainingContribution` decay the frozen `scaledAmount` independently and floor it at zero, `commitOutflow` / `reverseOutflow` should subtract the actual decayed share the send still occupies in the aggregate bucket — or checkpoint each pending send's contribution into `RateLimiterState` so a single decay applies consistently to both the bucket and the pending amount.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Stale prices undercharge executor fees

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/ver-endpoint/src/price-feed/price-feed.ts
REVIEW	Remediation Review 1

— DESCRIPTION

`PriceFeed.setPrice` stores each destination price without an update time or expiry. `estimateFeeByEid` later ignores its `Context` and accepts any nonzero stored record, regardless of its age. Therefore a destination price remains usable indefinitely when the updater stalls.

`ExecutorFeeLib.getFee` uses the stored ratio to convert sender-selected `lzReceive`, `NativeDrop` and `lzCompose` value into the source-native fee. It also uses the stored gas price to charge for destination work. An ordinary sender can wait for the destination-native token or gas price to rise, then submit value-bearing messages while the old lower price remains active. The sender pays the stale conversion while the executor must fund the current destination cost from its own balance. `nativeCap` limits one message, but it does not limit repeated messages. If the executor refuses underfunded work instead, already-paid packets can remain undelivered and value transfers can remain pending.

For example:

- 10:00 - Price ratio is `1`; the correct fee is `102`.
- 10:15 - The price updater goes offline.
- 10:30 - Destination gas or token costs rise significantly.
- 10:35 - The protocol still charges `102` using the old price.
- 11:00 - The updater returns and posts the new price.
- 11:01 - The protocol now charges the correct higher fee.

— RECOMMENDATION

Store an authenticated update time with every per-EID price and with dependent global inputs. Configure a conservative maximum age for each destination and reject quotes and sends when any required input is stale. Optimism quotes must check both the L1 and L2 records. Arbitrum quotes must also check the extension and other global inputs they use.

Use a trustworthy processing-time source or a signed expiry for the freshness check. Do not use the historical Request creation timestamp. Add regression tests at the maximum-age

L-13 Stale prices undercharge executor fees

CONTINUED

boundary and after updater outages for gas-only, `NativeDrop`, `lzReceive` value and `lzCompose` value quotes.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

ULN send support ignores unregistered DVNs

CATEGORY	Validation
LOCATION	contracts/protocol/ver-endpoint/src/message-library/uln/uln-302/uln-302.ts:1111
REVIEW	Remediation Review 1

DESCRIPTION

`Uln302.#isSupportedSendEid` reports a destination EID as supported for send whenever its default send ULN config has at least one DVN by count and a registered executor. It never checks that the referenced default DVNs are registered in `#dvns`.

Because `quote()` and `send()` resolve every required and optional DVN through `#getDvn` (which throws `Uln302_DvnNotRegisteredError` for any unregistered address), a default send config naming an unregistered DVN is advertised as usable yet reverts on the first quote or send. `setDefaultSendConfigs` does not close the gap: it only runs `assertValidUlnConfig` (structural checks only: sorted, no duplicates, address width, at least one DVN, threshold sanity, max count, confirmations) and never consults `#dvns`.

This is inconsistent with guards already present elsewhere: the receive side (`#isSupportedReceiveEid`) validates DVN registration, and the per-OApp send path (`#assert0appConfigDvnsRegistered`) validates DVN registration; the send side already validates executor registration. Only the default send DVN check is missing, so it also violates the `#isSupportedSendEid` doc comment, which states the check must never accept a send-library selection for an EID that can never quote or send.

The gate is reachable and exploitable end to end. `isSupportedEid = isSupportedSendEid && isSupportedReceiveEid`, and send and receive configs are independent. A broken send config (unregistered DVN) plus a genuinely valid receive config (registered DVN) passes the combined gate, so `MessageLibraryManager.setDefaultSendLibrary` accepts the library for the EID; `endpointV2.quote` then dispatches to `uln302.quote`, which reverts with `Uln302_DvnNotRegisteredError`.

Note send is stricter than receive: `#quoteDvns` and `#assignDvnJobs` iterate every required and optional DVN and call `#getDvn` on each with no threshold skipping, so send support is truthful only if every required and every optional default DVN is registered.

L-14 ULN send support ignores unregistered DVNs

CONTINUED

Impact is gated by owner and configuration (the owner controls both DVN registration and the default config), so it is not an unprivileged exploit and is fully recoverable. The effect is a false route-usable signal plus a hard revert on the send path for the affected EID until fixed. Confirmed by two passing PoC tests (see pocLink).

— RECOMMENDATION

Make `#isSupportedSendEid` mirror the send-time registry requirements rather than the receive-time threshold semantics:

1. Return unsupported unless every required default DVN is registered in `#dvns`.
2. Return unsupported unless every optional default DVN is registered in `#dvns`, because send quotes and assigns all of them, not just a threshold. (Do not copy the receive-side threshold logic here; it would still leave unregistered optional DVNs broken on the send path.)
3. Optionally also validate DVN registration at config time in `setDefaultSendConfigs` (mirroring `#assert0appConfigDvnsRegistered`), so an unusable default send config is rejected when configured rather than only reported unsupported later.

Add regression tests where a default send config references an unregistered required DVN and an unregistered optional DVN, asserting `isSupportedSendEid` returns false (and/or `setDefaultSendConfigs` rejects), so the route can never be selected as supported.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/8>

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

DVN ACL/msg-lib methods omit admin gate

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/8d7008e/c/contracts/protocol/ver-endpoint/src/dvn/dvn.ts#L298-L361>

REVIEW Remediation Review 1

DESCRIPTION

In the Solidity DVN reference, committee-signed ACL and message-library changes reach their mutators only via `execute(ExecuteParam[])`, which is `onlyRole(ADMIN_ROLE)`. `execute` verifies quorum signatures, then performs a low-level self-call to the target selector; the mutators are `onlySelf` (through `onlySelfOrAdmin`, which forces self for `ALLOWLIST` / `DENYLIST` / `MESSAGE_LIB_ROLE`). So setting the allowlist, denylist, or message-library set requires BOTH a valid quorum signature AND an admin submission.

This TypeScript port has no `execute` dispatcher: each quorum method is invoked directly with `context.msgSender`, so the admin gate must live inside each method. It is applied inconsistently. `verify` (dvn.ts:195), `setSigner` (dvn.ts:250), and `setQuorum` (dvn.ts:286) all call `await this.assertAdmin(context)` before signature verification, matching the reference. The three ACL/message-library methods do NOT; they proceed on quorum signature alone:

- `setAllowlist` (dvn.ts:298)
- `setDenylist` (dvn.ts:320)
- `setDvnMessageLibrary` (dvn.ts:342)

`assertAdmin` (`worker-base.ts:177`) is a real gate: it rejects unless `context.msgSender` is in the admins set. Its presence on the sibling quorum methods, plus the reference gating these selectors behind `execute`'s `onlyRole(ADMIN_ROLE)`, indicates the omission is an oversight rather than a deliberate design.

No signature forgery is possible: the signed hash binds the DVN address, `vid`, expiration, and the exact parameters. The residual risk is loss of the admin's control over when/whether a signed-but-not-yet-broadcast instruction is applied. An observer who obtains a signed blob (e.g. from a relayer or the mempool) can broadcast it prematurely or out of the admin's intended sequence, and the admin can no longer suppress it by rotating admins first. This is a defense-in-depth / spec-conformance downgrade, not a forgery bug.

The admin-change paths `quorumChangeAdmin` and `quorumReplaceAdmins` are intentionally excluded: the reference `quorumChangeAdmin` is a standalone `external` function that bypasses

L-15 DVN ACL/msg-lib methods omit admin gate

CONTINUED

`execute()` and grants `ADMIN_ROLE` on quorum signatures alone (the admin-recovery path), so their quorum-only authorization is correct.

— RECOMMENDATION

Add `await this.assertAdmin(context);` at the start of `setAllowlist`, `setDenylist`, and `setDvnMessageLibrary`, matching the Solidity reference (where these selectors are only reachable via the `onlyRole(ADMIN_ROLE)` `execute()` dispatcher) and the existing `verify` / `setSigner` / `setQuorum` handlers in this port.

Do not gate `quorumChangeAdmin` / `quorumReplaceAdmins`: the reference runs admin changes on quorum signatures alone as an admin-recovery path, so their quorum-only authorization is correct and must be preserved.

— RESOLUTION

Status: Acknowledged.

LOW **L-16**

RESOLVED

Whitespace-Only Authentication Passes Validation

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/2bf54c3f35631ca274aaa02dcb03bd853bd66dd7/packages/protocol/lz-ver-protocol/sequencer-sdk/src/clients.ts#L37
REVIEW	Remediation Review 1

DESCRIPTION

Authorization headers and validator access tokens containing only whitespace are treated as non-empty.

Clients therefore start with invalid credentials and fail later during authentication. This causes confusing operational errors but no authorization bypass.

RECOMMENDATION

Consider to trim authentication values before validation and reject them when the trimmed value is empty.

RESOLUTION

Status: Resolved.

LOW **L-17**

RESOLVED

Unprefixed Hex Bypasses Fixed-Width Checks

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/2bf54c3f35631ca274aaa02dcb03bd853bd66dd7/packages/protocol/lz-ver-protocol/ver-encoding-utils/src/utils/hex.ts#L57
REVIEW	Remediation Review 1

DESCRIPTION

`hexZeroPad` includes two characters for the expected `0x` prefix when checking the input length. Because the prefix is optional, an oversized unprefixed value can bypass the check.

For example, `hexZeroPad('ffffff', 2)` returns the three-byte value `0xffffffff` instead of rejecting it as too large for two bytes.

Current production callers validate input widths separately, so no direct impact was identified. However, future callers could incorrectly rely on `hexZeroPad` to enforce the requested width.

RECOMMENDATION

Consider to remove the optional `0x` prefix before checking the input length, then reject values containing more than `length * 2` hex characters. Or at least to be aware and document this behavior.

RESOLUTION

Status: Resolved.

LOW **L-18**

RESOLVED

Malformed Hex Inputs Are Silently Truncated

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/2bf54c3f35631ca274aaa02dcb03bd853bd66dd7/packages/protocol/lz-ver-protocol/ver-encoding-utils/src/utils/hex.ts#L17
REVIEW	Remediation Review 1

DESCRIPTION

`hexToBytes` passes input directly to `Buffer.from(value, 'hex')`. Node.js silently stops decoding at the first invalid character instead of throwing an error.

For example, `aabb-not-hex` is decoded as `aabb`.

This behavior is documented in the official Node.js Buffer documentation:
<https://nodejs.org/api/buffer.html#buffers-and-character-encodings>

As a result, unvalidated callers may hash or verify bytes that differ from the supplied input. Current RPC schemas validate hex inputs, limiting present exploitability, but future or internal callers may use this helper without equivalent validation.

RECOMMENDATION

Consider to validate the complete string before decoding and throw an error if it contains non-hex characters, or at least be aware about that and document this behavior.

RESOLUTION

Status: Resolved.

LOW **L-19**

RESOLVED

DVN accepts off-curve (unsignable) signer key

CATEGORY	Validation
LOCATION	contracts/protocol/ver-endpoint/src/dvn/dvn.ts:665
REVIEW	Remediation Review 1

— DESCRIPTION

When a DVN signer key is registered (`Dvn.create` or `setSigner`), `#assertValidSignerPublicKey` only checks that the string is 128 hex characters — never that the bytes form a point on the secp256k1 curve. A well-formed off-curve value (for example `0x` followed by 128 `f` s) is therefore accepted and counted toward the `size >= quorum` check, yet signature recovery can only ever yield an on-curve key, so no signature can ever match that slot.

If the quorum requires that phantom slot (say two real signers plus one off-curve key at quorum 3), every signature-gated operation — `verify`, `setSigner`, `setQuorum`, admin changes — reverts permanently, and lowering the quorum is itself quorum-gated, so the DVN's verification and governance are bricked with no recovery. Only a trusted deployer or the committee sets signer keys, so this is a self-inflicted configuration footgun rather than an attacker-triggered exploit — hence Low, despite the unrecoverable denial of service.

— RECOMMENDATION

In `#assertValidSignerPublicKey`, additionally parse the key as an uncompressed secp256k1 point (e.g. `secp.Point.fromHex('04' + key.slice(2))`) and reject it when off-curve. A single check at that call site covers both `Dvn.create` and `setSigner`.

— RESOLUTION

Status: Resolved.

LOW **L-20**

RESOLVED

Packet codec fails open on malformed input

CATEGORY	Validation
LOCATION	contracts/protocol/ver-endpoint/src/common/packet-v1-codec/index.ts:45-49
REVIEW	Remediation Review 1

— DESCRIPTION

The shared packet codec decodes fixed-width header and payload fields by slicing the input buffer, but never verifies the buffer is long enough for the field being read; its structural guard checks only that the buffer begins at offset zero. Because `Uint8Array.slice()` clamps to the available length instead of throwing, a packet shorter than a complete header-plus-payload decodes to short or empty values rather than being rejected — a sub-32-byte receiver or guid, a silently empty message, and a payload region that hashes to the well-known empty-string hash.

This diverges from the LayerZero v2 EVM reference, where the equivalent codec reads these fields with Solidity calldata slicing and reverts on any out-of-bounds access, rejecting a malformed packet outright (fail-closed). The TypeScript port fails open: a core wire-format primitive that every packet-decoding site depends on no longer guarantees that a decoded packet was well-formed, and a malformed input is silently reinterpreted as valid-but-different data instead of being rejected.

The exposure today is bounded because the primary receive path performs its own header-length check before decoding, so the fail-open behavior surfaces for a consumer that relies on the codec's own validation. Even so, hardening the shared primitive to fail closed restores the reference behavior and protects every present and future consumer.

— RECOMMENDATION

Add an explicit length assertion at decode time — in `assertValidPacket` or each field extractor — that rejects any packet shorter than the field's end offset, throwing a descriptive error rather than clamping, restoring the EVM reference's revert-on-out-of-bounds guarantee.

— RESOLUTION

Status: Resolved.

LOW **L-21**

RESOLVED

Quadratic DVN-options grouping DoS

CATEGORY	DoS
LOCATION	contracts/protocol/ver-endpoint/src/dvn/dvn-options.ts:190-215
REVIEW	Remediation Review 1

DESCRIPTION

When the send library estimates or pays DVN fees, it first regroups the caller-supplied DVN options blob by `dvn_index`. For each run of options that switches away from the previous index, `groupDvnOptionsByIndex` appends that run into a per-index accumulator via `#insertDvnOptions`, which rebuilds the accumulator as `new ByteCodec().bytes(existing).bytes(newOptions).toBytes()`. Because that expression allocates a fresh buffer and copies the entire growing accumulator on every append, a blob whose entries alternate between just two `dvn_index` values turns each entry into its own run and forces the accumulator to be fully re-copied each time — so the node does work quadratic in the blob length.

Nothing bounds the options length on this path: the option decoder only requires a length of at least 2, the message-size limit applies to the message payload (checked separately), and `MAX_DVN_COUNT` bounds only the number of distinct indices, not the number of runs. A caller that reaches `quote` (fee estimation) or `send` for a destination with a configured default send library can therefore submit a large options blob and force roughly N times the input size in byte copies of CPU and allocation churn on the node executing it. Reachability is gated — the external read paths that reach `quote` are authenticated and allowlisted, `send` is fee-bearing, and Canton bounds the inbound argument size — so this is an authenticated (or paid, quorum-amplified) algorithmic-complexity issue rather than an unprivileged unbounded one, and the grouping is functionally correct with no state corruption or fund loss.

RECOMMENDATION

Replace the concat-into-growing-buffer pattern in `#insertDvnOptions` with the linear approach the options decoder already uses — accumulate per-index chunks in an array and concatenate each index once at the end (a single $O(N)$ pass). Independently, enforce an explicit upper bound on the options length at decode time.

RESOLUTION

Status: Resolved.

LOW L-22

ACKNOWLEDGED

Treasury native-fee ceiling is inert at genesis

CATEGORY	Configuration
LOCATION	contracts/protocol/ver-endpoint/src/message-library/uln/uln-302/uln-302.ts:294
REVIEW	Remediation Review 1

DESCRIPTION

When a message is quoted, the send library adds a treasury fee on top of the total worker (DVN + executor) fee. The treasury fee is computed as $\text{totalWorkerFee} * \text{nativeFeeBp} / 10000$, and `nativeFeeBp` is set by the treasury owner through `setNativeFeeBp`, which accepts any value with no upper bound. To stop the treasury from ever charging a sender more than the workers themselves charge, the send library clamps the treasury fee to $\max(\text{totalWorkerFee}, \text{treasuryNativeFeeCap})$, which is intended to act as a 100%-of-worker-fee ceiling.

The clamp is inert at genesis. The treasury native-fee cap is seeded to $2^{256} - 1$ when the send library is created, so $\max(\text{totalWorkerFee}, \text{cap})$ always resolves to the cap and the clamp never binds. As a result, if the treasury fee is configured above 100% (for example 200%), the quoted and charged treasury fee exceeds the total worker fee and the sender is overcharged. The protective ceiling only starts to apply after the owner performs a separate action to lower the cap to a value at or below the total worker fee, and the cap setter is decrease-only, so it can never be relaxed once tightened.

In the LayerZero v2 EVM send library the equivalent cap is initialized from a finite constructor argument (and is likewise decrease-only), so the 100% ceiling is enforced from the first block. The TypeScript port loses that protection during the window between deployment and the owner explicitly lowering the cap, leaving senders exposed to treasury overcharging in that period.

RECOMMENDATION

Seed the treasury native-fee cap to a finite value at creation rather than $2^{256} - 1$, so the clamp enforces the intended ceiling from deployment, matching the EVM reference. If a configurable initial cap is desired, accept it as a creation parameter and validate it, so the 100%-of-worker-fee protection for senders is active from the first quote. Bounding `setNativeFeeBp` at or below 10000 (100%) would additionally prevent an above-100% configuration regardless of the cap.

L-22 Treasury native-fee ceiling is inert at genesis

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-23**

RESOLVED

Duplicate Keys Make Quorum Unreachable

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/bbb64b47ea976cecf1533f9b208aa5fe60933582/packages/protocol/lz-ver-protocol/sequencer-sdk/src/provider.ts#L87-L93>

REVIEW Remediation Review 1

DESCRIPTION

`parseCantonSequencerUri()` only checks that committee public keys begin with `0x`. It does not validate their encoding or reject duplicates, and quorum is checked against the raw number of entries.

The verifier later deduplicates configured keys and only counts unique recovered signers. Consequently, a committee such as `[A, A]` with quorum 2 is accepted during configuration, but only one unique signer can ever be counted. Malformed keys similarly act as nonexistent committee members.

This can make all quorum-verified sequencer reads and scans fail with Insufficient signature quorum. It cannot bypass signature verification because invalid and duplicate signers are not counted.

RECOMMENDATION

Consider to validate that every committee key is a valid unique public key.

RESOLUTION

Status: Resolved.

LOW **L-24**

RESOLVED

Executor SDK Does Not Support LockUnlockAdapter

CATEGORY	Suggestion
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/2bf54c3f35631ca274aaa02dcb03bd853bd66dd7/contracts/protocol/canton/sdks/src/executor/executor-sdk.ts#L232-L248
REVIEW	Remediation Review 1

DESCRIPTION

`ExecutorSdk.prepareFinalizeCallback` is implemented specifically for `OFT` callbacks: it expects `OftSelfConfig`, recognizes `LzSend_callback`, and builds an `OFT-style` execution context.

It therefore cannot prepare `LockUnlockAdapter` callback finalization.

RECOMMENDATION

Consider adding `LockUnlockAdapter` callback support to the Executor SDK.

RESOLUTION

Status: Resolved.

LOW **L-25**

ACKNOWLEDGED

Sequencer quorum body not bound to agreed hash

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/chain/canton/canton-chain-client.ts:306-308
REVIEW	Remediation Review 1

DESCRIPTION

When the sequencer assembles a write batch, each validator returns a signed response carrying a `preparedTransaction` body and a self-reported `preparedTransactionHash`. Signature verification checks only that the validator's signature is valid over `sha256(preparedTransactionHash)` in `verifyTransactionSignature`; it never re-derives the hash from the accompanying `preparedTransaction` body, and the response schema types that body as unstructured (`z.unknown()`). Responses are then grouped for quorum by the self-reported hash, and once quorum is reached the sequencer submits the first group member's `preparedTransaction` body together with every collected signature in `submitTransaction`. The body that is actually submitted is never bound to the hash the quorum agreed on.

A single validator within the quorum set can therefore poison an otherwise-honest batch. It computes the deterministic honest hash H , signs `sha256(H)` with its registered key — a fully valid signature that joins the honest group — but returns a corrupted `preparedTransaction` body. If its response arrives first, that corrupted body occupies index 0 and is the one submitted, alongside the honest signatures. The participant node recomputes the hash from the submitted body, finds it does not equal H , the signatures fail, and the entire batch is rejected. By repeatedly winning the response race, one validator can indefinitely stall write and genesis batches even with a healthy honest majority.

The impact is confined to liveness and censorship: because the node derives the transaction hash from the actual body and rejects any mismatch, no forged state can be committed and no value is lost. It is rated Medium as a repeatable, protocol-wide liveness failure triggerable by a single member of the validator set, because the aggregation binds only the transaction hash to quorum, not the transaction body that is actually submitted. The read path does not share this gap — it keys quorum on the encoded response body.

RECOMMENDATION

Bind the submitted body to the quorum-agreed hash: re-derive the hash from each response's `preparedTransaction` body and reject any response whose body does not hash to its

L-25 Sequencer quorum body not bound to agreed hash

CONTINUED

reported `preparedTransactionHash` before grouping or submitting, mirroring the read path which already keys quorum on the encoded body. Equivalently, verify each validator signature over the body-derived hash rather than over the self-reported hash field.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

OApp ID collision consumes adapter settlements

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LockUnlockAdapter.daml
REVIEW	Remediation Review 1

DESCRIPTION

`LockUnlockAdapter` exposes `IRequestCallbackView` with only its `oappId`, and callback finalization later dispatches by reading the `PendingCallback`. `AdapterConfig` also accepts any valid `OAppId` without an adapter-specific namespace or implementation discriminator.

The request layer finalizes a `PendingCallback` by accepting any `IRequestCallback` whose `oappIdToAppUID` and admin match the pending callback. It does not bind the callback implementation to `LockUnlockAdapter`. Consequently, if a vetted OFT and a lock/unlock adapter are deployed with the same `(admin, id)`, a visible finalizer can provide the OFT callback CID for a real adapter pending callback.

The reviewed scope docs and contracts do not document or enforce an invariant that forbids this collision. The docs describe `oappId.admin`, appUID binding, callback verification, and `treasury /= oappId.admin`, but they do not state that adapter `oappId` values must be disjoint from OFT `InstrumentId` values. On ledger, `isValidOAppId` only validates the shape of the identifier, and there is no contract key, registry check, namespace prefix, or implementation-kind check that reserves an `(admin, id)` pair for one OApp implementation.

For example, assume the adapter is deployed with `oappId = (Admin, "USDC")`, and an `OftSelf` token is also deployed with `instrumentId = (Admin, "USDC")`. A relayer submits a real adapter `OApp_LzReceive` request for an inbound transfer, and the gateway accepts it, creating a valid `PendingCallback`. A finalizer can then call `IPendingCallback_FinalizeRequest` with the colliding `OftSelf` callback CID instead of the adapter callback CID. The appUID/admin checks pass because both contracts derive the same appUID. The pending callback is archived after `OftSelf` handles the message, no adapter treasury transfer is created, and an unbacked `Oft` holding is minted for the receiver. This is not the prior generic callback forgery issue: the request and callback are both real contracts from vetted packages, and the break comes from cross-implementation appUID collision.

I-01 OApp ID collision consumes adapter settlements

CONTINUED

— RECOMMENDATION

Give adapter callback identity a stable implementation discriminator. For example, derive the adapter appUID from an adapter-specific namespace, or include an OApp kind in the pending callback check so a callback created by `LockUnlockAdapter` can only be finalized by `LockUnlockAdapter`.

Also avoid silently consuming unknown callback functions. Adapter callbacks should use adapter-specific receive and send callback keys, and callback dispatchers should reject unexpected functions instead of returning successfully and allowing `PendingCallback` to archive.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/OAppIdCollisionConsumesAdapterSettlement.daml>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Ownable2Step corrupts inherited state

CATEGORY	Upgradeability
LOCATION	contracts/protocol/ver-endpoint/src/endpoint-v2/endpoint-v2.ts
REVIEW	Remediation Review 1

DESCRIPTION

`EndpointV2.load` now constructs a two-slot `Ownable2Step` between `EndpointBase` and `MessagingChannel`. The previous release constructed the one-slot `Ownable` at the same position. Consequently, the current loader reads every field after the owner from an index one greater than the index used to write it.

This matters because `Container.initialize(previousStateRoot)` deliberately inherits an existing component root without calling its initializer or migrating its contract state. After a normal hard fork, the legacy EID at slot 2 is therefore exposed as `proposedOwner`, while `getEid` reads the empty slot 3 and rejects the endpoint as unconfigured. Every later nonce, verified payload hash, message-library setting and initialization record is also read under the wrong mapping base. Writing the EID again does not recover those mappings. A packet that locked value before the fork can remain undeliverable until another coordinated migration or rollback restores the old layout.

The same one-slot-to-two-slot substitution shifts later state in `SimpleMessageLibrary`, `Uln302`, `Treasury`, `PriceFeed` and `WorkerBase`. This failure does not require a malicious package or trusted administrator. It occurs during the documented state-inheriting hard fork.

RECOMMENDATION

Keep every legacy field at its previous slot. Store the proposed owner in a new explicitly namespaced slot or append it after all legacy fields in each affected contract so that adding it cannot advance the shared allocator before existing state. If the new layout must remain, add a versioned hard-fork amendment that migrates all affected scalar and mapping state before current loaders are used. Do not rely on re-running genesis initializers because inherited components intentionally skip them.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/ver-endpoint/test/Guardian/endpoint-v2-ownable2step-state-inheritance.test.ts>

I-02 Ownable2Step corrupts inherited state

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

RESOLVED

Receive caller may gain settlement veto

CATEGORY	Warning
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzReceive/C all/Impl.daml
REVIEW	Remediation Review 1

DESCRIPTION

`OApp_LzReceive` lets its controller supply `sender` and the adapter forwards that Party unchanged to `Request_Create`. Here, `sender` is the Canton Party that submits and pays for the receive request. It is not the source-chain token sender or the destination receiver. `Request` makes this fee payer a signatory. Normal acceptance then copies the same Party into `PendingCallback`, which is signed by both `sender` and the OApp admin.

The destination receive is split across two Canton transactions:

```
verified packet -> accept Request and consume endpoint payload -> PendingCallback -> treasury  
unlock
```

Validators run `EndpointV2.LzReceive` while processing the Request. The first Canton settlement transaction accepts the Request, commits the resulting endpoint state after the verified payload hash has been deleted and creates `PendingCallback`. The adapter transfers treasury assets to the receiver only when a later transaction consumes that callback. A Party that becomes a callback stakeholder can therefore become a confirmation dependency after the packet is no longer reusable. If that Party's hosting threshold includes an attacker-controlled confirming participant, the attacker can cooperate until Request acceptance and then withhold the confirmation needed to consume `PendingCallback`. The receiver remains unpaid while the backing stays in the adapter treasury. This is a conditional censorship and lockup risk, not theft or unbacked issuance.

The caller does not need an OApp role, `_DEFAULT_ADMIN_ROLE`, `_ENDPOINT_DELEGATE_ROLE` or a malicious DAR. It does need several nontrivial capabilities: a genuine verified packet, the ability to submit before the intended executor, valid explicit disclosures for the adapter/config/request-factory contracts and a Party whose confirming-participant threshold it can withhold. The last condition is not a LayerZero privilege, but it is a deployment and Canton-topology prerequisite. A user Party hosted only by honest protocol-operated participants cannot create this veto merely by refusing in a wallet.

The disclosure prerequisite is not established by the reviewed export. An attacker cannot derive a disclosure from a contract ID or obtain one merely by importing the SDK.

I-03 Receive caller may gain settlement veto

CONTINUED

`CantonClient.createDisclosure` first performs an authenticated event query for the configured `informee`. The Ledger API identity must have the corresponding `canReadAs` authority and contract visibility. `OftSelfSdk.prepareLzReceive` can use such an identity to return a direct receive command and raw `disclosedContracts`, but no production caller or HTTP/RPC route for that method is included in this repository.

There is a credible reuse scenario, but it is also integration-dependent. `prepareLzSend` and `prepareLzReceive` use the same base disclosure resolver. The `LockUnlockAdapter` tests likewise use one `sendDisclosures` bundle for both choices. A disclosed-contract record contains only the template ID, contract ID, created-event blob and synchronizer ID. It is not bound to a recipient, sender, choice or expiry. Therefore, if the production send integration returns the raw adapter disclosure bundle to a wallet, external signer or untrusted sequencer, that recipient can reuse the active contracts for a direct receive submission. Explicit disclosure bypasses visibility only; normal Daml authorization still applies. This is consistent with Digital Asset's [explicit-disclosure model](https://docs.digitalasset.com/build/3.5/sdlc-howtos/applications/develop/explicit-contract-disclosure.html).

The protocol documentation makes this scenario plausible because it describes the sequencer as untrusted and able to submit or order call-stage transactions. A related discovery service in the linked source revision also returns receive disclosure bundles to external OFT users. However, that service resolves the separate burn/mint OFT contracts, not `LockUnlockAdapter` and `AdapterConfig`. It is analogous evidence, not proof of this adapter's production exposure.

The intended executor integration avoids the caller substitution. `Executor.DelegateLzReceive` is controlled by `rich` and hardcodes the nested `0App_LzReceive.sender` to the protocol's `poor` Party. `ExecutorSdk.prepareDelegateLzReceive` does not accept a caller-selected sender. If production forces all inbound submissions through that choice and never releases the reusable adapter disclosures, an ordinary attacker cannot reach the condition described above.

— RECOMMENDATION

Do not rely only on keeping disclosure blobs secret. Either require `0App_LzReceive.sender` to equal the configured protocol executor before calling `Request_Create` or stop copying the fee payer into callback authority. If arbitrary fee payers are required, sign `PendingCallback` only with stable protocol Parties needed for settlement. Keep the caller neither a signatory nor an observer of that contract; store it only as non-stakeholder data if later logic still needs the value.

Confirm how production prepares both sends and receives. In particular, determine whether raw `disclosedContracts` or equivalent reusable input-contract data is returned to

I-03 Receive caller may gain settlement veto

CONTINUED

wallets, external signers or the untrusted sequencer; which OAuth identities have `canReadAs` for `oappGateway` ; whether direct `0App_LzReceive` submissions are accepted; and whether external Parties can use attacker-controlled confirming participants on the relevant synchronizer. Add a multi-participant regression that accepts a receive from such a Party, takes its confirming participant offline and proves callback finalization either remains possible or the receive is rejected before endpoint payload consumption.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-04

RESOLVED

Registered Simple library permits packet forgery

CATEGORY Warning

LOCATION contracts/protocol/ver-endpoint/src/message-library/simple-message-library.ts

REVIEW Remediation Review 1

DESCRIPTION

`SimpleMessageLibrary.send` is publicly dispatchable and never checks that the endpoint called it. An ordinary caller can therefore supply the complete `Packet` and `SendState`, including the source OApp, source EID, destination, nonce, GUID, message, options and library address. The function encodes these values and returns a continuation to `EndpointV2.finishSend`.

The container changes `msgSender` to the Simple library for that continuation. `finishSend` only checks that this new caller is registered and equals `state.library`. It does not prove that `EndpointV2.send` selected the library, incremented the nonce, derived the GUID or constructed the packet. Merely registering Simple is enough. The production server always instantiates the Simple component and registration is append-only. Exploitation therefore requires the endpoint owner to have registered it and the normal library and treasury payees to be active.

An unprivileged caller can invoke Simple directly and impersonate another OApp. The caller supplies the real local EID, the destination's next nonce and the correctly derived packet GUID while placing unrelated values in `SendState`. `finishSend` still emits the canonical `EndpointV2_PacketSentEvent` with the attacker-controlled packet even though `EndpointV2.send` never incremented the victim OApp's outbound nonce. The ordinary Canton entry is also reachable: public non-OApp `Request_Create` binds the runtime caller to the sender's Party address but does not restrict the target contract or function.

A Simple transport that relays canonical `PacketSent` events cannot tell from this event that endpoint nonce and library-selection checks were skipped. The forged packet can name a trusted source OApp and carry an arbitrary value-transfer message. A destination application using that transport can consequently mint or unlock assets without a legitimate source action. This requires no endpoint delegate, OApp admin, malicious package or key compromise. The prerequisite is that the production endpoint has registered the bundled Simple library, activated its payees and uses an off-chain Simple relay that treats the canonical event as send authority. Standard ULN DVN job assignment is not reached by this direct

I-04 Registered Simple library permits packet forgery

CONTINUED

call, so ULN-only deployments are not affected by this sequence. The attacker must also fund enough Request escrow to satisfy the current native-fee handling, which raises the cost but does not require additional privilege.

— RECOMMENDATION

Require `SimpleMessageLibrary.send` to accept calls only from its configured endpoint. Also make `EndpointV2.finishSend` consume an endpoint-created, one-time commitment that binds the encoded packet hash, original sender, destination, selected library, nonce, GUID and options. Do not rely on registration alone as proof that a send began at the endpoint.

If Simple is intended only for tests, remove it from production component configurations and prevent production endpoints from registering it.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

OFT exposes base units as whole CIP-0056 tokens

CATEGORY	Compatibility
LOCATION	contracts/protocol/canton/contracts/Layerzero/Oft/daml/Oft.daml
REVIEW	Remediation Review 1

DESCRIPTION

`Oft` stores LayerZero local base units in the integer `amount` field, but its CIP-0056 `Holding` view publishes `intToDecimal amount` without dividing by `10 ^ localDecimals`. Standard transfer, allocation and burn/mint implementations perform the inverse unscaled conversion: they truncate each caller-supplied `Decimal` amount directly to `Int` and use that integer as a raw `Oft.amount`.

For a six-decimal token, an inbound amount representing one remote token creates an `Oft` with raw `amount = 1,000,000`, while its `HoldingView.amount` is `1,000,000.0`. A standard allocation request for `1.0` consequently locks and transfers raw `1`, returns raw `999,999` as sender change and gives the receiver a raw-one holding whose view reports `1.0`. The native Daml test confirms this behavior using the real inbound callback and allocation contracts.

This is inconsistent with the [CIP-0056 specification](https://github.com/canton-foundation/cips/blob/main/cip-0056/cip-0056.md#relation-to-erc20), the released [token-metadata schema](https://github.com/canton-network/splice/blob/main/token-standard/splice-api-token-metadata-v1/openapi/token-metadata-v1.yaml#L112-L139) and the official [fractional transfer example](https://github.com/canton-network/splice/blob/main/token-standard/README.md#execute-transfer), which use Daml `Decimal` as the token quantity and use `decimals` to restrict display and input precision.

A generic client that follows those interfaces without OFT-specific scaling could display or request unexpected quantities. However, the scoped review did not identify a concrete deployed wallet, DEX, payment application, DvP counterparty, pricing mechanism or counter-asset settlement that relies on this convention. Supported integrations may already apply custom scaling. Therefore, no direct financial loss or exploitable value-transfer scenario is established from the scoped code alone.

I-05 OFT exposes base units as whole CIP-0056 tokens

CONTINUED

RECOMMENDATION

Choose and document one public denomination convention. If generic CIP-0056 compatibility is intended, keep raw integers internally but divide holding views and total supply by `10 ^ localDecimals`; convert standard `Decimal` inputs to raw units with checked multiplication, exact representability and integer-bound checks. If only OFT-specific clients are supported, remove or qualify the generic compatibility claim and document the required scaling in the registry, SDK and integration guidance. Add six- and eighteen-decimal tests covering view, transfer, allocation, burn/mint, metadata supply and send-back conversion.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

ACKNOWLEDGED

StateCommitment accepts non-bytes32 state roots

CATEGORY	Warning
LOCATION	contracts/protocol/canton/contracts/Layerzero/StateTransition/daml/StateCommitment/StateCommitment.daml
REVIEW	Remediation Review 1

DESCRIPTION

`StateCommitment.stateRoot` is documented as a bytes32 state root, but the template precondition only checks `isValidHex stateRoot`. `isValidHex` accepts any non-empty even-length hex string after stripping an optional `0x` prefix. It does not require exactly 32 bytes.

Consequently, a committer can create a `StateCommitment` with a valid hex value of the wrong length. That weakens the canonical encoding expected by off-chain consumers that treat `stateRoot` as a fixed-width bytes32 value. Those consumers may reject, normalize, or misinterpret an otherwise accepted on-ledger commitment.

RECOMMENDATION

Validate `stateRoot` with the existing bytes32 helper instead of the generic hex helper.

```
ensure isValidHexBytes32 stateRoot
```

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-07

ACKNOWLEDGED

Forkable state-commitment genesis equivocation

CATEGORY	Logical Error
LOCATION	Layerzero/StateTransition/daml/Committer/CommitterImpl.daml:47-54; Committer/Committer.daml:21-29; StateCommitment/StateCommitment.daml:9-21
REVIEW	Remediation Review 1

DESCRIPTION

StateCommitment (StateCommitment.daml:9-21) has NO ContractKey, NO maintainer, and NO singleton/uniqueness guard - its only precondition is `ensure isValidHex stateRoot`, with `signatory owner` / `observer viewers`.

Committer.CreateStateCommitment (Committer.daml:21-29) is a `nonconsuming` choice (controller owner) that accepts `prevStateCommitmentCid = None` to mint a genesis commitment. In `createStateCommitmentImpl` (CommitterImpl.daml:47-54) only the `Some` branch fetches+archives the previous commitment (enforcing linear extension of an existing tip); the `None` branch is `pure ()` - no lookup for an existing genesis/tip, no dedup, no archival. Because the choice is nonconsuming and nothing is consumed on the genesis path, the owner can call it with `None` arbitrarily many times, each creating an independent, live genesis chain with a different stateRoot. The single-chain invariant is a comment only ('One active chain per owner', Committer.daml:6-7): enforced by operator discipline, not the ledger.

Combined with per-commitment discretionary viewers (CreateStateCommitment `viewers` / CommitBatch `observers`) flow unfiltered into StateCommitment.observer via `dedup` only, CommitterImpl.daml:100-101), a malicious or compromised owner can build two divergent, individually-valid, version-monotonic chains committing to conflicting roots and disclose each to a disjoint audience. The chains are never co-visible, so the equivocation is undetectable on-ledger. The owner is the oappGateway/request handler used as the cross-chain state anchor (LzSendE2ETest.daml:315-326), so this defeats the non-repudiation / canonical-history purpose of the commitment.

RECOMMENDATION

Enforce a single canonical chain per owner on-ledger rather than by convention:

1. Give the genesis/tip a ContractKey (e.g. keyed on `owner`, maintainer `owner`) so at most one active StateCommitment head can exist; require extension to look up the tip by key and advance it consumingly.

I-07 Forkable state-commitment genesis equivocation

CONTINUED

2. Alternatively, track the current tip inside the Committer via a stateful/consuming choice, and reject `CreateStateCommitment{None}` when a genesis/tip already exists (add an `errCommitter_GenesisExists` assertion via `lookupByKey/fetchByKey`).

To close the equivocation amplifier, bind commitment visibility so all relying verifiers observe the same chain (mandatory shared observer / co-visibility set), and constrain viewers/observers to parties related to the batched requests instead of an unfiltered owner-supplied list.

If a single fully-trusted committer owner is an accepted design assumption, document it explicitly as a trust assumption for the state-commitment anchor (currently it is only an in-line code comment).

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/poc/forkable-state-commitment-genesis/contracts/protocol/canton/contracts/tests/daml/ForkableGenesisPoc.daml>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

ACKNOWLEDGED

Missing cost config skips options validation

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzSend/Call/Impl.daml:105-107
REVIEW	Remediation Review 1

DESCRIPTION

On outbound send, option semantics are validated only inside `assertOptionsCost`. When the destination endpoint has no cost-asserts entry configured, `adapterLzSendImpl` takes the `None` branch and skips `assertOptionsCost` entirely. The only remaining check on the options is the structural hex validation in `combineOptions`.

As a result, for any destination without a cost config, malformed-but-hex options bypass the mandatory options-validity checks that `assertOptionsCost` / `computeOptionsCost` would otherwise enforce (for example an unsupported executor option type). The send proceeds and custody / rate-limit capacity is committed before the endpoint rejects the malformed options downstream.

The underlying issue is that mandatory options-semantics validation is coupled to the presence of an optional cost configuration, so omitting the cost config silently disables it.

RECOMMENDATION

Perform the mandatory options-semantics validation independently of whether a cost-asserts entry exists for the destination (or require a default), so a missing cost configuration cannot disable options validation.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

ACKNOWLEDGED

Options value-cost rounds down, undercharging

CATEGORY	Rounding
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/daml/0App/CostAsserts.daml:120
REVIEW	Remediation Review 1

DESCRIPTION

`computeOptionsCost` computes the value contribution to the options cost as `valueCost = totalValue / shift` and only afterward multiplies the summed cost by `maxPriceRatio`. Dividing by the (large) native-decimal shift before the multiplication discards precision: when `totalValue` is small relative to `shift`, the division rounds down to zero at Daml `Decimal`'s ten-decimal precision, dropping the entire value term from the computed cost.

Because the rounding for sub-precision value amounts is always toward zero, the computed cost is lower than the value obtained by applying `maxPriceRatio` before the division. The resulting undercharge is bounded by the precision loss (sub- 10^{-10} cost-units) and is therefore small, but it is directional: the fee check can pass for slightly less fee than intended.

Reordering the arithmetic to multiply before the precision-reducing division removes the directional undercharge.

RECOMMENDATION

Apply `maxPriceRatio` (and any multipliers) before the precision-reducing division by the native-decimal shift, or otherwise preserve the value term's precision, so the computed options cost does not round the value contribution down to zero.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Config and state contracts over-disclose data

CATEGORY	Best Practices
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml:62-67
REVIEW	Remediation Review 1

— DESCRIPTION

The OFT config and the rate-limiter state contracts bundle all of their security-sensitive data into a single flat record with a broad observer surface. The config carries the full blacklist and whitelist fingerprint sets, per-user exemptions, fee routing and peers; the rate-limiter state carries the full per-EID usage map. Any party in the contract's observer set reads the entire record, not just the field relevant to it.

Because the data is not partitioned (for example by per-user or per-EID sub-records) and the observer sets are coarse, parties gain visibility into compliance lists, per-user exemptions and aggregate bridge volumes beyond what their role requires. This is a least-disclosure / data-minimization gap rather than an authentication bypass.

The effect is confidentiality-only: no unauthorized state change results, but sensitive compliance and volume data is more widely visible than necessary.

— RECOMMENDATION

Partition the sensitive data so observers see only what their role requires (for example per-user or per-EID sub-records), and narrow the observer sets on the config and rate-limiter-state contracts so compliance lists, exemptions and usage volumes are not broadly readable.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

RATE_LIMITER_MANAGER can nullify rate limit

CATEGORY	Access Control
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/AdapterConfig.daml:177-183
REVIEW	Remediation Review 1

— DESCRIPTION

The adapter configuration exposes a rate-limiter management choice that forwards a caller-supplied state entry directly to the rate-limiter state contract as a raw overwrite. The entry carries the accumulated outbound and inbound usage counters, so a caller can set them to any value. The choice is gated by the `RATE_LIMITER_MANAGER_ROLE`, which the admin may delegate to a non-admin party. A manager who resets the usage to zero bypasses the rate-limit cap (drain up to the limit, reset, repeat); one who sets it to the maximum blocks a direction (denial of service).

This is rated Informational because it is an operational trust consideration rather than a new externally-exploitable flaw: the same manager role already controlled the rate-limiter configuration in the prior release — it could raise the configured limit to effectively disable the cap — so the raw state overwrite is a stealthier route to a power the role already held. The rate limiter provides protection only to the extent the manager role is trusted.

— RECOMMENDATION

Treat `RATE_LIMITER_MANAGER_ROLE` as a high-trust role, held with the same care as the config admin, since it can nullify the rate-limit protection. Alternatively, restrict the manager to principled, checkpoint-based state updates (as the sibling checkpoint choices already do) so it can tune limits without silently resetting accumulated usage.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

ACKNOWLEDGED

Config create skips transfer-rule check

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ft/daml/0ftSelfConfig.daml:159
REVIEW	Remediation Review 1

DESCRIPTION

The OFT config's update path, `setTransferRuleImpl`, enforces that a new transfer rule's instrument matches the config's instrument and that the config admin is a signatory of the rule before repointing at it. The create-time `ensure` clause performs neither check: it validates the OFT id, decimals, fee, peer, rate-limiter, enforced-option and cost-assert fields, but never inspects the transfer-rule reference.

A config can therefore be created whose transfer-rule reference points at a rule belonging to a different instrument (or one the admin does not control). The invariant that `setTransferRuleImpl` relies on holds only after the first update, not from creation, so the initial config can be internally inconsistent and transfers executed under the mismatched rule fail.

The transfers are recoverable (the sender can withdraw the pending offer), so no value is permanently locked; the issue is the create-vs-update validation asymmetry.

RECOMMENDATION

Apply the same instrument-identity and admin-signatory check enforced by `setTransferRuleImpl` inside the create-time `ensure` /creation flow, so a transfer rule bound at creation satisfies the same invariant as one bound by an update.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-13

ACKNOWLEDGED

Options decoder overflows on large native values

CATEGORY	Error
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/daml/0App/OptionsDecoder.daml:177-190
REVIEW	Remediation Review 1

DESCRIPTION

`decodeExecutorOption` reads each u128 native-value option field — the `lzReceive` value, the native-drop amount, and the `lzCompose` value — with a fixed 32-nibble `readHex`, which routes through `hexToInt`. `hexToInt` throws an integer-overflow error for any value that does not fit a signed 64-bit integer (at or above 2^{63}). Native-value option fields are wei amounts that can legitimately exceed 2^{63} (roughly 9.2 in whole-ether terms), so `computeOptionsCost` aborts during decoding, before the intended native-value cap and insufficient-fee checks can run.

The `CostAsserts` comments state that the cost formula performs all arithmetic in `Decimal` to avoid `Int64` overflow on large native values, but that protection is illusory on this path: each native value has already been truncated through the `Int64`-bounded `hexToInt` in `decodeExecutorOption` before it reaches the `Decimal` sum. Any send whose combined options carry a native value at or above 2^{63} is therefore rejected with a generic decode error, and an admin-enforced option (prepended to every send for a destination endpoint and message type) carrying such a value bricks all sends to that destination.

This is the same `hexToInt` /`Int64` supported-maximum limitation reported for inbound amounts in I-08 ("Inbound amount $\geq 2^{63}$ SD undeliverable"). There the remediation both documented the supported maximum and added an explicit bound; the same treatment was not applied to this options native-value decode path, which the remediation left unchanged. The issue takes effect only when a cost-assertion config exists for the destination endpoint, so the practical impact is limited — hence informational.

RECOMMENDATION

Apply the same treatment the inbound-amount path received: either decode the u128 native-value option fields into a representation that spans their full range (read them directly into `Decimal` or an unbounded integer rather than through the `Int64`-bounded `hexToInt`), or explicitly document and bound the supported maximum so that legitimately large native values are handled deterministically instead of aborting during decode. Correct the

I-13 Options decoder overflows on large native values

CONTINUED

`CostAsserts` comment so it no longer claims an Int64-overflow safety that this path does not provide.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Default DVN Configs Accept Unregistered DVNs

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/dc77a706e26c0ce1f7b956c79205d905cb618456/contracts/protocol/ver-endpoint/src/message-library/uln/uln-302/uln-302.ts#L564-L595
REVIEW	Remediation Review 1

— DESCRIPTION

The default send and receive configuration setters validate the structure of the ULN configuration but do not verify that its required and optional DVNs are registered in `#dvns`.

Consequently, the LZ owner can store default configurations containing unusable DVN addresses. Send operations later revert when attempting to load these DVNs, while receive configurations may be unable to satisfy their required verification threshold.

The equivalent OApp configuration paths already reject unregistered DVNs.

— RECOMMENDATION

Consider to check that every referenced DVN is registered in `#dvns`, before storing a default send or receive configuration.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

Default Config Accepts Unregistered Executors

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/dc77a706e26c0ce1f7b956c79205d905cb618456/contracts/protocol/ver-endpoint/src/message-library/uln/uln-302/uln-302.ts#L598-L621>

REVIEW Remediation Review 1

DESCRIPTION

`setDefaultExecutorConfigs()` verifies that the executor address is non-zero and that the maximum message size is valid, but does not verify that the address exists in `#executors`.

Although `#isSupportedSendEid()` subsequently reports such a route as unsupported, the invalid configuration is still stored. Updating an existing route to an unregistered executor can make `quote()` and `send()` revert.

RECOMMENDATION

Consider to check `#executors.has(config.executor)`, before storing a default executor configuration.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-16

ACKNOWLEDGED

Worker defaultMultiplierBps unbounded (uint16)

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/dc77a706e26c0ce1f7b956c79205d905cb618456/contracts/protocol/ver-endpoint/src/worker/worker-base.ts#L231-L235
REVIEW	Remediation Review 1

DESCRIPTION

Solidity's `Worker.setDefaultMultiplierBps(uint16)` caps the fee multiplier at 65535 (~6.5535x) via ABI decoding. The TypeScript port types `multiplierBps` as an arbitrary `bigint`: neither `setDefaultMultiplierBps` (`worker-base.ts:231-235`) nor `initializeWorker` (`worker-base.ts:128`) bounds it before persisting via `BigUintState.set()` (a `uint256` field), and the event ABI is also `uint256`, so nothing enforces the range. The value flows into `FeeLibBase.applyPremiumToGas` (`fee-lib-base.ts:50`) as `(fee * multiplierBps) / 10_000n`.

This is not exploitable: the setter is admin-gated (`assertAdmin`, `worker-base.ts:232`) and the multiplier is the operator's own discretionary pricing knob, not a sender-protecting invariant. Downstream use is pure `bigint` math with no overflow.

The gap is isolated to this one field. The sibling per-`dstConfig` multipliers keep the `uint16` bound (`dvnDstConfigAbi` / `executorDstConfigAbi` declare `multiplierBps` as `uint16`, so `#dstConfigs.set(...)` throws on any value `> 65535`); only `defaultMultiplierBps` escapes it.

Proof of concept: a worker admin calls `setDefaultMultiplierBps(10n ** 18n)`. The value persists (admin-gated) and every subsequent quote from that worker computes `fee * 10^18 / 10000`, inflating that operator's own native fee beyond what the `uint16`-bounded Solidity implementation could produce. Only that operator's service pricing is affected; senders choose their workers.

RECOMMENDATION

Optionally validate `multiplierBps <= 65535` in `setDefaultMultiplierBps` and `initializeWorker` for parity with the Solidity `uint16` domain, matching the bound the per-`dstConfig` multipliers already enforce through their `uint16` ABI codecs. Not required for correctness.

I-16 Worker defaultMultiplierBps unbounded (uint16)

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

Bearer tokens never expire, audience unenforced

CATEGORY	Access Control
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/auth/jwt.ts:9-15
REVIEW	Remediation Review 1

DESCRIPTION

The bearer-token layer that protects every non-health endpoint on the sequencer and validator services has three hardening gaps. `signToken` sets an issued-at time but never an expiration, so a minted token is cryptographically valid forever. `verifyJwt` never passes the `audience` claim (or a `maxTokenAge`) to `jwtVerify`, so the audience — though set at minting — is not enforced, and a token's scope rests entirely on the static jti allowlist. That allowlist is read once from configuration at process start, so revoking a leaked token id requires editing the environment and restarting the service.

This is not an authentication bypass: tokens cannot be forged without the shared secret, and these are operator-provisioned service-to-service credentials. But the gaps remove the standard time-bound, audience-scoping, and hot-revocation defenses from the sole auth control on these services. A token that leaks (via logs, a proxy, or a client compromise) grants unbounded-duration access until an operator notices and redeploys, and if the same secret is reused across the sequencer and validators the missing audience check means cross-service isolation depends solely on the allowlists not overlapping.

RECOMMENDATION

Set a bounded expiration when minting and pass `audience` (and rely on `exp`) to `jwtVerify`; support live allowlist reload or short-lived tokens so a leaked id can be revoked without a redeploy; provision distinct secrets per service.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

ACKNOWLEDGED

Read endpoint signs reads at caller stale root

CATEGORY	Validation
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/controllers/validator.ts:138-146
REVIEW	Remediation Review 1

— DESCRIPTION

On the validator read endpoint the executed state is `request.stateRoot ?? findCurrent().stateRoot`, and the request schema accepts any 32-byte value. When a caller supplies a root, the normal path that resolves and version-validates the current committed commitment is skipped, and the read executes directly against the supplied root. The sequencer forwards the same root to every validator, so all validators read at it, reach quorum on the body, and return a quorum-signed response bound to it — unlike the scan and write paths, which pin to the current committed root.

The backing store retains only genuine historical roots, so a fabricated root simply errors rather than returning invented data; the residual risk is that an authenticated caller can obtain a validator-quorum-signed read taken at a stale (previously committed) root of their choosing. Every response carries the exact root it was computed at, so the attestation itself is truthful — but a downstream consumer that treats "quorum-signed" as "current" without comparing the returned root to the latest committed root can be served attacker-selected stale-but-genuine state.

— RECOMMENDATION

On the read path, when a caller supplies a stateRoot, verify it resolves to a persisted/current commitment (reuse the write path's `StateCommitmentFinder` / version validation, or reject roots that are not the latest committed), mirroring the scan controller's committed-root pinning.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-19

ACKNOWLEDGED

Free preapprovals retain receiver fee inputs

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/TransferPreapprovalFactory/damL/Preapproval.damL
REVIEW	Remediation Review 1

DESCRIPTION

The factory asks the receiver to supply Amulet holdings to cover the cost of creating or renewing a transfer preapproval. Its documented behavior is to use those holdings to pay the fee and return any unused value to the receiver. Even when the preapproval is free, the receiver cannot call the factory with no fee input. An empty list has a total value of zero and the factory rejects zero-value transfers. A successful call must therefore include at least one positive-value holding.

Before asking Splice to create or renew the preapproval, the factory transfers the full balance of every supplied holding from the receiver to the provider. This transfer is meant to be temporary: Splice should charge the required fee and the factory should return the remainder.

Splice 0.1.20 provides the first 90 days of a preapproval for free by default. For a request within that period, Splice charges zero. Because no payment is required, it does not consume the provider-owned input and does not create a separate change holding. It returns `amuletPaid = 0.0` and `senderChangeAmulet = None`. In this situation, `None` means that no new change holding was created; it does not mean that no unspent value remains.

The factory does not make that distinction. It only issues a refund when Splice returns a change holding. When Splice returns `None`, the factory completes the preapproval without transferring anything back. The original input remains active, but it is now owned and spendable by the provider.

Whatever positive amount the receiver supplies is retained in the free case. The receiver can reduce the loss by selecting a smaller holding, but cannot avoid supplying a positive input while using the current factory flow. The same helpers are used for creation and renewal, so both operations are affected.

RECOMMENDATION

Preserve the provider-owned input CID and inspect `result.amuletPaid`. When the reported fee is zero, transfer the untouched input back to the receiver even though

I-19 Free preapprovals retain receiver fee inputs

CONTINUED

`senderChangeAmulet` is `None`. Alternatively, determine that the requested duration is free before moving any receiver funds and call `Splice` with an empty input list. Continue using `senderChangeAmulet` for paid requests, where `Splice` actually consumes the input and creates change.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-20

ACKNOWLEDGED

Compose sends burn assets Canton cannot receive

CATEGORY	Logical Error
LOCATION	contracts/protocol/canton/contracts/Layerzero/OftCommon/daml/OApp/LzSend/Call/Prepare.daml:84-92
REVIEW	Remediation Review 1

DESCRIPTION

`prepareSend` treats every nonempty `composeMsg` as a supported `SEND_AND_CALL` message: it selects the send-and-call enforced options and encodes the compose payload into the outbound packet. The shared `validateSendArgs` never rejects it, so an ordinary user — with no admin, minter, or delegate role — can submit a compose send. Both the OFT and the LockUnlockAdapter then create a real settling source request and place the caller's principal into cross-chain custody, and `OftSelf_BuildQuote` returns a normal quote for the same send.

The current Canton receiver cannot process the packet these functions produce.

`validateLzReceiveRequest` requires every inbound message to have exactly the 80-character non-compose length, so every compose message is rejected deterministically before any destination Request is created. This rejection happens after the source has already settled and cannot reverse it:

`handleLzSendAccept` archives the locked OFT holding with no message-type check (the adapter likewise finalizes the outflow while the cross-chain amount stays in treasury), and no callback connects a later destination-validation failure back to the source reject path.

An ordinary user can therefore submit a currently-accepted Canton-to-Canton compose send of arbitrary size: the source OFT is burned or the adapter asset is locked in treasury, while the destination can never mint or unlock the corresponding value. There is no automatic refund; recovery requires privileged compensation or a protocol upgrade. No malicious package, administrator action, or endpoint-delegate role is required — it is the direct-loss consequence of the outbound path advertising and accepting `SEND_AND_CALL` while the inbound path rejects it.

RECOMMENDATION

Reject nonempty `composeMsg` on the send side — before quoting, rate-limit accounting, locking, or creating the source Request — whenever the configured destination does not support compose (fail closed). If compose must remain available for specific destinations, gate it behind an authenticated per-EID capability flag. Until such a capability exists, the

I-20 Compose sends burn assets Canton cannot receive

CONTINUED

safest change is to reject all nonempty compose payloads in `validateSendArgs` and remove the Canton `SEND_AND_CALL` quote and send paths, so a user's principal is never locked or burned for a transfer the destination cannot deliver.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/ComposeSendBurnsAssets.daml>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-21

ACKNOWLEDGED

Small outflows erase larger rate-limit usage

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml
REVIEW	Remediation Review 1

DESCRIPTION

`applyRateLimit()` rounds every positive transfer up before recording it:

```
scaledAmount = ceil(rawAmount / 10scaleDecimals)
```

Rounding up is conservative when `scaledAmount` is added to usage because a small transfer cannot become zero. The same rounded value is also subtracted from usage in the opposite direction when net accounting is applied. Rounding up is unsafe for this credit because it can cancel more raw usage than the reverse transfer represents. For example, consider the supported configuration `localDecimals = 8`, `sharedDecimals = 6` and `scaleDecimals = 3`. The local/shared-decimal difference means that cross-chain amounts are multiples of `100` raw units. The limiter nevertheless groups amounts into units of `1,000` and rounds up:

```
ceil(100 / 1,000) = 1 limiter unit
```

```
ceil(1,000 / 1,000) = 1 limiter unit
```

The limiter therefore treats a `100`-unit transfer and a `1,000`-unit transfer as equal even though one is ten times larger. With an inbound limit of one limiter unit, an ordinary token holder can perform the following genuine transfers without waiting for decay:

```
Initial state: outboundUsage = 0, inboundUsage = 0
```

```
Receive 1,000 units inbound: outboundUsage = 0, inboundUsage = 1
```

```
Send 100 units outbound: outboundUsage = 1, inboundUsage = 1
```

```
Accept the 100-unit outflow: outboundUsage = 1, inboundUsage = 0
```

```
Receive another 1,000 inbound: outboundUsage = 0, inboundUsage = 1
```

Accepting the `100`-unit outflow calls `CommitOutflow`. It subtracts the outflow's rounded value of `1` from the inbound usage of `1`, so the first `1,000`-unit inbound charge is erased completely. Correct raw accounting would subtract only `100` from `1,000` and leave `900` units of inbound usage. Only `100` units of capacity would remain, so the second `1,000`-unit inbound transfer should fail. Instead, the second inbound transfer succeeds. The user has received `2,000` units and sent only `100` units in the opposite direction, for `1,900` units of net inbound movement during a window intended to allow `1,000`. The sequence can be repeated because each accepted `100`-unit outflow clears the charge for another

I-21 Small outflows erase larger rate-limit usage

CONTINUED

`1,000`-unit inflow. The current adapter validation only requires `scaleDecimals <= localDecimals`. It does not ensure that the limiter scale is at least as precise as the smallest cross-chain amount, `10^(localDecimals - sharedDecimals)`. Exploitation requires net accounting, a coarse scale and genuine transfers in both directions through the same bucket.

— RECOMMENDATION

Perform net accounting in full raw precision or store enough remainder information to prevent a credit from exceeding the raw transfer it offsets. If the implementation must net scaled integers, use conservative asymmetric rounding: forward charges may round up, but backward credits must round down and must never cancel more raw usage than the opposite transfer represents.

At minimum, enforce `rateLimiterConfig.scaleDecimals <= localDecimals - sharedDecimals` in `AdapterConfig` and equivalent OFT configurations. This makes the limiter at least as precise as the smallest transferable cross-chain unit. Add regression tests where different raw amounts fall into one scale bucket and assert that the smaller transfer cannot erase the larger transfer's usage.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-22

ACKNOWLEDGED

Rejected writes shorten library grace periods

CATEGORY Unexpected Behavior

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/container/container.ts

REVIEW Remediation Review 1

DESCRIPTION

`Container.call()` increments the global nonce before dispatching a write. When contract execution throws `UserError`, the catch block returns the post-increment root and nonce instead of the original root. The production request executor treats that application-level rejection as a successful container update, so the increment is included in the next state commitment.

The endpoint also uses this request-count nonce as `Context.blockNumber`. Receive-library grace periods expire when `timeout.expiry <= context.blockNumber`. Consequently, rejected requests consume the same clock that protects in-flight messages during a receive-library migration.

An ordinary requester can submit calls with nonexistent functions or other deterministic user errors. Each rejected write advances the clock without a successful protocol state transition. The native proof replaces a receive library with a three-block grace period, submits two invalid writes and advances the virtual block from 5 to 7. The old library changes from valid to expired, after which its otherwise valid `verify()` call fails with `EndpointV2_InvalidReceiveLibraryError`.

This can invalidate the grace period before delayed messages authenticated by the old library arrive. Destination delivery then remains unavailable until the owner restores a compatible library or the message is verified again under the new configuration. The attacker must pay for enough Canton requests to consume the configured block count.

RECOMMENDATION

Do not use the request-sequencing nonce as the security clock for receive-library timeouts. Base grace expiry on a consensus timestamp or another clock that untrusted callers cannot advance by filling one request batch.

I-22 Rejected writes shorten library grace periods

CONTINUED

If the virtual block counter must remain, advance the timeout clock only after a successful state transition. Keep any nonce needed to order or deduplicate rejected requests separate from `Context.blockNumber`.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-23

ACKNOWLEDGED

No on-ledger inbound replay / nonce / guid dedup

CATEGORY	Logical Error
LOCATION	Layerzero/OftCommon/daml/OApp/LzReceive/Call/Validate.daml (validateLzReceiveRequest); Layerzero/Oft/daml/OApp/LzReceive/Callback/Impl.daml (lzReceive); Layerzero/I0App/daml/I0App.daml (OApp_LzReceive); Layerzero/Request/daml/Request.daml; Layerzero/Request/daml/PendingCallback.daml
REVIEW	Remediation Review 1

DESCRIPTION

The inbound cross-chain receive path has no on-ledger replay protection. A cross-chain message is identified by its origin (srcEid, sender, nonce) and its guid, but nothing on the ledger ever records or consults a set of consumed identifiers, so one endpoint-verified message can be delivered and minted more than once.

validateLzReceiveRequest only format-checks the inbound request: it asserts guid is bytes32, srcEid > 0, sender is bytes32, nonce > 0, an amount-overflow bound, and that origin.sender matches the configured peer. It never reads or writes any "already processed" state.

OApp_LzReceive is a nonconsuming choice whose controller is a caller-supplied sender, with the gateway only an observer. Each invocation creates a fresh Request, so the same (origin, guid, lzMessage) can produce multiple independent Requests. When a Request is accepted it spawns a PendingCallback, and finalizing that callback runs lzReceive, which unconditionally creates an Oft holding (the mint) for any positive amount with no idempotency key, no re-check of the peer, and no consumption of the guid/nonce.

The PendingCallback exactly-once property does not help: it guarantees a single callback contract is finalized once, but two callbacks derived from the same source message are distinct contracts and each mints independently.

This is a defense-in-depth gap rather than an anonymous-attacker exploit. The mint is gated by the handler (the gateway signs the request accept), so an honest gateway that deduplicates off-ledger masks the issue. It becomes a real double-mint under a duplicating, buggy, or compromised handler, or an accidental relayer retry. It is also the notable divergence from EVM LayerZero, where the Endpoint enforces inbound nonce ordering and clears the payload hash on receive, i.e. replay protection is on-chain; here that guarantee lives entirely off-ledger.

PoC: It reuses the repository's own receive-test setup and drives the full lifecycle (OApp_LzReceive -> Request -> IRequest_Accept -> PendingCallback -> IPendingCallback_FinalizeRequest -> create Oft) twice with a byte-for-byte identical message (same origin, same

I-23 No on-ledger inbound replay / nonce / guid dedup

CONTINUED

guid, same lzMessage). The receiver ends with two holdings totalling twice the sent amount. Contrast the repo's `testLzReceiveMultiple`, which runs the identical loop but with distinct guids.

— RECOMMENDATION

Add an on-ledger, single-use record of each consumed inbound message so replay protection does not depend solely on the off-ledger endpoint. Persist a dedup marker keyed by guid (or by (srcEid, sender, nonce)), created with a contract key and admin/gateway authority, and assert-and-create it inside the accept/finalize path so that a second finalize of the same message fails the uniqueness constraint. This mirrors the on-chain replay guarantee the EVM LayerZero Endpoint provides (inbound nonce ordering plus payload-hash clearing). Until that is in place, the off-ledger gateway must be treated as fully trusted for inbound deduplication and must guarantee each guid/nonce is delivered and accepted at most once.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/6>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-24

ACKNOWLEDGED

Executor fees ignore native-drop fanout

CATEGORY	Logical Error
LOCATION	contracts/protocol/ver-endpoint/src/executor/fee-lib.ts, contracts/protocol/ver-endpoint/src/message-library/uln/uln-302/uln-302.ts
REVIEW	Remediation Review 1

DESCRIPTION

An ordinary sender can attach many `NativeDrop` instructions to a single LayerZero send. Each instruction asks the destination executor to transfer native value to a separate receiver. The executor fee includes the total native value being distributed, but it does not increase with the number of receivers, the bytes needed to identify them or the transfer work that each additional receiver creates. A sender can therefore turn a quote for one native transfer into a request for hundreds of transfers without paying a corresponding execution fee.

The discrepancy starts in `Uln302`. When it requests an executor quote, it sets `calldataSize` to `packet.message.length`. Although the function also separates and forwards the executor options, their encoded length is not added to `calldataSize`. `ExecutorFeeLib` later parses every `NativeDrop` entry and adds its `amount` to `totalValue`, but the receiver address and the existence of another transfer do not affect the quote. There is no per-drop gas charge, drop-count charge or calldata charge for the receiver list.

The proof of concept compares one drop of 312 native base units with 312 distinct drops of one base unit each. Both requests ask for the same total native value and both receive the same complete endpoint quote of 101,337 units. However, the second request contains more than 16 KiB of additional executor options and requires 311 additional destination transfers. Under the configured fee model, charging for the omitted option bytes alone would add 258,752 units before accounting for any per-transfer execution cost.

This fanout fits within the source-side request limit. The 312 encoded drop options occupy 32,492 text bytes. Together with the map keys, destination EID, receiver and adapter message, the complete `params` argument is 32,669 bytes, which remains below the 32,768-byte limit. A 313th drop would exceed that limit, so 312 is the largest fanout admitted for the normal non-compose adapter request used by the proof. `nativeCap` does not reduce this count because it checks only the sum of the dropped value; using one positive base unit per receiver keeps the total small.

The unpriced work is performed by the scoped Canton executor. `Executor._nativeDrop` passes the complete receiver map to `distributeAmulet` and `distributePayouts` sequentially

I-24 Executor fees ignore native-drop fanout

CONTINUED

exercises `TransferFactory_Transfer` once for every distinct recipient. Consequently, honoring a 312-recipient request requires 312 value-transfer operations. If any transfer fails, the atomic native-drop transaction aborts. An executor that completes the request pays the omitted calldata and execution costs from its own operating funds. An executor that refuses the request or cannot process it within destination limits leaves a fully paid native-drop request undelivered. Repeated requests can compound the subsidy, consume worker capacity, congest delivery or repeatedly cause paid native-drop delivery to fail.

The request does not require an admin role, a malicious package, a fake interface or a compromised key. It is available wherever the destination and selected executor support `NativeDrop`, the aggregate amount is below `nativeCap` and the OApp is admitted by the worker. These are ordinary conditions for using the feature. The sender directly controls `extraOptions`. When no OApp allowlist is configured, the worker ACL is open. When an allowlist is configured, it identifies the shared OApp rather than the individual sender, so it cannot distinguish one user's high-fanout request from other traffic through that OApp.

For example, a user can submit an admitted request containing the mandatory receive option and 312 one-unit drops to distinct receivers. The source accepts the 32,669-byte argument and the aggregate amount remains below `nativeCap`. `EndpointV2` and `Uln302` then return the same fee charged for one 312-unit drop. After the user pays that quote, the executor must either perform all 312 transfers at the unquoted cost or withhold the requested delivery. An executor-side policy outside this repository could cap or split large fanouts, but quote and send do not enforce the same policy. Such a policy would therefore change the immediate result from executor subsidy to paid-request nondelivery without correcting the pricing discrepancy.

— RECOMMENDATION

Price native-drop work explicitly. Count decoded native-drop entries and add a destination-specific `nativeDropBaseGas * count` term. Include the encoded executor options or the exact destination-call encoding derived from them, in the calldata-size input used by every fee model.

Also set a conservative per-message native-drop count and options-byte limit that the destination executor can always process. Enforce those limits in both quote and send paths. Where Daml `CostAsserts` remains in use, update it to enforce the same count and size terms or verify a short-lived runtime quote instead of maintaining a second incomplete formula.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/commit/afe1be9d2a8e85f0aa4457933522f083fef56bdd>

I-24 Executor fees ignore native-drop fanout

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-25

ACKNOWLEDGED

Endpoint IDs not bounded to uint32 domain

CATEGORY Validation

LOCATION `contracts/Layerzero/I0App/daml/Types.daml (SendParam.dstEid, Origin.srcEid as Int); contracts/Layerzero/Oft/daml/OftSelfConfig.daml (peer-key validation, setPeerImpl); contracts/Layerzero/OftCommon/daml/OApp/LzSend/Call/Validate.daml (validateSendArgs); contracts/Layerzero/Oft/daml/OApp/LzSend/Call/Impl.daml; contracts/Layerzero/LockUnlockAdapter/daml/AdapterConfig.daml`

REVIEW Remediation Review 1

DESCRIPTION

LayerZero endpoint identifiers are externally defined in the uint32 domain, so a valid EID must satisfy `1 <= eid <= 4294967295`. The Canton implementation models EIDs as generic Daml Int and validates them only as positive. This affects outbound `SendParam.dstEid`, inbound `Origin.srcEid`, peer configuration, pause sets, fee maps, enforced options, cost assertions, and rate-limit EID config. Daml Int can hold values above `4294967295`, so values such as `4294967296 (uint32_max + 1)` can be installed as peers and accepted by quote/send preparation even though they are outside the canonical endpoint domain.

The issue is not on-ledger truncation: Canton preserves the oversized value and can carry it into request context as `AV_Int`. The defect is that local Canton state accepts and propagates an EID that should be rejected before quote, custody, rate accounting, or request creation. If the off-ledger bridge enforces uint32 it may reject the request or fail encoding; if any integration performs unsafe coercion it may wrap/truncate. That integration behavior is secondary; the confirmed Canton bug is the missing upper-bound validation before downstream use.

Impact: An authorized config can register an invalid peer, a quote can be produced, and a send can proceed far enough to consume rate capacity, lock OFT value, and create a Request. This alone does not prove permanent loss (the handler can reject and the OFT reject/finalize path unlocks principal and reverses rate usage), so residual risk is request/fee exposure, operational state pollution, and unresolved pending state if combined with missing timeout/recovery.

RECOMMENDATION

Two complementary layers are needed. Note DAML has no native `uint32` (or any unsigned/fixed-width) integer type, so this is not a primitive field swap.

1. Type-level (structural). Introduce an opaque Eid newtype (`newtype Eid = Eid Int`) whose smart constructor enforces `0 < eid <= 4294967295` as the only way to build a value, and migrate endpoint fields and EID-keyed collections to it: `SendParam.dstEid`, `Origin.srcEid`, `peers`, `pausedDstEids`, `issuerFeeBpsPerEid`, `enforcedOptions`, `costAsserts`, and

I-25 Endpoint IDs not bounded to uint32 domain

CONTINUED

rate-limit overrides. This makes the range correct-by-construction and stops future code paths from reintroducing an unchecked `Int`. A rename or bare type alias is insufficient: only a smart-constructor newtype (or explicit validation) actually enforces the range.

2. Boundary validation (behavioral). The wrapper does not remove the need for explicit checks. Raw `Int` still enters at ingress the type cannot cover: choice arguments submitted from off-ledger, values decoded from generated SDKs, and call-context payloads carried as `AV_Int`. Each `Int -> Eid` conversion is where the predicate must run. Apply the same `0 < eid <= 4294967295` predicate at every EID ingress plus config ensure clauses and `SetPeer`, and reject invalid EIDs before any quote result, holding lock, fee transfer, rate-limit transition, or request creation.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/7>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-26

ACKNOWLEDGED

Pending outflows use the wrong decay parameters

CATEGORY	Logical Error
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml
REVIEW	Remediation Review 1

DESCRIPTION

`remainingContribution` calculates how much of a pending outflow remains by applying one limit and window from `recordedAt` until callback settlement. The function receives the current configuration, even when the outflow was recorded under an older configuration.

Configuration updates checkpoint the aggregate usage bucket under the old parameters before installing the new parameters. They do not checkpoint each pending outflow or reset its `recordedAt`. The scoped `buildAdapterLzSendCallbackContext` stores only `scaledAmount`, `resolvedEid`, `forwardRecorded` and `recordedAt`. The scoped accept and reject handlers later resolve the rate-limiter configuration from the finalization-time `AdapterConfig` and pass it to `IRateLimitState_CommitOutflow` or `IRateLimitState_ReverseOutflow`. Consequently, the shared state implementation applies the new decay rate retroactively to time that elapsed before the update.

This is an incomplete fix of historical M-01. That finding covered subtraction of the full original amount after ordinary decay and was acknowledged as fixed. The new `recordedAt` calculation handles delayed callbacks while parameters remain unchanged, but it does not preserve configuration epochs when a limit or window changes.

For example, the test records an outflow of 1,000,000 units with a 3,600-second window. After 1,800 seconds, the checkpoint correctly leaves 500,000 units. The manager then changes the window to 36,000 seconds and a second user records 400,000 units, bringing aggregate usage to 900,000. Rejecting the first outflow should remove its remaining 500,000 units and leave the second user's 400,000 units. Instead, the current calculation treats 950,000 units of the first outflow as remaining and floors the aggregate bucket to zero.

This can erase usage belonging to later users and reopen rate-limit capacity. Changing to a faster decay rate has the opposite effect and can leave phantom usage that blocks transfers. Exploitation does not require a malicious administrator. It only requires a legitimate decay-parameter update while an outflow remains pending, followed by its normal accept or reject callback.

I-26 Pending outflows use the wrong decay parameters

CONTINUED

— RECOMMENDATION

Do not calculate a pending contribution by applying the current configuration across its entire lifetime. Track enough on-ledger information to calculate decay across each configuration epoch or checkpoint every pending contribution when decay parameters change. A simpler conservative alternative is to prohibit limit or window changes while pending outflows exist.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782226988410/blob/remediations/contracts/protocol/canton/contracts/tests/daml/Guardian/PendingOutflowWrongDecay.daml>

— RESOLUTION

Status: Acknowledged.

CRITICAL **C-01**

RESOLVED

Head race commits a stale-derived root

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/chain/canton/canton-chain-client.ts:189-221
REVIEW	Remediation Review 2

DESCRIPTION

`ValidatorController.#write` reads the current `StateCommitment` and executes the selected Requests from that commitment's `stateRoot`. However, `buildValidatorTransactionPayload` does not receive the commitment ID that was used for execution. `CantonChainClient.#buildCommitBatch` instead calls `getLatestCommitment()` again and uses the result as `prevStateCommitmentCid`, while keeping the root calculated from the earlier state.

If another authorized writer advances the head between these two reads, validators can sign a successor whose declared predecessor does not match the state from which its root was calculated. For example, batch B can be prepared from C0 before batch A starts. A then reads C0, B commits C1 and A's delayed Request poll selects different Requests that remain active. A executes those Requests from C0, but its prepared transaction names C1 as the predecessor. Canton can accept the transaction because C1 and A's Requests are active and all required parties have authorized it.

The Daml committer cannot detect this mismatch. It checks that the supplied predecessor is active and that the version does not decrease, then stores the supplied root without recalculating the VER state transition. State introduced by B can therefore disappear from the canonical VER root even though B's Requests and Daml settlement have completed. Depending on the affected Requests, this can roll back nonces, consumed-packet records, replay protection or configuration changes, allowing repeated cross-chain execution or inconsistent asset accounting.

The documented architecture expects one logical VER sequencer. It describes the stateless sequencer as the only party that submits transactions and a healthy `RequestProcessor` waits for one batch and its Canton submission before starting the next. Therefore, ordinary successful processing by one sequencer instance does not trigger this race. This issue concerns the LayerZero VER sequencer application, not the number of sequencer nodes used internally by the Canton network.

The single-sequencer design does not remove the race. `submitTransaction` first submits a signed transaction and then waits for its completion. If that wait times out, the processor treats the attempt as failed and starts another iteration after `errorPollDelay`, even

C-01 Head race commits a stale-derived root

CONTINUED

though Canton may still accept the first submission. The retry can read C0, the delayed first submission can then create C1 and the retry can select the remaining active Requests before `#buildCommitBatch` pairs their C0-derived root with C1. The same situation can occur after a network disconnect or process crash between submission and confirmation. A replacement process can retry while the transaction submitted by the previous process is still pending.

Other single-sequencer deployments can become multi-writer temporarily. A rolling or blue-green deployment can start the replacement before the old instance has stopped. Active/passive failover can split brain without a shared lease or fencing token. An operator or recovery tool can submit a commitment while the sequencer is preparing a batch. A leaked validator JWT can let another authenticated client issue a concurrent `WRITE`. Finally, one compromised or faulty sequencer process can deliberately issue two `WRITE` requests at once. Validator HTTP requests are not serialized or bound to a sequencer identity, lease or expected predecessor, so none of these cases is rejected before signing.

— RECOMMENDATION

Carry the exact `StateCommitment` selected by `ValidatorController.#write` through execution and transaction construction. `CantonChainClient.#buildCommitBatch` should use that commitment's CID as `prevStateCommitmentCid` and must not query the latest commitment again. Also verify before signing that the expected root and version are the values used during execution. If another batch archives the expected predecessor first, Canton will then reject the stale transaction.

The sequencer should reconcile a timed-out command ID and the current commitment before rebuilding or retrying a batch. Deployments should also enforce a shared single-writer lease with fencing across restarts, rollouts and failover. These controls reduce accidental concurrency, but they do not replace binding every prepared transaction to the exact predecessor used for execution. A process-local mutex protects only one running process and does not cover pending submissions, replacement instances or other authenticated writers.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/18>

— RESOLUTION

Status: Resolved.

CRITICAL **C-02**

RESOLVED

Forged callback bypasses gateway auth, mints OFT

CATEGORY Access Control

LOCATION contracts/protocol/canton/contracts/Layerzero/OftCommon/damL/OApp/LzSend/Callback/Fetch.damL:22-26; IPendingCallback.damL:46-48; IRequestCallback.damL:33-40; OftSelf.damL:391-395; LockUnlockAdapter.damL:203-207

REVIEW Remediation Review 2

DESCRIPTION

`consumeVerifiedPendingCallback` is documented as proving a pending callback was signed by the OApp gateway. It instead exercises any `IPendingCallback` and compares the returned view's `handler` and `oappId` with expected values:

```
v <- exercise pendingCid IPendingCallback_Consume
assertMsg _ERR_CALLBACK_UNAUTHORIZED_REQUEST $ v.handler == gateway
assertMsg _ERR_CALLBACK_APP_UID_MISMATCH $ v.oappId == oappId
```

Interface views are supplied by the implementing template and do not authenticate signatories. Rev1 `fetchAndVerifyPendingCallback` required `expectedSignatory elem signatory pending`. Rev2 removed that check when switching to an interface CID and OApp-driven `IRequestCallback_Finalize`.

An attacker can deploy a template signed only by the attacker whose interface view claims the victim gateway and victim OApp identity. `IPendingCallback_Consume` is controlled by `(view this).oappId.admin` from that same forgeable view. When the attacker calls permissionless `IRequestCallback_Finalize` on the victim OApp, the OApp contributes admin signatory authority, so `Consume` succeeds and both verifier comparisons pass.

Call tree (OFT mint):

1. Attacker creates `FakePendingCallback` (attacker signatory; forged `handler`, `oappId`, `isAccept=True`, `LzReceive_callback`).
2. Attacker exercises `IRequestCallback_Finalize` on victim `OftSelf`.
3. `requestCallback_FinalizeImpl` -> `consumeVerifiedPendingCallback`.
4. `Consume` returns forged view; equality checks pass.
5. `dispatchAccept` -> `handleLzReceiveAccept` -> `lzReceive` -> `create Oft` unlocked holding.

`LockUnlockAdapter` shares steps 1-4, then unlocks treasury assets to the forged receiver.

Validated impact:

C-02 Forged callback bypasses gateway auth, mints OFT

CONTINUED

- OFT: repeatable unbacked mint with no `Request`, genuine `PendingCallback`, source burn, or gateway accept (1000 then +5000; total 6000 unlocked).
- `LockUnlockAdapter`: treasury drain (2000 -> 1000; receiver got 1000 after accepting the pending transfer).
- Receiver fingerprint must resolve via Registry.

Preconditions: malicious `IPendingCallback` package must be vetted on the participant hosting the victim OApp (same model as `MaliciousRegistry`). Finalize needs `OftSelf` /adapter disclosures; public discovery finalize-request returns those. Finalize is permissionless by design; forging authenticity is what mints. Critical under the accepted foreign-package / interface-forgery threat model.

PoC PR: single-mint E2E and full-impact (repeatable mint + adapter treasury unlock) scripts.

— RECOMMENDATION

Fetch the underlying pending callback before consuming it and require the configured gateway to be an actual signatory:

```
pending <- fetch pendingCid
assertMsg _ERR_CALLBACK_UNAUTHORIZED_REQUEST $
gateway 'elem' signatory pending
```

Keep the full `oappId` equality check as defense in depth. Prefer accepting the concrete trusted `PendingCallback` template CID instead of an arbitrary interface CID when extensibility is unnecessary. Add a regression test with a foreign `IPendingCallback` implementation whose view reports the expected gateway and OApp but whose signatory set contains only the attacker.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/20>

— RESOLUTION

Status: Resolved.

HIGH **H-01**

RESOLVED

Stale admin nominees can take over the adapter

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/AccessControl/daml/AccessControl.daml:71-93
REVIEW	Remediation Review 2

DESCRIPTION

`AccessControl_CreateDefaultAdminTransfer` creates an independent `DefaultAdminTransfer` contract without consuming or updating `AccessControl`. Creating a newer nomination therefore leaves every older nomination active. During acceptance, `AccessControl_AcceptDefaultAdminTransfer` checks only the supplied transfer's assignee and static scope. It does not prove that the transfer is the latest nomination for the current `AccessControl` generation.

A superseded nominee can retain the first prepared acceptance transaction, wait for the administrator to nominate a replacement, then accept the old transfer. This differs from the repository's EVM

`AccessControl2StepUpgradeable`, where a new nomination overwrites the pending administrator and a stale nominee is explicitly tested to revert.

The Daml Script

`contracts/protocol/canton/contracts/tests/daml/ZZPOCs/Stale0AppDefaultAdminTransfer.daml` reproduces the issue against a fully configured LockUnlockAdapter. The adapter administrator first nominated the attacker and then nominated a replacement. The attacker accepted the stale transfer without the adapter administrator in `actAs`, became the sole `_DEFAULT_ADMIN_ROLE` holder, then successfully exercised `0App_CreateRequest` to create an arbitrary adapter request. The normal RequestFactory gateway was the only additional acting party.

This is a privilege-boundary break rather than trusted default-admin abuse. The attacker is a superseded nominee who was never granted the role. Once the stale transfer succeeds, the attacker receives the custody-equivalent generic request authority that the current design reserves for trusted default admins. They can submit attacker-chosen endpoint calls and callback contexts, rotate roles or disrupt adapter settlement before the immutable administrator recovers control.

RECOMMENDATION

Bind each pending transfer to a monotonic nomination epoch stored in `AccessControl`. Creating a nomination should consume and recreate `AccessControl` with an incremented epoch

H-01 Stale admin nominees can take over the adapter

CONTINUED

and the new `DefaultAdminTransfer` should record that epoch. Acceptance must require an exact epoch match before replacing `_DEFAULT_ADMIN_ROLE`. Consequently, a newer nomination invalidates every older transfer without relying on an unstable contract ID.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/9>

— RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

Default-admin assignee can seize sibling OApp

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/AccessControl/daml/DefaultAdminTransfer.daml:41–51; contracts/protocol/canton/contracts/Layerzero/AccessControl/daml/SetDelegateRequestArgs.daml:68–86
REVIEW	Remediation Review 2

— DESCRIPTION

Accepting an OApp-scoped default-admin transfer also creates an EndpointV2 `setDelegate` request. The pending transfer correctly binds the AccessControl update to its original `scope`, but it does not bind the separately supplied `ioAppCid`, OApp config or request-factory config in `extraContext` to `scope.id`.

`DefaultAdminTransfer_Accept` lets the assignee provide the entire `extraContext` map. `createSetDelegateRequest` fetches that caller-selected `ioAppCid`, derives the endpoint call's OApp ID from it and exercises `OApp_CreateRequest` with `caller = scope.admin`. Consequently, when two OApps share the same administrator, an assignee nominated only for OApp A can submit OApp B's contracts while accepting A's transfer. OApp B accepts the call because the authenticated caller is its administrator and the generated request sets the assignee as B's endpoint delegate.

The attacker therefore crosses an explicit OApp authorization boundary: nomination for one OApp grants control over the endpoint configuration of any sibling OApp with the same administrator. An endpoint delegate can alter the victim's messaging configuration and make attacker-controlled packet verification satisfy the endpoint, enabling forged inbound settlement and unbacked minting or treasury unlocks. The attack does not require the shared administrator to nominate the attacker for the victim OApp, authorize the victim request or act maliciously.

A Daml Script used as a POC reproduced the issue with two OApps under one administrator. Exercising `DefaultAdminTransfer_Accept` only as the assignee created a request whose `oappId` belonged to OApp B, whose caller was the shared administrator and whose `EndpointV2.setDelegate` target was the assignee.

— RECOMMENDATION

Bind every input used to create the `setDelegate` request to the pending transfer's scope before exercising `OApp_CreateRequest`. At minimum, fetch `ioAppCid` and require its

H-02 Default-admin assignee can seize sibling OApp

CONTINUED

`oappId` to equal `scope.id`, then authenticate the supplied OApp and request-factory configurations against that same OApp and administrator.

Prefer moving the typed OApp inputs into an OApp-specific acceptance path so the compiler and interface types enforce the relationship instead of decoding an unbound `Map Text AnyValue`.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/10>

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

Payout floor withholds charged fees

CATEGORY Warning

LOCATION contracts/protocol/canton/contracts/Layerzero/Request/daml/Request.daml:109-152

REVIEW Remediation Review 2

DESCRIPTION

`createPendingPayments` subtracts every declared payout when it calculates `remainderTarget`, but it later creates `PendingPayment` contracts only for amounts at or above `minPayoutValue`. The final change holdings from that split are discarded from the Request's state. Therefore every filtered payout remains spendable by the handler even though no contract records the intended recipient or amount.

The fee was already charged before this filtering occurs. `Context.pay` reduces the request's native value and records an equal obligation for each executor, DVN or other payee. The Canton client converts those obligations to exact ten-decimal Amulet values and `commitBatchImpl` forwards them to `IRequest_Accept`. Both accept and reject archive the Request after `createPendingPayments` returns, so a filtered obligation cannot be recovered through the Request or settled through `PendingPayment`.

For example, a valid configuration can have `minFee = 1.0` and `minPayoutValue = 0.5`. If a request escrows `2.2`, declares two service fees of `0.4` and has a sender remainder of `0.4`, the treasury receives `1.0`. All three deferred amounts are filtered. The transaction still succeeds and archives the Request, while the handler keeps the remaining `1.2`.

The loss is not bounded to one dust amount. ULN permits up to 255 DVNs and pays every configured DVN in addition to the executor. Each distinct fee and the sender remainder are filtered separately, so their combined value can be materially larger than `minPayoutValue` and can accumulate across requests. An ordinary sender needs no administrative role to encounter this behavior once a nonzero floor is configured.

RECOMMENDATION

Do not silently remove a charged payout from settlement. Pass the snapshotted `minPayoutValue` to the runtime and reject a send before acceptance if any positive payout is too small to transfer. Apply the same check during quote construction so users cannot fund requests that will produce unpayable service fees.

M-01 Payout floor withholds charged fees

CONTINUED

If sub-floor fees must be supported, record them as recipient-bound credits and combine them across requests until the recipient's balance is transferable. Keep an on-ledger assertion in `createPendingPayments` so an accepted Request cannot archive unless every charged payout is either paid or represented by an active obligation contract.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/15>

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Archived requests block recovery

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/services/state-recoverer.ts:250-264
REVIEW	Remediation Review 2

DESCRIPTION

The original H-13 trigger and zero-update finalization defects were fixed, but its recovery remediation remains incomplete. `StateRecoverer` reconstructs each missing commitment by passing its committed Request IDs to the ordinary `RequestExecutor`. That executor fetches every Request through `CantonChainClient.getRequest`, `RequestSdk.getRequest` and `CantonSdk.get`. The final lookup is explicitly active-only and returns no contract after archival.

Every successful `CommitBatch` archives its Requests in the same Canton transaction that creates the successor `StateCommitment`. A validator can therefore crash after Canton accepts the batch but before the asynchronous finalizer commits the corresponding commitment, Request and event rows to the local database. On restart, recovery finds the missing on-ledger commitment but cannot load its archived Requests. `RequestExecutor` converts the failed lookups into `RequestFetchError`, so the validator cannot reconstruct that commitment or advance to later state.

One affected validator can be repaired from Canton history, but simultaneous failures that leave fewer than the required validators with usable local state can stop write quorum and prolong pending requests. Exploitation requires a crash, restart or equivalent database-write loss in the post-ledger, pre-finalization window. Confidence is high because the production fetch path documents and implements active-only behavior. The existing recovery tests use `StubChainClient`, which returns historical Request payloads and therefore does not exercise this failure.

RECOMMENDATION

Add a history-aware Request lookup for recovery using Canton contract events or transaction history, authenticate and decode the original created event and replay that immutable payload instead of calling the active-only request API. Keep the normal live-processing fetch active-only. Add a regression test that commits and archives a Request, omits the validator's local finalization write, restarts recovery and proves that the missing commitment, Request records and events are reconstructed before a child commitment is processed.

M-02 Archived requests block recovery

CONTINUED

— RESOLUTION

Status: Resolved.

MEDIUM **M-03**

RESOLVED

Foreign config bricks context discovery

CATEGORY	DoS
LOCATION	contracts/protocol/canton/sdk/src/oapp/oft-registry-sdk.ts:260-280
REVIEW	Remediation Review 2

DESCRIPTION

`getRequestFactory` rejects every lookup when more than one `RequestFactoryConfig` is visible. It performs this cardinality check before filtering configs by the genuine factory's `handler`.

An ordinary Canton party can create the vetted `RequestFactoryConfig` template with itself as `handler` and the discovery service's `informee` as an observer. The template requires only `handler` as signatory, so the service party does not authorize or consent to becoming an observer (`RequestFactoryConfig.daml:82-102`). The foreign contract is therefore returned alongside the genuine config when the SDK queries contracts visible to `informee`. `configs.length > 1` then makes every lookup fail even though exactly one config matches the real factory.

The public discovery service uses `getRequestFactory` for request creation, `LzSend`, `LzReceive` and direct factory-context requests. Consequently, one foreign contract makes all of those endpoints return HTTP 500. The attacker can keep the contract active indefinitely because it is signed by the attacker, leaving the operator unable to archive it through normal Daml authorization. Exploitation needs no OApp, default-admin, endpoint-delegate, handler or package-upload privilege. It only requires knowledge of the service's `informee` Party and ordinary ledger submission access.

The conflicting config is created by an unrelated party and does not match the deployed factory's authenticated handler. Its only effect should be exclusion from discovery, but the SDK lets it disable the service.

RECOMMENDATION

Resolve the single visible `RequestFactory` first. Then filter `RequestFactoryConfig` contracts by `config.payload.handler == factory.payload.handler` before checking cardinality. Require exactly one matching config and ignore foreign configs. Keep the final handler equality check as defense in depth.

M-03 Foreign config bricks context discoveryCONTINUED

— RESOLUTION

Status: Resolved.

MEDIUM **M-04**

ACKNOWLEDGED

Sybil OApps can fill discovery registry

CATEGORY	DoS
LOCATION	contracts/protocol/canton/contracts/Layerzero/DiscoveryRegistry/daml/DiscoveryRegistry.daml:109-143
REVIEW	Remediation Review 2

DESCRIPTION

`SetUrl` admits every `IOApp` whose declared gateway equals the registry handler. It does not require the handler to approve that OApp. Every new app UID consumes one slot in the shared `urlMap` and the contract rejects all new registrations once `maxEntries` is reached.

An OApp administrator who is included in the registry's observers or otherwise receives the current contract through the intended registration mechanism, can exploit this. An actor that controls admin and treasury parties can create arbitrarily many valid `LockUnlockAdapter` contracts with unique OApp IDs and set `oappGateway` to the real handler. The handler is only an observer of those contracts, so its authorization is not required. The attacker can then call `SetUrl` for each adapter and fill the registry. Successful calls recreate the registry with the same observers, so an admitted observer retains visibility as the registry CID changes. This is the permissionless model described by the contract itself: OApp administrators are allowed to register and the fee is intended to prevent spam. The repository's `testRegistryFull` uses the same authority boundary and visibility precondition: an attacker observer owns an IOApp that names the real handler and is eligible to register until the global cap stops it.

The current registry has no entry-removal choice and no way to increase `maxEntries`. Updating a URL also keeps its app UID in the map, including when the URL is empty. Consequently, saturation permanently blocks discovery registration for every later legitimate OApp on that registry. A fee only raises the attack cost; the template permits `minFee` to be zero and even a nonzero fee does not impose an identity-based limit. The handler can deploy a replacement registry, but that changes the registry CID and requires operational migration of existing registrations.

The OApps and registry are the intended templates and the attacker passes the registry's stated admission checks. The resulting denial of registration is therefore caused by an ordinary actor exhausting shared state, not by violating the accepted package-vetting or singleton assumptions.

M-04 Sybil OApps can fill discovery registry

CONTINUED

— RECOMMENDATION

Require handler approval for the first registration of each app UID or authenticate the OApp against a handler-managed deployment allowlist. If registration must remain permissionless, replace the global immutable capacity with per-OApp registration contracts and index them off ledger. At minimum, add handler-controlled removal, an adjustable cap and a Sybil-resistant per-identity quota.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/16>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

RESOLVED

Zero-amount rejects strand adapter payouts

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/LzSend/Callback/Parser.daml:8-14
REVIEW	Remediation Review 2

— DESCRIPTION

`prepareSend` can produce a zero cross-chain amount from a positive send. It only requires payouts to be smaller than `amountLD`, then `removeDust` rounds the remaining base units down to zero. A caller-supplied `minAmountLD` of zero accepts that result. Existing tests also confirm that zero-value sends are intentionally valid and create a `Request`.

`adapterLzSendImpl` still transfers the contingent payouts into treasury custody and serializes the zero cross-chain amount into the callback. This can lock a material amount. For example, an external payout may consume all but one base unit of a large `amountLD`; that final unit becomes dust while almost the full amount is held for the payout.

If the gateway rejects the `Request`, `handleAdapterLzSendReject` immediately calls `parseAdapterLzSendCrossChainAmount`. The parser requires the amount to be strictly positive, so finalization aborts before adding the payouts to `refundBaseUnits` or creating the sender's refund instruction. The rejected `Request` has already been archived and its immutable `PendingCallback` contains the same zero amount. Consequently, every retry fails with the same error. The locked payouts remain in the treasury and the sender has no normal on-ledger recovery.

Recovery requires treasury intervention rather than the refund guaranteed by the reject handler. The specific conditions are `minAmountLD = 0`, a post-payout remainder below the decimal conversion rate, at least one positive payout and rejection of the resulting request.

— RECOMMENDATION

Allow `parseAdapterLzSendCrossChainAmount` to return zero. The reject handler can then refund `sum payouts` when the cross-chain amount is zero and finalize a fully zero callback without creating a transfer.

Alternatively, reject every positive send whose dust-adjusted cross-chain amount is zero before assets enter the treasury and before the `Request` is created.

M-05 Zero-amount rejects strand adapter payouts

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/17>

— RESOLUTION

Status: Resolved.

MEDIUM **M-06**

RESOLVED

Limiter disable strands pending transfers

CATEGORY Unexpected Behavior

LOCATION contracts/protocol/canton/sdk/src/executor/executor-sdk.ts:396-414

REVIEW Remediation Review 2

DESCRIPTION

`ExecutorSdk` includes `RateLimiterState` in a callback finalization only when the current config says rate limiting is enabled. `OftRegistrySdk` uses the same condition at lines 810-827. This condition is wrong for an outbound transfer that recorded rate-limit usage before the manager disabled the limiter.

An outbound callback freezes the positive `scaledAmount`, `resolvedEid` and `recordedAt` produced by `RecordOutflow`. The OFT accept and reject handlers therefore require `RateLimiterState` at `0App/LzSend/Callback/Accept.daml:42-49` and `0App/LzSend/Callback/Reject.daml:37-47`. The adapter handlers impose the same requirement at

`LockUnlockAdapter/daml/LzSend/Callback/Accept.daml:72-84` and

`LockUnlockAdapter/daml/LzSend/Callback/Reject.daml:64-75`. This remains required after a global disable because `CommitOutflow` and `ReverseOutflow` reconcile usage that was already recorded.

Consequently, disabling the limiter while positive outbound transfers are pending makes both official builders omit a required contract ID and disclosure. Every accept or reject finalization built this way aborts. On rejection, atomic rollback also undoes the attempted refund or holding unlock. On acceptance, it undoes the burn and payout distribution. All affected transfers remain pending until an operator re-enables rate limiting or constructs a custom execute context. This can strand an unbounded amount of in-flight user funds during the emergency in which the kill switch is most likely to be used.

RECOMMENDATION

Derive the state requirement from the frozen callback data. For every outbound `LzSend` callback with a positive recorded `scaledAmount`, include the current `RateLimiterState` contract ID and disclosure even when `isGloballyDisabled` is true. Keep the current-config check only for callbacks that record new inbound usage. Apply the same predicate in `ExecutorSdk`, `OftRegistrySdk` and any adapter-specific finalization builder.

M-06 Limiter disable strands pending transfersCONTINUED

— RESOLUTION

Status: Resolved.

MEDIUM **M-07**

RESOLVED

Direct accept bypasses admin transfer lifecycle

CATEGORY	Logical Error
LOCATION	contracts/protocol/canton/contracts/Layerzero/IAccessControl/daml/IAccessControl.daml:100-117; AccessControl.daml:76-93; DefaultAdminTransfer.daml:41-54
REVIEW	Remediation Review 2

DESCRIPTION

`AccessControl_AcceptDefaultAdminTransfer` is documented as an implementation choice for `DefaultAdminTransfer_Accept`, but it is a public interface choice (controllers: current `roleAdmin` + nominated `assignee`).

The direct choice validates the pending transfer and replaces `_DEFAULT_ADMIN_ROLE`, but it does not archive the `DefaultAdminTransfer` and does not run the OApp `setDelegate` branch. Those steps exist only in `DefaultAdminTransfer_Accept`:

1. Exercise `AccessControl_AcceptDefaultAdminTransfer` (role flip)
2. For `_CATEGORY_OAPP`, create the EndpointV2 `setDelegate` request
3. Archive the transfer

In Daml, interface choices cannot be internal-only, so any qualifying `roleAdmin` + `assignee` can submit the incomplete path. Existing tests only cover unauthorized / assignee-mismatch direct accepts; they do not prove an authorized direct accept is impossible or that it consumes the transfer.

Impact (not outsider privilege escalation; requires `roleAdmin` + nominee):

- OApp: Canton `_DEFAULT_ADMIN_ROLE` moves while EndpointV2 can keep the prior runtime delegate (split authority).
- Transfer remains active after "acceptance".
- Later reclaim: leftover transfer `T1` can still be accepted via `DefaultAdminTransfer_Accept` against a newer `AccessControl`, including after another administrator `B` was installed. Accept is controlled only by `assignee`; `scope.admin` authorizes as transfer signatory, so the original nominee can unilaterally reclaim `_DEFAULT_ADMIN_ROLE` from `B`.

Distinct from the submitted stale-nomination issue (multiple creates leaving older nominees live). That bug is about superseding creates; this is an incomplete accept that never consumes the transfer it applied. Fixing `setDelegate` encoding or nomination epochs does not close this path.

M-07 Direct accept bypasses admin transfer lifecycle

CONTINUED

PoC (PR): `roleAdmin` + `assignee` jointly exercise `AccessControl_AcceptDefaultAdminTransfer`; `_DEFAULT_ADMIN_ROLE` is granted and `DefaultAdminTransfer` remains active. OApp path likewise grants the role with no `setDelegate` side effect.

RECOMMENDATION

Make the complete transfer lifecycle unavoidable on every acceptance path.

Preferred: move transfer consumption and OApp `setDelegate` / `extraContext` handling into `accessControl_AcceptDefaultAdminTransferImpl`, and keep `DefaultAdminTransfer_Accept` as a thin wrapper that only exercises that choice. Alternatively, fold role replacement into `DefaultAdminTransfer_Accept` only and remove `AccessControl_AcceptDefaultAdminTransfer` from the public `IAccessControl` interface. Documentation alone cannot hide an interface choice.

Minimum bar for every successful accept:

1. Consume or epoch-invalidate the exact pending transfer before returning (if the impl archives, drop the redundant archive from the wrapper).
2. For OApp scopes, require authenticated `setDelegate` inputs and create the delegate-rotation request on the same path that flips the Canton role.
3. Add a regression that joint-exercises `AccessControl_AcceptDefaultAdminTransfer` and proves the role cannot flip without consuming the transfer and completing required OApp side effects; also assert a leftover transfer cannot reclaim after a later proper accept.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/19>

RESOLUTION

Status: Resolved.

MEDIUM **M-08**

ACKNOWLEDGED

Factory rotation hides pending requests

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/chain/request-poller/canton.ts:70-116
REVIEW	Remediation Review 2

DESCRIPTION

When a validator has no persisted request offset, `CantonRequestPoller.poll` queries the single active RequestFactory and seeds its cursor from that factory's creation offset. This assumes no relevant Request can predate the currently active factory.

Factory and Request activeness are independent. `RequestFactory.Request_Create` creates a separate handler-signed Request and archiving or replacing the factory does not consume Requests it already created. After an ordinary factory rotation, a Request from the previous generation can therefore remain active while the replacement factory has a later creation offset. A fresh, rebuilt or newly added validator seeds after that Request's `Request_Create` exercise and never scans it. Fast-forward persistence makes the omission durable.

Existing validators with preserved offsets may select the old Request while fresh validators omit it, preventing the expected validator set from agreeing on batches. If enough validators are rebuilt after rotation, the Request can remain active indefinitely and its user fee, custody action or callback settlement stays pending. The condition requires a supported factory rotation while at least one prior-factory Request remains active, followed by a fresh, rebuilt or newly added validator with no preserved offset.

RECOMMENDATION

Seed new validators from a stable deployment or recovery checkpoint that predates every unresolved Request, not from the current factory's creation. Track factory generations explicitly or query active trusted Requests and recover the earliest relevant creation/exercise offset before advancing the cursor. Preserve the handler and interface-based identity checks rather than pinning one factory CID.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

RESOLVED

Losing batch pollutes canonical scan data

CATEGORY Unexpected Behavior

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/services/request-executor.ts:232-275

REVIEW Remediation Review 2

DESCRIPTION

`RequestExecutor.execute` gives its finalizer only the first request ID from the locally executed batch and the resulting state root. `#finalize` uses that request ID to find an on-ledger commitment, then checks only that the commitment has the same state root. It does not verify the commitment's complete ordered request list, predecessor, version, transaction hash or write entries. After the root check passes, it stores every locally executed request and event under the returned commitment.

One normally operating sequencer processes successful submissions one at a time, so the ordinary successful case does not create competing batches. This does not remove the issue. Validators start `finalize()` asynchronously before returning their signed transactions. If a quorum-signed submission fails before it commits, the sequencer retries after the error delay while the previous finalizers are still polling. With the default configuration, retries begin after five seconds while finalizers remain alive for thirty seconds. A malicious sequencer holding an allowlisted validator credential can also initiate competing write requests directly. The validator API has no global single-flight guard or sequencer epoch that would reject the second request.

Overlapping finalizers are necessary but are not sufficient. The losing and winning candidates must start from the same predecessor, share their first request ID, contain different later requests and produce the same final root. Validators must therefore select different request sets, for example because their Canton participant views diverge. A malicious sequencer can exploit that divergence by sending competing requests to selected validator subsets, but a healthy retry against one consistent request view does not trigger the bug. For example, assume a quorum first signs losing candidate `[R1, R3]`, but that submission does not commit. A later attempt commits `[R1, R2]`, where `R2` and `R3` produce equivalent transitions. The old finalizers find the winning commitment through `R1`. Their root check succeeds, so they store `R3` and its events under a commitment that contains only `R1` and `R2`.

If enough validators signed the losing candidate, their repositories can agree on the same phantom rows and return a quorum-signed scan response for `R3`. Canton never committed

M-09 Losing batch pollutes canonical scan data

CONTINUED

R3 in that commitment. This can mislead indexers, accounting systems, relayers and monitoring without compromising a validator signing key or database. The canonical trie root remains correct, which limits the impact to repository and signed-scan integrity.

RECOMMENDATION

Bind finalization to the exact canonical batch. Compare the complete ordered request IDs, predecessor ID, version, state root and a digest of the normalized write entries before opening the database transaction. Persist the commitment's request IDs and version so repository rows can be checked against them. Recovery should pass the exact commitment being replayed into the finalizer instead of resolving another commitment through one request ID.

RESOLUTION

Status: Resolved.

MEDIUM **M-10**

ACKNOWLEDGED

Pending offers erase adapter obligations

CATEGORY Unexpected Behavior

LOCATION contracts/protocol/canton/contracts/Layerzero/FeeUtils/daml/Transfer/Core.daml:176-181

REVIEW Remediation Review 2

DESCRIPTION

`proposePendingTransfer` treats `TransferInstructionResult_Pending` like a completed transfer and discards the returned instruction CID. A pending result only means that the factory created an offer. The recipient has not received the tokens yet.

`PendingCallback` is consumed

|

v

`TransferFactory` returns `Pending`

|

v

Pending instruction CID is discarded

|

v

Adapter considers settlement complete

This sequence removes the adapter's obligation too early. Consuming the `PendingCallback` deletes the contract that records an inbound payout or rejected-send refund. The `Pending` result confirms only that an offer exists. Discarding its CID means the adapter cannot track that offer or respond if it fails. The transaction then succeeds, so the original callback cannot be retried.

The first-party OFT factory demonstrates the failure. It creates an `0ftTransferOffer` that the recipient must accept within 24 hours. Acceptance fails after that deadline. The treasury can also withdraw the offer before acceptance, which returns the locked holdings to treasury.

If the offer expires or is withdrawn, the recipient cannot complete the transfer. The adapter has already deleted the callback and cannot retry. An inbound recipient can therefore lose the destination payout, while a rejected sender can lose the refund. Recovery requires a manual treasury transfer.

M-10 Pending offers erase adapter obligations

CONTINUED

Requiring the recipient to accept an offer is intended. The issue is that the adapter removes its own obligation before that acceptance succeeds.

— RECOMMENDATION

When the factory returns `TransferInstructionResult_Pending`, create an adapter-owned settlement contract that stores the instruction CID, recipient, amount, instrument and callback identity. Mark the settlement complete only after the instruction returns `Completed`. If the offer expires, is rejected or is withdrawn, keep enough state to issue a replacement transfer.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

One validator blocks sequencer startup

CATEGORY Unexpected Behavior
LOCATION apps/ver/ver-sequencer/src/server.ts:52-64
REVIEW Remediation Review 2

DESCRIPTION

`initializeServer` fetches every configured validator's public key inside one `Promise.all` before it constructs the quorum checker, initializes genesis, creates the HTTP application or exposes the health route. A timeout, connection failure or malformed response from any one validator therefore rejects the whole initialization even when the remaining validators are sufficient to meet `quorumThreshold`.

This contradicts the service's q-of-N runtime fault model. Once started, `ResponseQuorumChecker` deliberately tolerates non-contributing validators and resolves when the configured threshold agrees. During startup, however, the same below-threshold validator is mandatory. A failed or malicious non-quorum validator can keep a restarted, replaced or newly deployed sequencer offline by failing each public-key request. Pending adapter Requests then cannot reach write quorum or settlement until that validator recovers or operators remove it and restart again.

RECOMMENDATION

Pin validator public keys in authenticated configuration so startup does not depend on live discovery. If discovery remains supported, use settled results, require at least the configured number of unique valid keys, start with that quorum and retry unavailable members in the background without changing committee identity. Expose degraded membership through health and metrics.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/21>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

RESOLVED

Internal faults become final request rejects

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/container/component.ts:220-231; packages/protocol/lz-ver-protocol/ver-api-common/src/services/request-executor.ts:194-208
REVIEW	Remediation Review 2

DESCRIPTION

The validator does not preserve the boundary between a protocol-level user error and an internal execution failure. `ContainerComponent.#callContract` catches every JavaScript `Error` thrown by a contract method and rewraps it as `UserError`. This includes explicit validation errors, but also trie/database failures raised inside state accessors, codec/runtime faults, failed internal invariants and programming defects. `Container.call` returns the resulting `UserError` in-band. If an internal error escapes that wrapper, `RequestExecutor.#executeRequest` catches it anyway and unconditionally builds a normal `action: "reject"` entry.

The validator then signs a Canton transaction containing that reject entry and returns HTTP 200. If the same dependency failure or software defect affects enough validators, the sequencer obtains an ordinary write quorum and `CommitBatch` exercises `IRequest_Reject`. That archives the Request, transfers the nonrefundable `minFee` to treasury and creates a reject callback.

This is economically material because an OFT send locks principal and contingent payouts before creating its Request, while a LockUnlockAdapter send transfers the source asset into treasury custody first. A callback may later recover principal, but it cannot restore the requested cross-chain operation or refund the minimum fee. A retryable validator outage is therefore converted into an irreversible protocol decision after custody has committed.

One isolated validator failure cannot cause settlement because it will not join the honest response quorum. Exploitation or failure requires a quorum-correlated backend fault, a deterministic release defect or another shared failure mode, which supports Medium severity. It does not require a malicious DAR, OApp administrator, endpoint delegate, treasury party or package-vetting failure.

RECOMMENDATION

M-12 Internal faults become final request rejects

CONTINUED

Introduce an explicit protocol-error type and convert only that type into an in-band `UserError` or reject entry. Let storage, trie, codec, invariant and unexpected runtime errors abort the validator response so the validator drops out of quorum and the Request remains retryable.

Apply the same allowlist in both `ContainerComponent.#callContract` and `RequestExecutor.#executeRequest`; the outer executor catch must not turn unknown exceptions into rejects.

— RESOLUTION

Status: Resolved.

MEDIUM **M-13**

RESOLVED

Executor SDK cannot settle adapter callbacks

CATEGORY	Logical Error
LOCATION	contracts/protocol/canton/sdk/src/executor/executor-sdk.ts:282-315
REVIEW	Remediation Review 2

DESCRIPTION

`ExecutorSdk.prepareFinalizeCallback` treats every OApp configuration as an `OftSelfConfig`. It unconditionally reads `config.payload.instrumentId` and `config.payload.transferRuleCid`, then requires an `OftTransferRule` disclosure. `AdapterConfig` instead stores `oappId` and `assetInstrumentId`; it has no `instrumentId` or `transferRuleCid`. Supplying a real adapter config therefore throws while deconstructing `instrumentId`, before the SDK can build an `Executor.FinalizeCallback` command.

The remaining builder logic is also OFT-specific. `buildFinalizeExecuteContext` can include only `configCid`, `registryCid` and `rateLimiterStateCid`. Every value-bearing adapter callback additionally reads `transferFactoryCid`, `treasuryAssetCids` and `assetTransferContext` from the execute context. These fields and their disclosures cannot be supplied through `FinalizeCallbackParams`, so changing only the config cast would still produce a transaction that aborts on ledger.

This mismatch is reachable during ordinary operation. `CantonIOAppResolver` deliberately resolves non-OFT applications through `IOApp` and `IOAppConfig` interface views and its existing test uses an `AdapterConfig`. `ExecutorSdk.prepareDelegateLzReceive` can then create a genuine adapter receive request. After the gateway accepts it, the destination payment exists only as a `PendingCallback`, but the shipped executor builder cannot prepare its treasury unlock. The same defect prevents an outbound rejection from returning assets that `adapterLzSendImpl` already moved into treasury custody. Users can remain unpaid or unrefunded until an operator constructs the adapter-specific transaction manually.

RECOMMENDATION

Make callback preparation implementation-aware. Resolve the callback and config through their interfaces, then dispatch to an OFT builder or an adapter builder. For an adapter, derive the app UID from `AdapterConfig.oappId`, omit the OFT transfer-rule lookup and build the adapter-specific execute context with the current authenticated transfer factory, treasury-owned holding CIDs, asset transfer context, registry when needed and rate-limiter state when required. Include every fetched contract as a disclosure.

M-13 Executor SDK cannot settle adapter callbacksCONTINUED

— RESOLUTION

Status: Resolved.

MEDIUM **M-14**

ACKNOWLEDGED

DVN approvals replay across deployments

CATEGORY Unexpected Behavior

LOCATION contracts/protocol/ver-endpoint/src/dvn/multisig/hashes.ts:83-111

REVIEW Remediation Review 2

DESCRIPTION

`hashQuorumChangeAdmin` and `hashQuorumReplaceAdmins` bind the function name, DVN address, requested administrator, VID and expiration. They do not bind an immutable network or deployment identifier. A signature is therefore valid in every deployment that reconstructs the same field values, even though each deployment has an independent DVN state and administrator policy.

Identical DVN addresses are realistic in this implementation. The ULN address is derived from the fixed string `u1n302` (`packages/protocol/lz-ver-protocol/ver-protocol-common/src/common/address.ts:68-81`). `#deriveWorkerAddress` then uses only that ULN address, the worker type and a local nonce (`contracts/protocol/ver-endpoint/src/message-library/u1n/u1n-302/u1n-302.ts:1608-1622`). Repeating the provisioning order in two environments consequently produces the same DVN address. The runtime explicitly supports separate sandbox, testnet and mainnet environments, but none of that environment state enters the digest.

Replay protection does not separate these authorizations. Each `Dvn` stores `#usedHashes` in its own trie (`contracts/protocol/ver-endpoint/src/dvn/dvn.ts:86-110,607-618`). Consuming a digest in one deployment does not mark it as used in another. In addition, `quorumChangeAdmin` and `quorumReplaceAdmins` intentionally require only committee signatures. They do not require the caller to be a current administrator (`contracts/protocol/ver-endpoint/src/dvn/dvn.ts:363-425`).

For example, assume testnet and mainnet both provision a DVN at address `D`. Both instances use VID `7` and signers `S1`, `S2` and `S3` with a quorum of two. Testnet trusts administrator `T`, while mainnet independently trusts administrator `M`.

The testnet committee decides to replace `T` with `X`. Signers `S1` and `S2` approve `quorumReplaceAdmins(D, X, 7, E)`, where `E` is the expiration. Both deployments compute the following digest:

```
keccak256("QuorumReplaceAdmins" || D || X || uint32(7) || uint64(E))
```

`X` receives the signature bundle as the intended testnet administrator or relayer. Before `E`, `X` submits the same arguments and signatures to mainnet. The mainnet DVN recognizes

M-14 DVN approvals replay across deployments

CONTINUED

the same signer keys and VID. Its local `#usedHashes` set has not consumed the digest, even if testnet already executed it. Consequently, `quorumReplaceAdmins` removes `M` and every other mainnet administrator, then installs `X` without any mainnet-specific approval.

After the replay, `X` can call `proposePayee` with an account they control and accept it, redirecting future mainnet DVN fees. `X` can also alter the price feed, destination fee configuration and supported option types to overcharge users or make sends fail. This permits fee theft and disruption without approval from the affected deployment's committee.

— RECOMMENDATION

Give every runtime deployment an immutable, globally unique domain identifier and include it in every DVN-signed digest, including all governance operations and `verify`. The domain should remain stable across supported hard forks but differ across mainnet, testnet, sandbox instances and independent deployments. Do not rely only on the DVN address because this implementation deliberately derives the same addresses from the same provisioning sequence.

Rotate or reissue outstanding DVN approvals after introducing the domain.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-15**

RESOLVED

OneSig cannot run default-admin transfers

CATEGORY	Logical Error
LOCATION	IAccessControl.daml:54-98; OneSigTypes.daml:207-218; OneSig/IAccessControl/Call.daml:16-43
REVIEW	Remediation Review 2

DESCRIPTION

Rev2 makes the two-step default-admin transfer the only valid way to assign or replace `_DEFAULT_ADMIN_ROLE`. `AccessControl_GrantRoles` now rejects this role, and `AccessControl_RevokeRoles` rejects its removal.

OneSig was not extended for that state machine. Its closed `OneSigCall` variant and dispatcher still expose only `OpGrantRoles` and `OpRevokeRoles`. There is no signed operation for creating, accepting, or cancelling a default-admin transfer.

A valid quorum-signed default-admin grant or revoke therefore passes OneSig verification and deterministically aborts when dispatched to `IAccessControl` with `DefaultAdminRequiresTransfer`. Daml atomicity rolls back the OneSig nonce, so retries always fail the same way.

Direct exercise by the underlying Canton `admin` Party remains possible, so this is not a universal deadlock. Deployments that use OneSig as their documented M-of-N governance boundary must either halt default-admin rotation or bypass the signer policy through direct Party authority. Unless that Party is independently controlled by an equivalent threshold, this weakens authorization for a core recovery operation — operators cannot rotate or remove a compromised delegated default admin while preserving the OneSig approval path.

Repository tests confirm both sides: normal delegated-role grants via OneSig succeed, while `testGrantRolesRejectsDefaultAdmin` confirms the legacy path now rejects. No OneSig operation covers the mandatory transfer lifecycle.

RECOMMENDATION

Extend OneSig with typed signed operations for the complete default-admin transfer lifecycle. At minimum, add nomination and acceptance operations with target scope and assignee in the signed payload. Bind every executor-supplied contract ID to that signed identity before exercise.

Update the Daml call variant, encoder, execution context, dispatcher, TypeScript model, documentation, and tests together. Add regressions proving quorum-signed rotation succeeds

M-15 OneSig cannot run default-admin transfers

CONTINUED

for OApp and non-OApp scopes, while legacy default-admin grant/revoke reject without consuming the OneSig nonce.

— RESOLUTION

Status: Resolved.

MEDIUM **M-16**

RESOLVED

Poison AccessControl bricks OApp administration

CATEGORY	DoS
LOCATION	contracts/protocol/canton/sdk/src/shared/access-control.ts:25-59; resolve-ioapp-config.ts:37-47; oft-self-sdk.ts:134-136
REVIEW	Remediation Review 2

— DESCRIPTION

The new `findAccessControlCid` helper queries every `AccessControl` visible to an informee, then selects by `category` and `id`. Its payload type includes `admin`, but the filter never compares it and never verifies that the expected admin is a contract signatory.

The `AccessControl` template lets its signatory choose `category`, `id`, and observers. An ordinary Canton party can create a valid contract with itself as `admin`, copy a victim OApp's public category and instrument id, and add the LayerZero informee as an observer.

When the genuine `AccessControl` exists, the helper sees two matches and throws before any choice reaches the ledger. When the OApp uses solo-admin mode with no genuine `AccessControl`, the helper selects the attacker's CID and on-ledger scope checks reject it. Both cases deny service rather than escalate privilege.

This affects `resolveIoAppConfig` and `OftSelfSdk` peer / send / receive configuration paths. The victim cannot archive an attacker-signed contract, and the poison contract has no expiry.

Same authenticate-before-count defect class as Guardian `6a68a88234dea27378ae3bbd`, but a new helper and distinct execution surface. Fixing only the registry discovery lookup does not protect these OApp administration paths.

PoC: `access-control-origin-poisoning.test.ts` on PR.

— RECOMMENDATION

Authenticate before counting. Require the expected OApp/`AccessControl` admin to match payload `admin` and to be a signatory (or trusted origin) before cardinality checks. Reject observer-only foreign contracts.

Prefer selecting by trusted admin + scope rather than informee-visible duplicates. Add a regression with a genuine contract plus an attacker-signed contract that copies `category` / `id` and lists the informee only as observer.

M-16 Poison AccessControl bricks OApp administration

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

— RESOLUTION

Status: Resolved.

MEDIUM **M-17**

RESOLVED

Admin API sends OAuth tokens over plaintext

CATEGORY	Authentication & Session flaws
LOCATION	packages/vms/canton/common/src/provider.ts:144-166; client/topology/utils.ts:80-92; client/grpc.ts:55-89
REVIEW	Remediation Review 2

DESCRIPTION

The production Canton provider obtains an OAuth client-credentials token and shares it between the JSON Ledger API and the gRPC Admin API. It constructs `CantonGrpcClient` with only `host` and `tokenProvider`, so it never supplies optional TLS settings.

`buildChannelCredentials` enables default TLS only when the host string ends in `:443`. Every other port, including conventional Canton Admin API ports, selects `ChannelCredentials.createInsecure()`. `CantonGrpcClient` then attaches `Authorization: Bearer <token>` to every topology and connectivity RPC.

A non-local deployment whose Admin API is exposed on any non-443 port therefore sends its reusable OAuth bearer token without transport encryption. An on-path attacker can capture and replay that token until expiry, and can also alter unsigned topology generation responses before an automated signing workflow signs the returned hash.

New topology helpers trust the Admin API to construct the exact mapping that an external key signs (for example `createPartyToKeyTx` via `GenerateTransactions`) without decoding/checking the mapping against requested parameters. On a plaintext channel, request tampering yields an internally consistent malicious transaction and hash.

`isLocal` selects the token provider but is not passed into the credential decision, so production OAuth and local unsafe JWT deployments share the same port-based TLS behavior. Rev2 expands this channel to topology generation, submission, key mapping, party hosting, and synchronizer connectivity.

PoC: `admin-grpc-plaintext.test.ts` on PR.

RECOMMENDATION

Always use TLS for non-local Admin API endpoints. Do not infer security from port `:443`. Pass explicit TLS options from provider config (CA, server name) into `CantonGrpcClient` / `buildChannelCredentials`.

M-17 Admin API sends OAuth tokens over plaintext

CONTINUED

Before signing topology transactions, decode the returned mapping and assert it matches the requested party, keys, and threshold. Add regressions for conventional Admin ports (e.g. 5002) selecting secure credentials and rejecting insecure production configs.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

— RESOLUTION

Status: Resolved.

MEDIUM **M-18**

ACKNOWLEDGED

Factory offset seeding strands active requests

CATEGORY	DoS
LOCATION	ver-api-common/src/chain/request-poller/canton.ts:70-112; canton-chain-client.ts:224-232; request-executor.ts:156-160
REVIEW	Remediation Review 2

DESCRIPTION

A validator without a persisted request offset initializes `CantonRequestPoller` from the creation offset of the currently active `RequestFactory`. That assumes no active `Request` can predate the factory, which is false after a factory replacement while older requests remain active.

The sequencer and validators then disagree: `hasPendingRequests` sees the old active `Request`, but the poller only scans `Request_Create` exercises at or after the replacement factory offset, so it never returns that request. Empty ranges are persisted, making the omission permanent across restarts.

`RequestExecutor.execute` rejects the empty array, so no validator write response is produced.

An ordinary user can keep a request active across a planned factory cutover. If enough validators start with empty offset storage after replacement (fresh quorum / DR), every sequencer batch fails while the active request keeps `hasPendingRequests` true. Assets and fees already in flight remain pending until operators repair offsets.

Distinct from archived-request recovery: the request is active and fetchable by CID; it is omitted only because its creation event is before the poller's new seed.

Rev2 also moved offset `upsert` inside the empty-range fast-forward loop. When the first scanned range contains an active request, the in-memory offset advances but is never persisted, so a fresh validator stays dependent on the moving factory seed.

PoC: `request-factory-offset-wedge.test.ts` on PR.

RECOMMENDATION

Do not seed the poller exclusively from the current factory creation offset. Persist and restore the last processed ledger offset independently of factory identity. During factory replacement, continue scanning from the prior offset (or min of old/new) until all pre-cutover active requests are observed.

M-18 Factory offset seeding strands active requests

CONTINUED

Persist offsets even when a range returns active work. Add a cutover regression: active request created before replacement factory offset remains visible to the poller and is not permanently skipped.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-19**

ACKNOWLEDGED

Initial migration bricks existing Postgres DBs

CATEGORY	DoS
LOCATION	ver-datasource/src/migration.ts:20-38; apps/ver/ver-vapp/src/repositories.ts:65-79; migrations/202607040000000000_initial-schema.sql:3-56
REVIEW	Remediation Review 2

DESCRIPTION

Rev2 makes every PostgreSQL-backed vApp run `node-pg-migrate` before constructing repositories. The new migration history starts with an initial schema migration whose statements use bare `CREATE TABLE` / `CREATE INDEX`.

An existing pre-Rev2 validator database already contains `tries`, `request_offsets`, `state_commitments`, `requests`, and `events`, but has no `migrations` bookkeeping row for the newly introduced migration. `node-pg-migrate` therefore treats the initial migration as pending and re-runs it. Bare `CREATE TABLE` fails because the tables already exist, so startup aborts before repositories initialize.

Every upgraded validator with a preserved Postgres volume is stuck offline until operators manually baseline the migration history or recreate state. This is an availability / upgrade-path break for production deployments that retain local validator DBs across Rev2.

PoC: `existing-postgres-migration.test.ts` on PR (requires Postgres).

RECOMMENDATION

Make the initial migration idempotent (`CREATE TABLE IF NOT EXISTS` / equivalent) or ship a baseline/no-op path for databases that already have the Rev1 schema. Document and automate marking the initial migration as already applied when the legacy tables exist.

Add a CI integration test that creates the pre-Rev2 tables without a migrations row, then asserts Rev2 startup/`migratePostgresDatabase` succeeds.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-20**

RESOLVED

Observer config blocks Canton request signing

CATEGORY	DoS
LOCATION	packages/vms/canton/common/src/refresh/address-resolver.ts:55-64,153-161; wrapper-lz-sdk.ts:19-45; request-sdk.ts:206-217
REVIEW	Remediation Review 2

— DESCRIPTION

The new sign-time refresh layer resolves rotating contract references by querying active contracts visible to signing parties, then enforcing exact cardinality by template. It does not authenticate that returned contracts are signed by the expected handler/owner before counting.

An ordinary party can create a first-party template such as `RequestFactoryConfig` with itself as `handler` /signatory and the victim handler only as an observer. The victim becomes able to see the poison contract without authorizing it. At sign time, genuine + poison configs make the resolver see two matches and throw, blocking `prepareCreate` / write paths that depend on that reference.

This is the same authenticate-before-count class as Guardian `6a68a88234dea27378ae3bbd` (`0ftRegistrySdk` discovery), but a separate new refresh subsystem after the initial SDK lookup already authenticated the genuine contract. Fixing only the submitted discovery call site does not restore generic Canton request signing.

The poison contract has no expiry and cannot be archived by the victim. One unauthenticated create can permanently block signing for the targeted handler until operators intervene.

PoC: `observer-config-signing-dos.test.ts` on PR.

— RECOMMENDATION

Authenticate before counting in `queryLiveContractsForRef` / `resolveActiveContractId`. Require expected handler/owner to be a signatory (and match payload fields) before cardinality checks. Discard observer-only foreign contracts.

Carry expected origin through `ContractDataIdentifier` / `wrapLzSdk` refs. Add a regression with one genuine config and one attacker-signed config that lists the victim only as observer.

M-20 Observer config blocks Canton request signing

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

— RESOLUTION

Status: Resolved.

MEDIUM **M-21**

ACKNOWLEDGED

Postgres TLS disables server authentication

CATEGORY Trust Assumptions

LOCATION ver-datasource/src/utility.ts:11-18; ver-api-common/src/config/config.ts:61-74; apps/ver/ver-vapp/src/repositories.ts:66-90

REVIEW Remediation Review 2

DESCRIPTION

The base revision configured `node-postgres` with `ssl: true` when `POSTGRES_SSL=true`. Rev2 replaces that with `{ rejectUnauthorized: false }`. The connection is still encrypted, but certificate-chain validation and hostname verification are disabled. There is no configuration path to supply the expected PostgreSQL CA.

An on-path attacker between a validator and its remote PostgreSQL service can present any self-signed certificate and complete the TLS handshake. A malicious endpoint can select

`AuthenticationCleartextPassword`, causing the pinned `pg` client to send `POSTGRES_PASSWORD` inside the attacker-terminated TLS session. SCRAM does not prevent this because the server selects the auth method; channel binding is not enabled.

Compromised credentials expose that validator's `tries`, `state_commitments`, `requests`, `events`, and `request_offsets`. This enables disclosure/tampering of one validator's local protocol state and DoS. Protocol-level integrity still requires compromising enough validators or another trust boundary, which bounds severity to Medium.

The override appears to be an insecure workaround for Aurora/RDS CA trust issues. The correct fix is installing the required CA, not disabling server authentication.

PoC: `postgres-tls-server-impersonation.test.ts` on PR.

RECOMMENDATION

Restore server authentication for `POSTGRES_SSL=true`. Support configuring a CA file / `sslrootcert` (and optional hostname) instead of `rejectUnauthorized: false`. Fail closed if TLS is requested without a trust store.

Add a regression that a self-signed rogue Postgres endpoint is rejected when production TLS config is used.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

M-21 Postgres TLS disables server authenticationCONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-22**

RESOLVED

Local state roots split ordinary read quorum

CATEGORY	DoS
LOCATION	ver-api-common/src/controllers/validator.ts:168-200,220-287; response-quorum-checker.ts:95-132; contract-client.ts:108-138
REVIEW	Remediation Review 2

DESCRIPTION

The base validator resolved a read without an explicit `stateRoot` from the latest on-chain state commitment, so every validator shared one chain anchor. Rev2 instead calls each validator's local `stateCommitmentRepository.findLatest()`.

The standard Canton contract client does not provide a state root for read methods, so this local fallback is the normal production path. Local repositories are not synchronized atomically. After a write, each validator returns its signed transaction before `finalize()` in a fire-and-forget promise. Finalization independently polls for the on-chain commitment and persists it. During propagation, validators can have different local tips.

Each validator signs a read result containing its selected `stateRoot`. `ResponseQuorumChecker.checkRead` hashes the complete response body and groups by that hash, so different tips land in different groups even when each result is internally valid. If no tip group reaches `VALIDATOR_QUORUM_THRESHOLD`, the sequencer returns `Failed to reach quorum` and ordinary contract reads fail.

Guardian `6a57fcde80c206cf40b86af3` concerned caller-selected stale roots. Rev2 correctly requires an explicit root to exist in the repository, but the new default branch replaces a shared on-chain anchor with an eventually consistent local anchor — a separate availability regression.

PoC: `validator-local-read-quorum.test.ts` on PR.

RECOMMENDATION

For reads without an explicit `stateRoot`, resolve a shared on-chain / quorum-agreed commitment tip (or the sequencer-supplied root) rather than each validator's local `findLatest()`.

If local tips must be used, exclude `stateRoot` from the quorum body hash for this path or require validators to wait until their local tip matches the agreed head before responding. Add a regression where two validators at successive tips fail or succeed consistently under the intended policy.

M-22 Local state roots split ordinary read quorum

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

— RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

Pending callbacks leak ledger data

CATEGORY	Warning
LOCATION	apps/ver/discovery-service/src/controllers/choice-context.ts:175-182
REVIEW	Remediation Review 2

DESCRIPTION

`GET /exercise/pending-callback/list` returns each active callback's complete contract payload. Both `appId` and `limit` are optional, so an HTTP caller can request every `PendingCallback` visible to the service's informee. The server installs no authentication or authorization middleware before mounting this router.

On Canton, `PendingCallback` is signed only by the LayerZero handler. Its payload includes the fee payer, target OApp, callback arguments, action context and accept or reject decision. These values are private to the ledger stakeholders until the discovery service reads them with its backend credential. The endpoint then copies them into an unauthenticated HTTP response. Consequently, any client that can reach the service can enumerate pending cross-chain activity and recover message and settlement data that Canton did not disclose to that client.

The permissionless Daml finalizer prevents this exposure from becoming a direct custody bypass. It does not make the complete callback payload public or remove the off-ledger service's responsibility to preserve Canton visibility.

RECOMMENDATION

Authenticate clients before exposing callback data and authorize each request for the relevant OApp or party. Require an `appId`, enforce a bounded page size at the ledger query and return only fields needed by the caller. If callback finalization is meant to be permissionless, run a trusted finalizer worker or publish a deliberately minimal public work item instead of proxying the handler's full contract view.

PROOF OF CONCEPT https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/128a60e386d4849bdfcd25af072832d7f1c622a6/apps/ver/discovery-service/test/zz_POCs/PendingCallbackDisclosure.test.ts

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

OFT supply excludes hidden holdings

CATEGORY	Warning
LOCATION	contracts/protocol/canton/sdk/src/oapp/oft-registry-sdk.ts:312-368
REVIEW	Remediation Review 2

— DESCRIPTION

`getInstruments` treats the `Oft` contracts visible to one configured informee as the complete token supply. The discovery service creates the SDK with only `config.informee`, then publishes the resulting sum as `totalSupply`.

An unlocked `Oft` holding is signed by the instrument admin and observed by its owner. An unrelated discovery informee is not a stakeholder. Consequently, the ledger query cannot return every holding unless that one informee is also the admin of every listed instrument. This conflicts with the SDK's stated support for multiple issuers and makes the registry publish zero or partial supply for instruments administered by other parties. The response also labels the incomplete result with the current wall-clock time as `totalSupplyAsOf`, which makes it appear authoritative and current.

This issue does not depend on a malicious package or an unvetted interface. It follows from the visibility of the scoped production `Oft` template under the accepted package model.

— RECOMMENDATION

Do not derive global supply from contracts visible to one observer. Publish supply from an issuer-maintained aggregate or query each instrument through an authenticated source that is authorized to read every holding for that instrument. Return the ledger offset or record time used for the aggregate. If complete supply is unavailable, omit `totalSupply` or mark it explicitly as partial instead of returning it as the instrument's total supply.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/128a60e386d4849bdfcd25af072832d7f1c622a6/contracts/protocol/canton/contracts/tests/daml/ZZPOCs/DiscoveryInformeeMissesOftHoldings.daml>

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Pagination follows full ledger scans

CATEGORY	Warning
LOCATION	apps/ver/discovery-service/src/controllers/choice-context.ts:175-182
REVIEW	Remediation Review 2

— DESCRIPTION

The callback endpoint accepts a `limit`, but it calls `listPendingCallbacks(appUid)` without that limit and slices the completed array afterward. The SDK has already queried and materialized every matching active contract before the response is bounded. Omitting the optional limit also returns the entire set.

Instrument pagination has the same resource behavior. `getInstruments` queries every visible `0ftSelf`, config and `0ft` holding, builds the complete aggregate and only then applies the requested offset and page size. A single public HTTP request can therefore trigger multiple full active-contract scans even when the client asks for one result.

The server mounts both endpoints without authentication, request throttling or concurrency controls. Repeated cheap requests can consume ledger API bandwidth, backend memory, JSON processing time and wallet or identity-provider capacity. The exact availability impact depends on the number of active holdings and callbacks, so this is a bounded deployment risk rather than a demonstrated protocol-wide outage.

— RECOMMENDATION

Apply limits and cursors in the ledger query instead of after materialization. Require a bounded `limit` for callback listing. Serve instrument metadata and supply from an indexed, incrementally maintained view rather than recomputing it from the full active contract set for every page. Add deployment-level authentication or rate limiting and cap concurrent backend queries as defense in depth.

PROOF OF CONCEPT https://github.com/GuardianOrg/audit-external-team1-1782495786757/blob/e578dd31f1de7742d05bffd07e772ce2cac8cf9c/apps/ver/discovery-service/test/zz_POcs/UnboundedCallbackQuery.test.ts

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Backdated sends preload rate-limit decay

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/RateLimiter/daml/RateLimiterState.daml:234-261
REVIEW	Remediation Review 2

DESCRIPTION

`applyRateLimit` treats its `now` argument as the accounting clock. For outbound calls, however, this value is the timestamp supplied by the sender in `0App_LzSend`, not ledger `getTime`. When an EID bucket does not exist yet, `lookupOrDefaultState` copies that value into `lastUpdated`. The first send therefore installs a caller-selected historical decay anchor.

Suppose the transaction ledger time is T , the timestamp validity period is P , the outbound window is W and the limit is L . A sender can submit the first send with timestamp $T-P$ and amount L . The timestamp is valid and the absent bucket records usage L with `lastUpdated =  $T-P$` . The sender can then submit another send with timestamp T . `computeDecay` treats P seconds as elapsed and releases $\text{floor}(L * P / W)$ capacity even though the first send was only just recorded.

Configuration validation only requires $W > P$. It permits $W = P + 1$, so the two immediate sends can move almost $2L$ in total. The practical effect depends on the ratio P / W , not merely on whether the window is small in absolute terms. For a 10-second validity period:

Decay window Extra first-use capacity
--- ---
11 seconds 90.9%
60 seconds 16.7%
100 seconds 10%
1 hour 0.28%
1 day 0.012%

The excess is material when the decay window is close to the validity period. It becomes negligible when production uses an hour- or day-long window with a 10-second validity period. The trigger is also limited to one use of the global bucket or one use of each new EID bucket.

L-04 Backdated sends preload rate-limit decay

CONTINUED

Both OFT and LockUnlockAdapter pass the sender's timestamp into `IRateLimitState_RecordOutflow`, then burn or lock real assets and create the outbound request. An ordinary funded sender can therefore exceed the configured immediate outbound exposure without admin action, a foreign config or an unvetted package.

— RECOMMENDATION

Use ledger `getTime` for rate-limit decay and every `lastUpdated` or `recordedAt` anchor. Keep the caller timestamp only for fee-transfer and interactive-signing validity checks. The simplest change is to remove `timestamp` from `IRateLimitState_RecordOutflow` and obtain `now <- getTime` inside its implementation, as `RecordInflow` already does.

If public sends must avoid `getTime` because of external-signing latency, initialize every enabled resolved bucket through an admin-authorized choice that obtains ledger time and reject `RecordOutflow` while the bucket is absent. Do not synthesize an accounting anchor from a sender timestamp.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Cached synchronizer can halt settlement

CATEGORY	Unexpected Behavior
LOCATION	packages/vms/canton/common/src/client/canton-client.ts:236-244
REVIEW	Remediation Review 2

DESCRIPTION

`#ensureSynchronizerId` takes the first connected synchronizer and wraps the lookup in `once`. The returned promise is therefore permanent for the lifetime of the client. A successful lookup is never refreshed after a synchronizer disconnects or the connection set changes. A rejected lookup is also cached, so one call made while no synchronizer is connected keeps failing after connectivity is restored.

The client applies this global choice to every command submission and interactive preparation. It also applies it to disclosed contracts. In the latter case, `#fetchActiveContractEvent` discards `response.created.synchronizerId`, even though the Ledger API identifies the synchronizer that hosts the contract and `createDisclosure` replaces it with the cached first ID.

Consequently, a participant connected to more than one synchronizer can prepare LayerZero state commitments against an unrelated synchronizer merely because that ID appears first in the API response. The same failure occurs after an ordinary synchronizer migration or failover. Contract lookups, disclosed contracts or transaction preparation then target incompatible ledger state and are rejected. Since the verifier service keeps one `CantonClient`, retries reuse the same stale value or rejected promise. State commitments cannot be submitted until the process is restarted, which can halt message settlement and leave requests pending.

RECOMMENDATION

Require the LayerZero synchronizer ID in the Canton chain configuration and use it for every submission and preparation. Do not choose the first item returned by `connectedSynchronizers`.

Build disclosed contracts with the `synchronizerId` returned alongside the created event, then reject any attempt to combine disclosures from a different synchronizer. If automatic discovery remains available, accept it only when exactly one synchronizer is connected. Refresh the result after connection changes or submission failures and do not cache a rejected discovery promise.

L-05 Cached synchronizer can halt settlement

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-06**

RESOLVED

Plaintext JSON API can forge settlements

CATEGORY Best Practices

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/config/config.ts:90-140

REVIEW Remediation Review 2

DESCRIPTION

Production validator and sequencer configuration accepts any nonempty `CANTON_RPC_URL`, including non-loopback `http://` URLs in testnet and mainnet. `buildCantonClient` installs that URL as the JSON Ledger API base and attaches the OAuth bearer provider to every request. Each validator then sends its intended `Committer.CommitBatch` command to `/v2/interactive-submission/prepare` over this connection.

The validator signs the transaction returned by the participant without checking that its Daml nodes still encode the locally computed entries and state root. `CantonChainClient.#prepareAndNormalize` canonicalizes the returned protobuf and recomputes its hash, but it never compares the transaction with the original command. Therefore an on-path actor can modify the prepare request or replace its response with a different valid Canton-prepared transaction. If the actor supplies the same replacement to a validator quorum, every signature is valid over the same body-derived hash. The sequencer's signature and quorum checks accept it and Canton executes it with `lz0wner` authority.

For example, an ordinary party can create an `0App_LzReceive` request with a fresh message that pays the party's fingerprint. Request creation intentionally moves no adapter assets because the runtime's later decision is meant to prove that the remote packet is valid. The network actor can make the validators sign a `CommitBatch` that accepts this request even though the Endpoint runtime rejected or never processed it. Acceptance creates a valid gateway-signed `PendingCallback`; permissionless finalization then transfers the decoded amount from the lock/unlock adapter treasury. The actor can repeat this while treasury liquidity and inbound rate-limit capacity remain available.

RECOMMENDATION

Reject non-loopback `http://` JSON Ledger API URLs in testnet and mainnet. Require authenticated TLS with an operator-configured trust store or an equivalently authenticated private channel and rotate any OAuth credential that may have crossed plaintext transport.

Before signing, decode the prepared transaction and verify that it has exactly the expected root command, actors, `CommitBatch` contract and choice, entry list, predecessor

L-06 Plaintext JSON API can forge settlements

CONTINUED

commitment, successor state root, version and no additional root nodes. Treat this semantic comparison as mandatory defense in depth even when TLS is enabled.

— RESOLUTION

Status: Resolved.

LOW **L-07**

RESOLVED

HTTP key bootstrap lets MITM halt settlement

CATEGORY Best Practices
LOCATION apps/ver/ver-sequencer/src/server.ts:40-56
REVIEW Remediation Review 2

DESCRIPTION

`initializeServer` discovers every validator public key by calling `getPublicKey()` through the configured validator client, then immediately trusts those unsigned responses when constructing `Secp256k1Verifier`. The production configuration accepts every nonempty validator URL. It does not require HTTPS or restrict plaintext HTTP to a loopback sandbox. `HttpClient` passes the URL directly to `fetch`, so an `http://` deployment learns its validator trust anchors without authenticating the responding servers.

An attacker who intercepts a quorum of these plaintext connections can replace the startup responses with public keys they control. They can then answer later validator calls with responses signed by those keys. The sequencer accepts the responses as a validator quorum and forwards the resulting prepared transaction to Canton. Canton rejects the foreign signing keys, so the attacker cannot forge ledger authority through this issue alone. They can keep every write or genesis batch failing and halt settlement while interception continues.

The same connection carries the per-validator bearer token on `/vapp` and `/scan` requests. A passive observer can reuse a captured token for as long as it remains allowlisted, gaining authenticated access to signed reads, pending-write processing, genesis handling and validator repository queries. Repeated calls can consume chain, database, trie and signing work.

RECOMMENDATION

Pin each validator public key in sequencer configuration or another authenticated deployment artifact. Do not learn the trust anchor from the same connection it is expected to authenticate.

Parse every validator URL during configuration and require HTTPS for non-loopback hosts. Reject HTTP in `testnet` and `mainnet`; permit it only through an explicit sandbox-only development option. Support trusted CA configuration or mutual TLS for private deployments. Rotate any token that may have crossed plaintext transport.

L-07 HTTP key bootstrap lets MITM halt settlementCONTINUED

— RESOLUTION

Status: Resolved.

LOW **L-08**

ACKNOWLEDGED

Revoked libraries still receive valid quotes

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/ver-endpoint/src/dvn/dvn.ts:473-526
REVIEW	Remediation Review 2

— DESCRIPTION

`Dvn.assignJob` requires the immediate caller to be one of the DVN's authorized message libraries, but `Dvn.getFee` does not apply that check. `Executor` has the same inconsistency: `assignJob` checks `assertMessageLibrary`, while `getFee` checks only pause state, payee activation and the OApp ACL. ULN302 uses `getFee` when quoting a send and `assignJob` when executing it.

An authorized worker administrator or DVN committee can remove ULN302 from the worker's message-library set while the worker remains registered and referenced by the ULN send configuration. This can happen during an ordinary worker retirement. In that state, ULN302 still reports the destination as supported and returns a current fee quote. A send using the same configuration then fails deterministically when `assignJob` rejects ULN302 as an unauthorized caller.

A Canton user can therefore receive a valid quote and create a send request that cannot be accepted by VER. The request is rejected only after the user's upstream Canton transaction has committed. Consequently, the user can lose the non-refundable minimum fee and remain dependent on callback recovery to reverse custody and rate-limit changes.

— RECOMMENDATION

Use the same worker-eligibility check for quotation and assignment. Either require `getFee` to authenticate the calling message library or expose an `isMessageLibrary` query that ULN302 checks before including the worker's fee. Prevent a message library from being removed while an active ULN configuration still references the worker, or make revocation atomically disable those configurations.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Same-millisecond batches share command identity

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/chain/canton/utility.ts:12-23
REVIEW	Remediation Review 2

DESCRIPTION

`deriveTransactionNormalizeParams` derives the transaction UUID, command ID, root seed and ledger-time bounds only from `timestampMs`. Two different batches prepared with the same millisecond timestamp therefore receive the same normalized command identity even when they contain different Requests or produce different transaction hashes.

The interactive execution request creates a random `submissionId`, but `submitSigned` does not return it. `CantonChainClient.submitTransaction` later waits for a completion using only the shared `commandId`. `waitForCommand` accepts the first completion with that ID and does not compare the `submissionId` included by Canton.

If two writers prepare different batches with the same user, acting parties and millisecond timestamp, both can record the completion offset before either submission finishes. The first completion then matches both waiters. One writer can report that its batch was accepted even when its own submission is still pending, rejected or rejected as a duplicate. This can produce incorrect retry and recovery decisions. The condition is unlikely with one strictly sequential sequencer, but it can occur during failover, a rolling deployment or concurrent authenticated writes.

RECOMMENDATION

Derive the normalized identity from the logical batch as well as its timestamp. A canonical digest can include the predecessor commitment, ordered Request IDs, resulting state root and VApp version. The same logical batch must still produce identical command IDs, transaction UUIDs and root seeds across validators, while different batches must never share them.

Return the generated `submissionId` from `submitSigned` and require `waitForCommand` to match both `commandId` and `submissionId`.

L-09 Same-millisecond batches share command identityCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Old signed responses can pass as current

CATEGORY	Validation
LOCATION	packages/protocol/lz-ver-protocol/ver-sequencer-client/src/clients/sequencer.ts:38-57; packages/protocol/lz-ver-protocol/ver-sequencer-client/src/clients/sequencer-scan.ts:72-91
REVIEW	Remediation Review 2

— DESCRIPTION

The sequencer clients verify validator signatures over `{type, request, response}`. This proves that the validators signed the returned value for that request, but it does not prove when they signed it or that the value still represents the latest state. Root-less reads contain no client challenge, issuance time, expiry or other value that changes between otherwise identical calls. A signed response therefore remains valid indefinitely when the client later repeats the same request.

For example, assume a client calls `PriceFeed.getPrice` for destination EID `30101` while the current VER root is `S1`. The validators return and sign `{priceRatio: 100, gasPriceInUnit: 1}` together with `stateRoot: S1`. The price updater later changes the price, producing root `S2` where the values are `{priceRatio: 150, gasPriceInUnit: 2}`. If the client calls `getPrice(30101)` again, the encoded request is identical to the first call. A malicious or compromised sequencer can therefore return the complete response recorded at `S1`. The old validator signatures still verify because both the request and signed response are unchanged.

The signed `stateRoot` only proves which snapshot produced the value; it does not prove that the snapshot is still current. The generic contract client then returns only `response.value` and discards `response.stateRoot`, so the caller receives the obsolete price without even seeing that it came from `S1`. An integration using that value can build an outdated fee quote or make another decision using obsolete pricing. The same replay works for support flags, configuration, nonces, balances and scan results, including a previously signed response showing that no matching events existed.

This gives the sequencer a rollback oracle over the client's view. It cannot forge a value that validators never signed or roll back the actual ledger, but it can present genuine historical data as current.

Example:

1. At state root `S1`, a client asks for `PriceFeed.getPrice(30101)`. The validators sign a response containing the then-current price and `stateRoot: S1`.

L-10 Old signed responses can pass as current

CONTINUED

2. The sequencer stores the complete response and validator signatures.
3. A price update advances the state to `S2`, where the price for EID `30101` is different.
4. The client repeats the same root-less `getPrice(30101)` call. Because the request has no fresh challenge or expiry, its signed representation is byte-for-byte identical to the request made at `S1`.
5. Instead of obtaining a new response, the sequencer returns the stored `S1` envelope. Signature verification succeeds because it is an authentic response to the same request.
6. The contract client discards `stateRoot: S1` and returns only the obsolete price. The caller cannot tell from the typed result that the value predates the current `S2` state.

— RECOMMENDATION

Bind freshness to the signed protocol. Add a client-generated random challenge and a bounded expiry or ledger-head reference to every read and scan request and require validators to include them in the signed response. For current-state reads, also bind the response to a commitment that the client can independently recognize as current; keep explicitly historical reads as a separate API.

— RESOLUTION

Status: Acknowledged.

LOW **L-11**

RESOLVED

Admin transfer leaves old delegate active

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/daml/0App/Endpoint/SetDelegate.daml:28-34
REVIEW	Remediation Review 2

DESCRIPTION

`buildSetDelegateCallContext` encodes the OApp as its raw `appUID`. It also encodes the incoming admin as the raw 32-byte fingerprint from its Canton Party ID. EndpointV2 uses neither value as a Canton runtime address.

The validator converts the call-context caller into `deriveGlobalAddress(appUID, AddressDomain.CHAIN)`. The resulting `msgSender` does not equal the raw `appUID` supplied as the `oapp` argument. Therefore `EndpointBase.setDelegate` fails with `EndpointV2_UnauthorizedError` before it writes the new delegate.

This failure happens after `DefaultAdminTransfer_Accept` has already replaced the on-ledger `DEFAULT_ADMIN_ROLE` holder and created the request. A previous endpoint delegate remains stored in the vApp. That party can submit a public non-OApp request as its full Canton Party ID, which converts to the same runtime address already stored in the delegate map. Consequently, a former admin can still call delegate-authorized methods such as `skip`, `clear`, `burn` and the message-library configuration setters after losing its on-ledger role. These methods can discard or preempt adapter messages and strand the corresponding transfer.

The raw incoming fingerprint is also incorrect independently of the rejected call. Public Canton requests derive their runtime caller from the full Party ID with the CHAIN domain prefix, so a raw fingerprint would not authorize the incoming admin even if the `oapp` argument were fixed.

RECOMMENDATION

Use one shared, cross-language helper for every Canton runtime address. Encode `oapp` as `deriveGlobalAddress(oappIdToAppUID(oappId), AddressDomain.CHAIN)`. Encode `delegate` as `deriveGlobalAddress(partyToText(assignee), AddressDomain.CHAIN)`. Add golden vectors that compare the Daml result with the TypeScript implementation.

Do not finalize the on-ledger default-admin replacement until the vApp has accepted the delegate rotation. Keep the transfer pending and complete it from an authenticated success

L-11 Admin transfer leaves old delegate active

CONTINUED

callback or implement an equivalent acknowledgement that cannot leave the old runtime delegate active when the request is rejected.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/13>

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

ACKNOWLEDGED

Gateway can forge settlement callbacks

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/Request/daml/PendingCallback.daml:13-28
REVIEW	Remediation Review 2

DESCRIPTION

`PendingCallback` uses the gateway as its only signatory. The gateway can choose the OApp, callback data, decision, receiver and amount. The contract contains no proof that it was created from a real `Request` accepted by the validators.

Consequently, the configured gateway can create a `PendingCallback` directly and bypass request execution, fee escrow and `StateCommitment` creation. Finalization only checks that `handler` is the configured gateway and that `oappId` matches the target OApp. A directly created callback therefore passes the same checks as a genuine callback.

For example, the gateway can create an accepted receive callback with an arbitrary receiver and amount. `OftSelf` then mints tokens even though no source-chain burn occurred. `LockUnlockAdapter` can instead release treasury assets even though no source-chain lock occurred. Exploitation requires control of the configured gateway, but it does not require the OApp administrator or treasury to approve the forged settlement.

RECOMMENDATION

Create a single-use callback authorization when the original `Request` is created. Bind it to the request identifier, OApp identity, callback data and final decision. Only `Request` acceptance or rejection should be able to consume that authorization and create a valid `PendingCallback`.

Finalization must verify and consume this authorization. Storing only a parent CID inside `PendingCallback` is insufficient because the gateway could choose that value when creating a forged contract.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

RESOLVED

Non-443 admin gRPC sends plaintext tokens

CATEGORY Best Practices

LOCATION packages/vms/canton/common/src/client/topology/utils.ts:86-92

REVIEW Remediation Review 2

DESCRIPTION

`buildChannelCredentials` enables TLS only when the caller supplies `tls` or the Admin API host literally ends in `:443`. Every other host uses `createInsecure()`. This treats the port number as the transport security policy.

The production `createCantonChainProvider` cannot supply `tls`. It passes only the parsed `admin-api` host and the shared OAuth token provider to `CantonGrpcClient`. Consequently, a non-local deployment on a standard Canton port, a TLS port other than 443 or a host without an explicit port sends each Admin API bearer token over plaintext gRPC.

An on-path attacker can steal the OAuth token and alter Admin API responses. This is especially dangerous for `GenerateTransactions`: the client returns the server-supplied serialized topology transaction and hash without checking that they encode the requested mapping. The API documentation then instructs the caller to sign that hash with the namespace key. The attacker can therefore replace a requested Party-to-Key or Party-to-Participant transaction with an attacker-chosen topology transaction and obtain a valid namespace signature. Depending on the affected OAuth permissions and topology mapping, this can expose Ledger API authority or change party hosting and signing keys.

RECOMMENDATION

Make TLS explicit and fail closed for every non-local Admin API connection. Expose CA and optional client-certificate settings through `createCantonChainProvider`; do not infer security from the port number. Permit `createInsecure()` only behind an explicit local-development option.

As defense in depth, decode every generated topology transaction before returning it for signing. Verify its operation, serial, store and mapping against the request, then recompute and compare the transaction hash.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

ACKNOWLEDGED

Callbacks can settle in the wrong asset

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/LockUnlockAdapter/daml/AdapterConfig.daml:292-298
REVIEW	Remediation Review 2

DESCRIPTION

`IRequestCallback_Finalize` is permissionless and lets the finalizer supply the `executeContext`. `adapterDispatchAccept` and `adapterDispatchReject` read an `AdapterConfig` CID from that context. `fetchAndValidateAdapterConfig` authenticates only the config's `oappId` and administrator signatory. It does not bind `assetInstrumentId`, `localDecimals` or `sharedDecimals` to the adapter or to the pending callback.

`LockUnlockAdapter` stores only the OApp identity and treasury. The callback data likewise omits the asset identity. Therefore a genuine callback created under one asset configuration can be finalized against another administrator-signed config with the same OApp ID but a different instrument. This can occur when a config is replaced during an asset migration while an older callback remains active or when two operational configs reuse one OApp ID.

For an inbound callback, the finalizer supplies the second config, its honest `TransferFactory` and treasury holdings for that config's asset. `handleAdapterLzReceiveAccept` converts the authenticated wire amount using the second config's decimals and asks the factory to transfer its `assetInstrumentId`. The adapter contract contributes treasury authority to the nested transfer. The callback is then consumed even though the source message represented the first asset. An attacker can bridge a lower-value asset to their own receiver fingerprint and receive a higher-value treasury asset instead. An outbound rejection has the same defect: it can refund the sender with the substituted asset.

RECOMMENDATION

Bind a stable asset identity to every adapter callback without pinning an upgrade-unstable config CID. Store `assetInstrumentId`, `localDecimals` and `sharedDecimals` in `LockUnlockAdapter` or freeze them in the callback data when the request is created. During finalization, require the supplied config to match those authenticated values before reading treasury holdings or calling a transfer factory.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/22>

I-03 Callbacks can settle in the wrong asset

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-04

RESOLVED

Plaintext wallet API can halt settlement

CATEGORY Best Practices

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/config/config.ts:90-140

REVIEW Remediation Review 2

DESCRIPTION

`buildValidatorChainConfig` and `buildSequencerChainConfig` accept any nonempty `CANTON_WALLET_URL`. They do not reject a non-loopback `http://` URL in testnet or mainnet. Production then gives this URL to `AmuletSdk` with the same OAuth token provider used by the Canton chain service.

`AmuletSdk.getDsoParty` and `AmuletSdk.getAmuletContext` send that bearer token to the configured wallet or scan-proxy server. `getDsoParty` trusts the returned `dso_party_id` and caches it for the lifetime of the process. `getAmuletContext` also trusts the returned choice context and disclosed contracts. `StateCommitmentSdk.prepareCommitBatch` needs these values before it can prepare any nonempty batch.

If a validator or sequencer connects to a non-loopback wallet over HTTP, an actor who can intercept the connection can read the bearer token and alter the responses. For example, the actor can replace the first DSO response with another party ID. The service caches that value, requests the wrong Amulet context and keeps submitting invalid data on later attempts. The actor can also remove or replace disclosures in each context response. Canton authenticates disclosed contracts, so fabricated data is rejected rather than accepted as valid settlement input. Consequently, the actor cannot create value through this issue, but can prevent new VER batches from being prepared and leave pending adapter requests unsettled. A poisoned DSO value continues causing failures until the process restarts.

Exploitation requires an operator to configure a non-loopback plaintext wallet URL and an actor with a network position between the service and that endpoint. It does not require a malicious DAR, an OApp administrator or a validator signing key. This is separate from plaintext validator key discovery because the wallet connection carries a bearer token and supplies the Amulet data needed for settlement.

RECOMMENDATION

Parse `CANTON_WALLET_URL` during configuration and require HTTPS for testnet, mainnet and every non-loopback host. Permit HTTP only through an explicit sandbox-only option. Support a trusted CA bundle or mutual TLS for private wallet deployments.

I-04 Plaintext wallet API can halt settlement

CONTINUED

Treat response validation and cache recovery as defense in depth. Validate the DSO response before caching it and clear the cached identity after authentication or context failures. Rotate any OAuth token that may have crossed plaintext transport.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

Random signatures inflate validator quorum

CATEGORY Validation

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/services/response-quorum-checker.ts:220-230

REVIEW Remediation Review 2

DESCRIPTION

`ResponseQuorumChecker.#check` counts distinct signature strings instead of distinct validator identities. `checkRead` and `checkWrite` verify that each signature belongs to some configured key, but `Secp256k1Verifier.verify` discards the recovered public key. Consequently, two different signatures from one authorized key count as two validators.

This matters because KMS ECDSA signing is randomized. Repeatedly signing the same response can produce different valid signature bytes. The code itself acknowledges this property in the TODO immediately before `group.set(signature, response)`.

The sequencer also permits one identity to occupy multiple committee entries. `buildSequencerConfig` only checks that the threshold does not exceed the raw `validators.length` at `config.ts:174-190`. During startup, `server.ts:40-62` creates one client per entry and accepts the discovered keys without requiring unique URLs or public keys. If the same validator key occupies enough entries to reach the threshold, its byte-distinct signatures all pass verification and increase `group.size`.

A malicious validator in such a configuration can return the same signed error through its aliases. Those responses can reach the threshold first, causing `checkWrite` to throw at `response-quorum-checker.ts:189-197` and repeatedly preventing sequencer batch submission. The same validator can manufacture an internal read quorum. A committee-aware external client rejects that read because it counts recovered keys, but a client using `NoopQuorumVerifier` accepts it. Duplicate same-party transaction signatures may still be rejected by Canton, so this issue does not claim that one validator can submit arbitrary state transitions.

This issue requires duplicate endpoint entries, endpoint aliases sharing a key or multiple services backed by one KMS key.

RECOMMENDATION

Make signature verification return the recovered canonical public key, then key each response group by that signer identity rather than by the signature bytes. Reject duplicate

I-05 Random signatures inflate validator quorum

CONTINUED

validator URLs and duplicate discovered public keys during startup. Validate the threshold against the number of unique keys.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

RESOLVED

Atomic DVN admin replacement is unreachable

CATEGORY Best Practices

LOCATION packages/protocol/lz-ver-protocol/ver-api-common/src/schemas/vapps/uln-302.ts:398-455

REVIEW Remediation Review 2

DESCRIPTION

`Dvn.quorumReplaceAdmins` removes every current DVN administrator and installs one new administrator in a single state transition. It requires committee signatures, checks the VID and expiry, then consumes the signed hash to prevent replay. However, the public DVN ABI defines only `quorumChangeAdmin`, which adds or removes one administrator at a time. The production vApp method map also exposes `quorumChangeAdmin` but does not register `quorumReplaceAdmins`. Consequently, callers cannot encode or dispatch a production request to the atomic method even though its implementation and direct unit tests exist.

During a planned rotation or compromise recovery, operators must instead submit several independent `quorumChangeAdmin` requests. Until every request succeeds, the DVN can contain a mixture of old and new administrators. A delayed, reordered or rejected request can leave obsolete administrators active for longer than intended or leave the replacement incomplete.

RECOMMENDATION

Add a `quorumReplaceAdmins` ABI with `newAdmin`, `vid`, `expiration` and `signatures`, matching the implemented method and `hashQuorumReplaceAdmins`. Register it in the production DVN write-method map so the container can dispatch it. Document when operators should use the atomic reset instead of several single-admin changes.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-07

ACKNOWLEDGED

Ledger offsets lose precision in JavaScript

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/chain/request-poller/canton.ts:70-93
REVIEW	Remediation Review 2

DESCRIPTION

Canton Ledger API offsets are 64-bit integers, but the common Canton client represents ledger ends, completion offsets, update offsets and active-contract snapshot offsets as JavaScript `number` values. `CantonRequestPoller.poll` also converts its persisted `bigint` cursor with `Number(...)` before pagination, then converts the resulting `number` back to `bigint` when it saves the cursor.

JavaScript cannot represent every integer above `Number.MAX_SAFE_INTEGER`, which is $2^{53} - 1$. At that boundary, two adjacent ledger offsets can become the same `number`. The poller can therefore query or persist a rounded range boundary, causing an update range to be repeated or skipped. `waitForCommand` has the same weakness because it parses completion offsets as `number` and advances its cursor with `Math.max`. A rounded completion cursor can make reconciliation resume from a different offset than the participant returned.

This could hide a Request, repeat request processing or confuse command completion tracking if a participant offset exceeds the safe-integer range. No current deployment at such an offset was provided and a normally growing ledger is unlikely to reach it soon.

RECOMMENDATION

Represent every ledger offset as `bigint` or a canonical decimal string from JSON decoding through comparison, pagination, persistence and completion waiting. Use a lossless JSON decoder for offset-bearing responses. Convert an offset to the Ledger API's expected wire representation only when constructing a request and reject any numeric input that is not a safe integer.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

ACKNOWLEDGED

Hard-fork recovery omits required parent

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/services/state-recoverer.ts:182-209
REVIEW	Remediation Review 2

DESCRIPTION

`StateRecoverer` cannot seed a fresh PostgreSQL validator database after a versioned hard fork. When the backward walk reaches the previous VApp version, it saves only that commitment's state root and stops. It does not persist the previous-version commitment. The new version's genesis is then inserted unchanged, including its `previousId` that names the omitted commitment.

The production migration defines `state_commitments.previous_id` as a foreign key to `state_commitments.id`. PostgreSQL therefore rejects the recovered genesis because its parent is absent. The recovery loop repeats the same failing insert on every cycle. The hard-fork tests do not expose this failure because they use `MemoryStateCommitmentRepository`, which accepts a record whose parent was never inserted.

This prevents a new validator from joining after a hard fork unless it receives the complete previous-version database. It also prevents a validator from recovering after losing or rebuilding that database. If the hard fork is deployed with fresh validator databases, the genesis can be committed on Canton while every validator fails to persist it. Subsequent writes then fail because the current state root is absent locally. Losing enough validators this way removes write quorum and stops pending cross-chain requests from finalizing.

RECOMMENDATION

Store a valid local version boundary before inserting the hard-fork genesis. For example, persist the previous-version fork point as an anchor with no local parent, then insert the new genesis in the same database transaction. Alternatively, store the new genesis without a local `previousId` after verifying its inherited root against the on-chain predecessor. Add a PostgreSQL-backed regression test that starts with an empty database and recovers a genesis whose on-chain parent belongs to the previous VApp version.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

ACKNOWLEDGED

Issuer filters break transfer context API

CATEGORY Unexpected Behavior

LOCATION contracts/protocol/canton/sdk/src/oapp/oft-registry-sdk.ts:447-453,519-529

REVIEW Remediation Review 2

DESCRIPTION

The discovery service is configured around one `INFORMEE` party. `OftRegistrySdk` first finds each `OftSelf`, `OftSelfConfig` and `OftTransferRule` through that party and the contracts are therefore already visible to it. When the service builds the disclosed-contract bundle for a transfer, however, it changes the Ledger API event filter to `[instrumentId.admin]`.

Canton authorizes every key in `eventFormat.filtersByParty` as a `ReadAs` claim. The service user therefore needs `CanReadAs` for each independent issuer admin, even though it only needs the configured informee to read the same contracts. A production OAuth user provisioned with the least-privileged `CanReadAs(INFORMEE)` right receives `403` from `createDisclosure`. This deterministically breaks `/transfer-factory` and the accept, reject and withdraw choice-context endpoints for issuers other than the service party.

The alternative deployment workaround is to grant the public discovery backend `CanReadAs` for every issuer or `CanReadAsAnyParty`. That defeats the intended single-informee observer model and unnecessarily broadens the credential's visibility across issuer-private ledger data. Sandbox testing can conceal the defect because `LocalTokenProvider` creates an admin token by default, while the controller tests mock both affected SDK methods and never exercise Canton authorization.

RECOMMENDATION

Create all three disclosures through `[this.#informee]`, as the existing `#createDisclosure` helper and `getOftSelfContracts` path already do. At startup, inspect the service user's rights and fail clearly if it cannot read as the configured informee; do not require arbitrary issuer rights.

Add a Ledger API integration test with distinct informee, issuer, sender and receiver parties. Give the service user only `CanReadAs(informee)` and confirm that transfer-factory plus accept, reject and withdraw context construction succeeds without placing the issuer in `filtersByParty`.

I-09 Issuer filters break transfer context API

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Removing Simple ML blocks the vApp hard fork

CATEGORY	DoS
LOCATION	apps/ver/ver-vapp/src/server.ts:62-88; ver-api-common/src/container/container.ts:99-133; validator-genesis-initializer.ts:66-75
REVIEW	Remediation Review 2

DESCRIPTION

Rev1 constructed the production container with four ordered components: Endpoint, ULN-302, Simple Message Library, and Blocked Message Library. Rev2 removes Simple Message Library and builds only Endpoint, ULN-302, and Blocked Message Library.

Container component roots are positional. Hard forks may only inherit the existing prefix and append new components. `Container.initialize` loads the previous root and throws `Cannot drop container components` when the new list is shorter. `ValidatorGenesisInitializer` always passes the prior version state root during a hard fork, so every Rev2 validator fails before building a genesis transaction. The sequencer cannot obtain write quorum or submit the new genesis commitment.

Deploying without incrementing `VAPP_VERSION` does not safely avoid failure: the sequencer skips genesis, but three runtime components are paired by index with four inherited roots. The new Blocked Message Library receives the old Simple Message Library root at index 2, and the old Blocked root at index 3 is unused — breaking existing routes.

Distinct from the Low finding about unpinned future container field slots: this is a concrete Rev2 production component-list shortening that blocks the upgrade path.

PoC: `simple-library-removal-hardfork.test.ts` on PR.

RECOMMENDATION

Do not drop positional components on a hard fork. Keep a deprecated/no-op Simple Message Library component at the inherited index, or migrate with an explicit remapping strategy supported by `Container.initialize`.

Add a release gate that initializes Rev2 components from a Rev1 state root in CI and fails if components are dropped. Document the supported upgrade matrix.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/23>

I-10 Removing Simple ML blocks the vApp hard fork

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-11

ACKNOWLEDGED

Completed transfers can short-pay recipients

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/contracts/Layerzero/FeeUtils/daml/Transfer/Core.daml:176-182
REVIEW	Remediation Review 2

DESCRIPTION

`proposePendingTransfer` accepts every `TransferInstructionResult_Completed` result but discards `receiverHoldingCids`. It does not verify that the returned holdings belong to the intended receiver, use the expected instrument, are unlocked or total the amount in the inbound message.

`handleAdapterLzReceiveAccept` then archives the `PendingCallback`. Consequently, a recipient can be underpaid even though the callback succeeds and cannot be retried.

This does not require a malicious DAR or fabricated value. The repository's adjacent `transferViaFactory` helper states that a valid `TransferFactory` may charge fees or otherwise deliver less than the nominal amount. That helper checks the completed holdings, while `proposePendingTransfer` does not.

Exploitation requires all of the following conditions. There must be a genuine accepted inbound callback for a positive amount and enough treasury holdings to settle it. The finalizer must have the contracts and disclosures needed to submit the transaction. At least one vetted, issuer-authenticated factory and context must be able to return `Completed` while delivering less than the nominal amount. Finally, the finalizer must be able to choose or modify the unbound `executeContext`, which contains the factory CID, treasury holding CIDs and factory-specific context.

This finding assumes that the executor is trusted to relay the transaction but is not trusted to decide how much treasury value the recipient receives. The executor does not need treasury or OApp admin keys because it only supplies the unbound finalization inputs. If the executor is expressly trusted to choose arbitrary fees or settlement amounts, this is trusted executor behavior rather than a privilege-boundary violation.

The issue is not exploitable if the deployment technically enforces any one of these protections: every reachable factory always delivers the full amount, an on-ledger rule independently validates every completed receiver output or a trusted service cryptographically binds the callback and the entire `executeContext` so that the executor cannot substitute any input. Merely sending the executor a recommended context is not enough if it can rebuild the transaction.

I-11 Completed transfers can short-pay recipients

CONTINUED

The generic adapter does not enforce an exact-output factory restriction. The Token Standard does not guarantee that every `Completed` result equals the requested amount. This repository explicitly accounts for fee-charging factories elsewhere. The test fixtures use fee-free Amulet, but that does not prove the same guarantee for every production factory or future upgrade. The live deployment configuration is outside the reviewed scope, which is why this finding is rated Medium.

The PoC uses an issuer-authenticated factory that conserves all value. For an authenticated inbound amount of 1,000, Bob receives 100 and the issuer receives a 900 fee. The callback is still consumed, so Bob cannot retry it to recover the missing 900.

— RECOMMENDATION

Handle `TransferInstructionResult_Completed receiverHoldingCids` separately in `proposePendingTransfer`. Use `sumUnlockedHoldingAmounts receiver instrumentId receiverHoldingCids` and abort unless the total equals the requested amount. This also authenticates the holding owner, instrument and unlocked state.

This on-ledger result check is preferable because it preserves factory and package upgradeability while making correctness independent of the executor and off-ledger transaction-construction policy. A deployment may additionally restrict admissible factories and contexts or bind the full `executeContext` to a trusted prepared transaction, but an operational convention alone should not replace the result check. If the adapter is intentionally supported only for a fee-free factory such as the CIP-0078 Canton Coin implementation, document and enforce that compatibility restriction rather than relying on the current test fixture to imply it.

Keep the current pending-instruction behavior because the receiver validates delivery when accepting later.

PROOF OF CONCEPT <https://github.com/GuardianOrg/audit-external-team1-1782495786757/pull/12>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

ACKNOWLEDGED

Losing validators keep buffering responses

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-transport/src/transport/http.ts:22-39; packages/protocol/lz-ver-protocol/ver-api-common/src/services/response-quorum-checker.ts:244-320
REVIEW	Remediation Review 2

DESCRIPTION

Every validator fan-out uses `HttpClient.fetch`, which calls `response.text()` and materializes the complete response before parsing it. The transport has a time limit but no `Content-Length` check, streaming byte cap or incremental JSON limit. `ResponseQuorumChecker.#fanOut` also starts detached work for every validator and has no cancellation path after a quorum result resolves.

Consequently, a configured validator that is not needed for quorum can return or continuously stream an oversized body. Healthy validators may satisfy q-of-N and let the caller continue, but the losing request keeps receiving and retaining bytes until it completes or its 30-second timer fires. Repeating ordinary sequencer reads, scans, writes or background processing can overlap these detached buffers and exhaust the sequencer's memory or event-loop capacity. A single below-threshold malicious or compromised validator can therefore degrade or crash a service that the quorum model is supposed to keep available.

Because of this issue a below-threshold validator can consume unbounded sequencer memory and event-loop capacity, degrading or crashing the settlement infrastructure even after honest validators have satisfied quorum.

The actor must be a configured malicious or compromised validator, but it does not need enough identities to control quorum. The 30-second default timeout bounds duration per call but not bytes or overlapping calls.

RECOMMENDATION

Read response bodies through a bounded stream, reject an excessive declared `Content-Length` and enforce a hard maximum number of received bytes regardless of transfer encoding. Pass a caller-owned `AbortSignal` into each validator request and abort all unfinished fan-outs as soon as quorum succeeds or becomes impossible.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Rule rotation blocks callback finalization

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/sdk/src/executor/executor-sdk.ts:468-499
REVIEW	Remediation Review 3

DESCRIPTION

`ExecutorSdk.prepareFinalizeCallback` discovers every active `ITransferRule` visible to the executor whose instrument matches the pending callback's OApp. It then calls `atMostOne` on the result. Two matching rules therefore throw `Multiple contracts found (2)` before the SDK constructs the callback-finalization command.

Two matching rules are an expected result of the supported OFT rule-rotation flow. An OApp administrator creates a replacement `OftTransferRule` and exercises `I0AppConfig_SetTransferRule`. That choice consumes and recreates only `OftSelfConfig`; it does not archive the prior rule. Archiving the old rule is not generally safe while an active `OftTransferOffer` still stores that exact rule CID and must exercise it when the receiver accepts. Since rule observers are fixed at creation, the old and replacement rules remain co-visible when both were created for use by the official gateway/executor.

After such a rotation, any ordinary user's OFT accept or reject callback that reaches `prepareFinalizeCallback` makes the interface query return both genuine, admin-signed rules. `atMostOne` rejects the result before producing `Executor.FinalizeCallback`, leaving outbound locked holdings awaiting burn/unlock or inbound mint callbacks pending through the official SDK. Operators can recover with a custom transaction, an SDK fix or carefully timed archival, so the failure is recoverable and Low severity rather than a permanent custody loss.

The transfer-rule disclosure is unnecessary for these callbacks. The target OFT callback accept/reject handlers do not fetch or exercise `ITransferRule`; they use the OApp config, holdings or mint/burn state, registry and rate limiter. The baseline SDK selected the exact rule CID from the config rather than enumerating all matching implementations, so the target's implementation-aware discovery introduced this multiplicity failure.

The violated property is that supported rule rotation and implementation-CID coexistence must not disable unrelated callback settlement. This is not an arbitrary conflicting singleton deployed by an untrusted party: the documented admin rotation creates and selects a valid replacement while old offers can still depend on the valid prior rule.

L-01 Rule rotation blocks callback finalization

CONTINUED

RECOMMENDATION

Remove transfer-rule discovery and disclosure from `prepareFinalizeCallback`, because the target callback handlers do not use it. If a future callback path requires a rule, select the rule explicitly bound by the current authenticated `0ftSelfConfig` rather than requiring exactly one visible interface implementation for the instrument.

RESOLUTION

Status: Resolved.

LOW **L-02**

ACKNOWLEDGED

Nonpositive poll settings stall validators

CATEGORY	Validation
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/config/config.ts:108-119
REVIEW	Remediation Review 3

DESCRIPTION

`buildValidatorConfig` parses `CANTON_REQUEST_BATCH_SIZE` and `CANTON_REQUEST_OFFSET_LIMIT` with bare `Number(...)` calls. It does not require either value to be a positive safe integer. Explicit zero and an empty environment value both become `0`; negative, fractional, infinite and `NaN` values are accepted too.

The offset limit must always advance the ledger cursor. `CantonRequestPoller.poll` repeatedly calls `#getNextOffset` while it fast-forwards through empty ranges. At `request-poller/canton.ts:70-93,155-158`, a zero limit makes that function return the current offset. Consequently, an empty range never makes progress. The validator either repeats the same equal-bound Ledger API query indefinitely or receives an error for that range, depending on the participant API. In both cases it cannot produce a write response.

A zero batch size fails independently. The poller can discover active Requests but `requestIds.slice(0, this.#batchSize)` always returns an empty array. `RequestExecutor.execute` rejects that result at `request-executor.ts:156-160`. The sequencer continues seeing pending Requests and retries, but those Requests cannot enter a state commitment.

The realistic failure condition is an operator-supplied zero value or a deployment template that emits an empty value. The production defaults are positive and no remote caller can change these environment variables. If enough validators share the malformed deployment setting to lose quorum, all pending adapter Requests remain unsettled until operators correct the setting and restart them. No permanent loss or unauthorized value movement is established.

RECOMMENDATION

Parse both variables with one bounded positive-safe-integer schema and fail startup on invalid input. Reject empty strings, zero, negatives, fractions, non-finite numbers and values above a documented operational maximum. Keep the existing positive defaults only when the variable is genuinely absent.

L-02 Nonpositive poll settings stall validators

CONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Interface registries break finalization

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/sdk/src/executor/executor-sdk.ts:178-183
REVIEW	Remediation Review 3

DESCRIPTION

`ExecutorSdk.prepareFinalizeCallback` discovers registries through the stable `IRegistry` interface and correctly filters them with `IRegistryView.oappId`. Its `LzReceive` resolver then discards that interface view and casts each implementation's concrete `createArgument` to the canonical `Registry` template. `findRegistryForFingerprint` consequently reads canonical-only fields `oappId` and `fingerprintToHint` from an arbitrary interface implementation. The baseline queried only the canonical template through `getRegistries`; the target generalized discovery to the interface without generalizing the subsequent payload handling, so the regression is introduced by the active delta.

Daml does not require an `IRegistry` implementation to expose either field in its concrete payload. The interface view supplies `oappId` and the `GetPartyId` choice supplies fingerprint resolution. The target's own `MaliciousRegistry` test fixture demonstrates the schema freedom by implementing `IRegistry` without either canonical field; its wrong signatory is independently rejected on-ledger. A reviewed alternative can likewise use implementation-specific fields while returning the expected OApp ID and party for the same fingerprint. The TypeScript cast is erased at runtime, so such a contract either throws while reading `payload.oappId` or `fingerprintToHint` or is incorrectly reported as not containing the fingerprint.

The on-ledger finalizer intentionally accepts this implementation.

`OApp.Callback.RegistryResolve.resolvePartyByFingerprint` fetches `IRegistry`, verifies that the expected OApp admin is an actual signatory, compares the interface view to the expected OApp ID and exercises `GetPartyId`. It never requires the canonical `Registry` payload. Thus the official builder is narrower than the transaction it is meant to prepare.

The realistic failure condition is an honest OApp administrator deploying or rotating to a reviewed, admin-signed `IRegistry` implementation with a different concrete schema. When an ordinary inbound `LzReceive` callback becomes pending, the executor's interface query finds the registry, but the concrete cast fails before the SDK can disclose it or build `Executor.FinalizeCallback`. Destination mint or unlock remains pending until operators use a custom builder or restore a canonical registry. No unauthorized value movement or permanent

L-03 Interface registries break finalization

CONTINUED

loss is established and the change requires legitimate operator adoption of an alternate implementation.

— RECOMMENDATION

Keep registry selection interface-driven. Extend the stable interface view with the minimum fingerprint-membership data needed for discovery or add an interface choice/query that lets the executor resolve a fingerprint without reading the concrete payload. Use the returned interface view and interface contract ID throughout callback preparation. Do not cast an interface query result to `Registry`.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Transfer context drops the offer-bound rule

CATEGORY Unexpected Behavior

LOCATION contracts/protocol/canton/sdk/src/oapp/oft-registry-sdk.ts:489-495,530-553

REVIEW Remediation Review 3

DESCRIPTION

`OftTransferOffer` stores the exact `transferRuleCid` that was selected when the sender created the offer and acceptance later exercises that same contract. `buildTransferInstructionContext` fetches the offer but does not use its stored rule CID. Instead, it independently queries the first active transfer rule for the instrument and discloses that contract.

This breaks the official acceptance context after a normal rule rotation. The supported `I0AppConfig_SetTransferRule` flow creates a replacement config but does not archive the previous rule, because outstanding offers can still refer to it. If the SDK selects the replacement rule while an offer remains bound to the previous one, the acceptance command exercises the previous rule without its required disclosure and fails. The receiver cannot complete the transfer through the advertised context until the sender rejects or withdraws the offer, the offer expires or a client constructs the correct disclosure manually.

RECOMMENDATION

Read `transferRuleCid` from the fetched offer, fetch and validate that exact active interface contract and disclose it. Confirm that its interface view is bound to the offer's instrument and expected administrator.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Sequencer SDK sends bearer tokens over HTTP

CATEGORY Validation

LOCATION packages/protocol/lz-ver-protocol/sequencer-sdk/src/provider.ts:67-68

REVIEW Remediation Review 3

DESCRIPTION

`parseCantonSequencerUri` accepts every URL scheme supported by `URL`, including non-loopback `http://` endpoints. `createCantonSequencerProvider` passes that URL to the read and scan clients. Those clients require an authorization header and attach it to requests at `sequencer-sdk/src/clients.ts:47-56` and `77-86`. `HttpClient` then sends the request without adding any transport policy.

An on-path network actor can therefore read a provider's bearer token when an operator configures a remote HTTP sequencer. The token protects all non-health sequencer endpoints, so the actor can reuse it while its JTI remains allowlisted to query contract reads, requests, events, nonces and state commitments. The same actor can also replace HTTP responses. This becomes an integrity failure when the supported optional-committee configuration is omitted: `resolveQuorumVerifier` selects `NoopQuorumVerifier`, which accepts any schema-valid response even with no validator signatures.

The realistic preconditions are a non-loopback HTTP provider URI and an actor able to observe or alter that connection. No OApp administrator, default administrator, malicious DAR or validator signing key is required. The violated property is that credentials and authenticated read results must remain confidential and authentic between the SDK consumer and the sequencer. The impact is limited to credential disclosure and off-ledger read/scan manipulation; this issue does not prove an on-ledger settlement forgery.

RECOMMENDATION

Reject non-loopback HTTP in `parseCantonSequencerUri` before constructing either client. Reuse the shared secure-transport validator or an equivalent package-local helper that permits HTTP only for loopback development and requires HTTPS everywhere else. Consider also requiring a validator committee for every non-loopback deployment so transport authentication is not the only read-integrity control.

L-05 Sequencer SDK sends bearer tokens over HTTPCONTINUED

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

RESOLVED

Rights RPC targets the participant admin API

CATEGORY	Warning
LOCATION	packages/vms/canton/common/src/client/grpc.ts:80-88
REVIEW	Remediation Review 3

DESCRIPTION

`CantonGrpcClient` creates one gRPC transport from `opts.host`, which the public provider documents and supplies as the participant Admin API endpoint. The new `grantCanActAs` method attaches `com.daml.ledger.api.v2.admin.UserManagementService` to that same transport.

Despite `admin` in its protobuf package name, `UserManagementService` belongs to the gRPC Ledger API. Canton exposes the Ledger API and participant Admin API on separate endpoints. Consequently, a standard split-port participant does not serve `GrantUserRights` at the configured Admin API host. SDK consumers that call the new public method cannot grant the service user `CanActAs`, so external-party onboarding or later command submission remains incomplete until an operator grants the right through a Ledger API client or manual tooling.

RECOMMENDATION

Do not attach `UserManagementService` to the Admin API transport. Keep rights management on the existing JSON Ledger API client or add a distinct gRPC Ledger API endpoint and transport to `CantonGrpcClient`. Add a split-port integration test that calls `grantCanActAs` and confirms the right through the Ledger API.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Request poller fans out before batch cap

CATEGORY	Unexpected Behavior
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/chain/request-poller/canton.ts:77-93,122-152
REVIEW	Remediation Review 3

DESCRIPTION

`CantonRequestPoller` is configured to return at most `batchSize` active Requests for one validator write round. The default batch size is five while the default offset window is 1,000.

The poller does not apply that work limit before resolving the Requests. `#fetchRequestIds` obtains every `Request_Create` result in the offset range without passing the client supported `limit` option. It then calls `fetchEventsByContractId` for every result through one `Promise.all`. Activeness and trusted origin checks run only after all those calls finish. Finally, `poll` reduces the resulting list to `batchSize`.

The first nonempty offset range is also resolved twice. The fast forward loop resolves it to determine that it is nonempty and the collection loop immediately resolves the same range again. While Requests remain pending, `RequestProcessor` begins the next write round without an additional delay and each validator repeats the polling work.

A dense page can therefore open far more participant API requests and promises than the number of Requests selected for execution. Ordinary Request creation requires a positive configured fee and the participant may impose deployment specific response limits. No production resource threshold or validator outage has been established, so this is an operational hardening concern rather than a demonstrated denial of service.

RECOMMENDATION

Page through each offset range with a continuation cursor and stop resolving candidates once the configured batch has been filled. Resolve contract histories through a small fixed concurrency pool or a bounded batch query. Keep active Requests in a separate retry set so completed Requests do not force repeated history lookups while preserving complete offset processing before the durable cursor advances.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

ACKNOWLEDGED

Cached package IDs block DAR switchovers

CATEGORY	Unexpected Behavior
LOCATION	contracts/protocol/canton/sdk/src/shared/utils.ts:47-57
REVIEW	Remediation Review 3

DESCRIPTION

Canton package name identifiers let the ledger select a vetted package version during a compatible upgrade. `ExecutorSdk`, `StateCommitmentSdk`, `OfSelfSdk` plus `RequestSdk` instead resolve each package name through `getPackageId` and store the returned exact package ID in an instance map through `getCached`. The cache has no expiry or invalidation hook and is not scoped to a vetting generation. `CantonSdk.buildCreate` and `CantonSdk.buildExercise` then replace the generated `#packageName` prefix with that cached hash. Direct submission omits `packageIdSelectionPreference` while interactive preparation always provides an empty list.

After an SDK instance caches version one, vetting a compatible version two does not alter commands prepared by that instance. While version one remains vetted, Executor and Committer choices continue using its choice bodies after the intended switch. Once version one is unvetted, later operations keep submitting the stale package ID and fail until the affected SDK instances are recreated. This can prevent a reviewed package fix from taking effect and can pause callback finalization, request creation or state commitment processing during a normal upgrade.

RECOMMENDATION

Preserve the generated `#packageName` identifiers in `buildCreate` and `buildExercise` for upgrade compatible packages so Canton resolves the vetted version at submission. Remove the exact package ID caches from the four SDKs. If a workflow deliberately selects an exact version, resolve it for the current submission and send the same ID in `packageIdSelectionPreference` instead of retaining it across submissions.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Dense request ranges halt validator polling

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/config/config.ts:38-39,108-118; packages/protocol/lz-ver-protocol/ver-api-common/src/chain/request-poller/canton.ts:64-87,116-165
REVIEW	Remediation Review 4

— DESCRIPTION

The Canton request poller scans ledger updates in offset ranges, selects active Requests and persists its progress after it has passed a range. Production currently uses the shipped `CANTON_REQUEST_OFFSET_LIMIT` default of 1,000 while the participant limits blocking HTTP lists to 200 elements. However, `#fetchRequestIds()` calls `getUpdates()` without the supported result limit. The `/v2/updates` endpoint therefore returns HTTP 413 when one range contains more than 200 matching updates.

The exception occurs before `poll()` calls `IRequestOffsetRepository.upsert()`. The validator produces no signed write response and the failed request does not contribute to quorum. The sequencer retries after its error delay, but each affected validator starts from the unchanged offset and receives the same deterministic 413. When enough validators share the production configuration, no write batch can reach quorum until an operator changes the configuration or the poller recovery behavior.

An ordinary fee paying party can create a generic Request because `IRequestFactory.Request_Create` is controlled by `sender` when `oAppInput` is `None`. The project confirmed that production has no request rate limit beyond the minimum request fee and Canton traffic fees. More than 200 matching Request updates can therefore accumulate in one unadvanced range through a burst or sustained backlog. At the stated expected mainnet minimum fee of 5 CC, an aligned set of 201 Requests costs at least 1,005 CC plus traffic fees. Once the range exceeds the node limit, unrelated Requests also remain unsettled until operator intervention.

Starting the initial cursor at the RequestFactory creation offset only skips earlier ledger history. It does not prevent a later range from exceeding the node limit.

— RECOMMENDATION

M-01 Dense request ranges halt validator polling

CONTINUED

Derive the poll range and the participant list limit from one configuration source and reject startup unless `CANTON_REQUEST_OFFSET_LIMIT` is no greater than `http-list-max-elements-limit`. When the participant still returns HTTP 413, reduce the current range and resume from the last consumed offset so the poller makes progress without skipping unread updates.

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Zero peers strand outbound transfers

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/OneSig/daml/OneSig/I0AppConfig/Cal l.daml:278-281; contracts/protocol/canton/contracts/Layerzero/Oft/daml/OftSelfCo nfig.daml:174-175,393-395
REVIEW	Remediation Review 4

DESCRIPTION

`I0AppConfig_SetPeer` manages the remote OApp address for a destination EID. Canton exposes no `RemovePeer` choice or any other administrative operation that deletes an EID from `peers`. `PauseDstEids` can stop outbound sends, but it leaves the peer registered and therefore is not a removal mechanism. `I0AppConfig_SetPeer` is consequently the only supported path through which an administrator can remove a peer.

The other LayerZero implementations confirm the removal semantics of this setter. The sibling EVM OApp documents `bytes32(0)` as peer removal and its `_getPeerOrRevert` treats zero as absent. The Stellar OApp uses `None` through the same setter to remove a peer. Because the Canton choice accepts a `bytes32 Text` value rather than an optional value, the all zero `bytes32` value is its corresponding removal sentinel and must not remain usable for outbound messages.

The production config invariants require `bytes32` syntax but do not reject the all zero value. As a result, `setPeerImpl` inserts zero into `peers` instead of removing the EID. The adapter implementation mirrors the same behavior.

There are two routes to this faulty state update. A valid signer quorum can execute `OneSig OpSetPeer`, whose dispatcher forwards `peer` to `I0AppConfig_SetPeer`. Independently, a configuration administrator can exercise `I0AppConfig_SetPeer` directly. Both routes call the same production `setPeerImpl` implementation.

Canton's `getPeerOrRevert` checks only whether the EID is present in the map. After either route stores zero to disable a peer, both quote and send therefore return zero as a usable peer. The OFT path locks the sender's tokens and creates a request whose `EndpointV2` receiver is zero. The adapter similarly moves the user's assets into treasury custody. The shipped VER Endpoint accepts zero as a `bytes32` receiver, advances the outbound nonce and emits the packet instead of rejecting it.

When that request is accepted, the OFT callback archives the locked holding while the adapter retains the backing assets in its treasury. No destination OApp exists at the LayerZero zero sentinel to mint or release the corresponding assets. A later peer repair only

M-02 Zero peers strand outbound transfers

CONTINUED

protects future transfers and cannot restore transfers already committed to zero. Thus either a quorum approved OneSig update or a direct administrator update intended to remove a peer can make subsequent users irreversibly lose or strand cross chain assets. The general administrator route does not depend on a OneSig specific root cause.

— RECOMMENDATION

Preserve the LayerZero zero as removal semantic in every `setPeerImpl` implementation. Delete the EID from `peers` when `peer` is all zero instead of inserting it. As defense in depth, make `getPeerOrRevert` reject both a missing entry and an all zero value. Exclude zero peer values from template construction so legacy or externally created configs fail closed before assets move.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

ACKNOWLEDGED

Recovery buffers the full missed chain

CATEGORY	DoS
LOCATION	packages/protocol/lz-ver-protocol/ver-api-common/src/services/state-recoverer.ts:140-164,180-225, packages/protocol/lz-ver-protocol/ver-api-common/src/chain/canton/canton-chain-client.ts:381-399
REVIEW	Remediation Review 4

DESCRIPTION

A validator is expected to recover missed history within the same version even when its local database is empty or stale. It walks from the latest on ledger `StateCommitment` back to the last locally persisted commitment or to that version's genesis, then replays the missing commitments in order. Restoring a recent database snapshot is an optional way to shorten this process. A hard fork has a different requirement because the database must retain the trie nodes inherited from the earlier version.

`StateRecoverer.recover` implements same version catch up by calling `#collectMissedCommitments` before replay begins. The collector follows every predecessor and adds it to `pastCommitments`. It only reverses the array and starts forward replay after reaching the local recovery point. The complete array stays live throughout replay.

Each backward step is also serial. `getStateCommitmentById` resolves the historic create event and fetches its transaction by offset before the recoverer checks the local database for the predecessor. Recovery memory and collection time therefore grow linearly with the entire missed chain. Source faithful profiling with Canton shaped identifiers retained about 395 MB for 100,000 commitments with five request IDs each and about 3.95 GB for 1,000,000 commitments before the first request was replayed. The larger case used about 91 percent of the default V8 heap available in the validator's Node 22 runtime, excluding Ledger API responses, database work, Merkle updates and normal server activity.

A validator still replaying history cannot contribute a write signature for the current on ledger state root. It may continue serving reads from an older persisted commitment once that commitment satisfies the finality delay, while an empty database cannot serve reads. If enough validators recover together to fall below write quorum, new batches wait until enough nodes catch up.

The operating model does not define a maximum missed commitment backlog, recovery time objective or production memory limit. Operators that require bounded catch up are expected to restore a recent snapshot and avoid restarting a quorum sized validator set while the head is advancing. This makes the behavior an operational design risk rather than a confirmed availability vulnerability. However, retaining the entire chain means that a sufficiently

I-01 Recovery buffers the full missed chain

CONTINUED

stale same version validator can consume most of its heap before replay starts, which makes successful recovery depend on the available memory and missed history.

— RECOMMENDATION

Be aware of this behaviour. Consider processing the missing lineage in bounded chunks and persist collection progress before replaying each chunk so memory depends on the configured chunk size rather than the total backlog. Expose recovery through readiness so a replaying validator is not reported as ready. Until recovery is bounded, document the memory sizing, snapshot and quorum restart requirements.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Uppercase peers block inbound messages

CATEGORY	Validation
LOCATION	contracts/protocol/canton/contracts/Layerzero/0ftCommon/damL/0App/LzReceive/Call/Validate.daml:66
REVIEW	Remediation Review 4

DESCRIPTION

`OpSetPeer` is intended to register the remote bytes32 address that may send messages to an OApp. The OneSig dispatcher forwards the signed `peer` text without changing its representation. Both production config templates validate the value with `isBytes32Hex`, which accepts uppercase hexadecimal characters, then their `setPeerImpl` branches store `arg.peer` verbatim.

The inbound path also accepts either hexadecimal case as valid bytes32 syntax. However, it checks the registered peer with the case sensitive expression `peer == origin.sender`. The VER address codec and the official Canton peer derivation helper emit lowercase hexadecimal addresses. An uppercase peer therefore denotes the same 32 bytes as the incoming sender but does not compare equal to it.

A signer quorum can approve an uppercase peer and the OneSig batch succeeds, advances the nonce and installs that value. Every normal inbound message from the intended remote OApp then fails with `ERR_LZ_RECEIVE_PEER_MISMATCH` before request creation. A source transfer may already have locked or burned assets, so delivery remains unavailable until another authorized config update stores the lowercase representation.

RECOMMENDATION

Convert peer addresses to one 64 character lowercase representation before storing them in both production config implementations. Apply the same representation to `origin.sender` before comparison and enforce it when configs are created so existing construction paths cannot introduce a case mismatch.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.